Année universitaire 2026–2027

Risk Analytics & Scoring de Crédit

Un système backend SQL d'entreprise

PostgreSQL · Machine Learning · MLOps

Cours complet sur un cas réel de 32 409 prêts, exercices guidés et corrigés — pour préparer tes entretiens de stage en Data Science / IA



Mohamed Fofana
M1 Mathématiques Appliquées, Statistique — Université Clermont Auvergne

Avant-propos

Ce cours t'accompagne dans la construction d'**un seul projet SQL, de niveau entreprise** : le système de base de données qui, dans une banque, gère le risque de crédit. On y suit chaque prêt de sa demande jusqu'à son dernier remboursement ou jusqu'à son défaut, on y range les prédictions des modèles de Machine Learning, et on y calcule chaque mois l'argent que la banque doit mettre de côté pour couvrir ses pertes.

Il a un double objectif. D'abord te faire **comprendre** : le métier du risque de crédit, les notions SQL avancées et la raison d'être de chaque choix technique. Ensuite te permettre de **tout coder toi-même**, de A à Z, avec une feuille de route précise et des résultats attendus pour vérifier ton travail à chaque étape.

Philosophie pédagogique

Comme dans un cours de Machine Learning bien construit, chaque notion est présentée selon une progression en **cinq étapes** :

1. **Mise en situation d'abord** : un problème concret de l'équipe Risque, avant toute théorie.
2. **Intuition et analogies** : l'idée expliquée avec des images de la vie quotidienne.
3. **Notions détaillées** : la syntaxe, les règles, les formules dérivées pas à pas, et les **pièges** que les recruteurs adorent tester.
4. **Exercices guidés sur ta base** : du vrai SQL, exécuté sur tes données, corrigé ligne à ligne.
5. **Exercices à faire seul** : énoncés et indices dans le chapitre, corrigés à la fin du cours.

Chaque chapitre se termine par une **synthèse pour les entretiens** : ce qu'il faut retenir et les phrases que tu pourras dire.

Le fil rouge : ta base de données

Tout au long du cours, on travaille sur **la même base**, `risk_analytics_db`, construite à partir des données réelles de ton projet de scoring de crédit :

— **32 409 prêts réels** (jeu de données *Credit Risk Dataset*, après nettoyage), dont **7 088 ont fait défaut** (21,9 %) ;

— les **prédictions réelles de tes trois modèles** R : la grille de score, la régression logistique et le XGBoost monotone (le « champion »), avec l'explication SHAP de chaque décision ;

— **1 107 936 mensualités** et leurs paiements, simulés de façon cohérente avec le défaut réel : un prêt qui a fait défaut cesse de payer, un prêt sain rembourse.

Quatre personnages reviennent souvent, parce qu'ils illustrent chacun une situation typique :

Awa Koné (prêt 32582)	nouvelle cliente : 12 000 € sur 36 mois pour ses études. Tu créeras son dossier au chapitre 1.
Prêt 67	28 000 € de travaux, en retard de **3 jours** au 30/09/2026.
Prêt 61	30 000 € pour une création d'entreprise, en retard de **46 jours**.
Prêt 9	35 000 € de prêt personnel, **388 jours** de retard : il est en défaut.

Une date de référence figée

Dans une vraie banque, « aujourd'hui » est la date du jour. Ici, la base contient un paramètre `date_reference` fixé au **30 septembre 2026**. Tes résultats seront donc exactement ceux de ce cours, quel que soit le jour où tu les calcules.

Comment ce cours suit ton plan

Le cours reprend exactement le plan que tu as demandé :

Partie 1	Description du projet : titre, contexte et enjeux métier, cas d'usage.
Partie 2	Problématique et objectifs : les quatre problèmes métier, les objectifs techniques.
Partie 3	Architecture de la base : modèles conceptuel et relationnel, dictionnaire de données, index, code DDL complet.
Partie 4	Modules de cours : **Chapitre 1** Modélisation, DDL et contraintes ; **Chapitre 2** SQL analytique avancé ; **Chapitre 3** Triggers et procédures stockées. Chacun avec cours, exercices guidés, exercices seuls et synthèse.
Partie 5	Feuille de route : les tâches 1 à 6, à cocher au fur et à mesure.
À la fin	Corrigés des exercices à faire seul, aide-mémoire SQL, glossaire métier et questions d'entretien.

Le dossier du projet

`docs/`	ce cours (PDF et sources LaTeX).
`donnees/`	les données sources : `credit_risk_dataset.csv` (les prêts), `predictions_modeles.csv` et `shap_xgboost.csv` (sorties de tes modèles R), `echantillon.csv` (découpage apprentissage / test).
`sql/01_schema.sql`	le schéma de la base (fourni : c'est le code DDL de la partie 3).
`sql/`	**ton dossier de travail** : tu y écris tes propres scripts, chapitre par chapitre.
`python/charger_donnees.py`	le script d'alimentation (Faker, Pandas, SQLAlchemy), présenté à la tâche 3.
`corriges/`	les corrigés complets des trois chapitres, exécutables tels quels.

Prérequis et méthode

— **SQL de base** : `SELECT`, `WHERE`, `JOIN`, `GROUP BY`. Tout le reste est expliqué.

— **Python de base** et quelques notions de probabilités (une probabilité de défaut, une AUC) : tu les as déjà vues en master.

— **Tape le code toi-même**, ne le copie pas : c'est en écrivant qu'on retient. Cherche chaque exercice seul au moins 20 minutes avant d'ouvrir le corrigé.

— **Compare tes résultats** à ceux du cours. Un écart n'est pas un échec : c'est la meilleure occasion de comprendre quelque chose.

Les encadrés du cours

Imagine cette situation...

Une situation concrète pour entrer dans le sujet.

Définition — Notion

Une définition précise, à connaître par cœur.

Piège

Une erreur fréquente, souvent testée en entretien.

Exercice 1.1 — Titre

Un exercice : guidé (corrigé juste après) ou à faire seul (corrigé à la fin).

Ce que tu sauras dire en entretien

Les phrases à savoir dire à un recruteur.

Table des matières

Avant-propos **1**

1 Description du projet **6**
1.1 Le titre du projet . . . 6
1.2 Contexte et enjeux métier . . . 7
1.3 Cas d'usage concret : la vie d'un prêt . . . 10

2 Problématique et objectifs **13**
2.1 Les quatre problèmes métier à résoudre . . . 13
2.2 Les objectifs techniques . . . 17

3 Architecture de la base et DDL **20**
3.1 Du besoin au modèle conceptuel . . . 20
3.2 Les choix de conception, et pourquoi . . . 21
3.3 Dictionnaire de données . . . 23
3.4 Les index B-Tree . . . 26
3.5 Le code DDL complet . . . 26

Partie 4 — Modules de cours **30**

1 Modélisation, DDL et contraintes d'intégrité bancaire **31**
1.1 Cours et notions importantes . . . 31
1.2 Exercices guidés . . . 39
1.3 Exercices à faire seul . . . 42
1.4 Synthèse pour les entretiens . . . 43

2 SQL analytique avancé : CTE et fonctions de fenêtre **45**
2.1 Cours et notions importantes . . . 45
2.2 Exercices guidés . . . 52
2.3 Exercices à faire seul . . . 57
2.4 Synthèse pour les entretiens . . . 59

3 Automatisation métier : triggers et procédures stockées **60**
3.1 Cours et notions importantes . . . 60
3.2 Exercices guidés . . . 65
3.3 Exercices à faire seul . . . 71
3.4 Synthèse pour les entretiens . . . 73

5 Feuille de route de réalisation **74**
5.1 Tâche 1 — Configurer l'environnement (PostgreSQL et DBeaver) 74
5.2 Tâche 2 — Exécuter le script DDL 75
5.3 Tâche 3 — Générer et injecter les données (Python) 76
5.4 Tâche 4 — Rédiger les requêtes analytiques métier 82
5.5 Tâche 5 — Implémenter la détection automatique du défaut 83
5.6 Tâche 6 — Publier sur GitHub et valoriser le projet 83

Corrigés des exercices à faire seul **86**
Chapitre 1 86
Chapitre 2 88
Chapitre 3 93

A Aide-mémoire PostgreSQL **97**
A.1 Définir les données 97
A.2 Transactions 97
A.3 Interroger 97
A.4 Fonctions de fenêtre 97
A.5 PL/pgSQL 98
A.6 Commandes psql et diagnostic 98

B Glossaire du risque de crédit **99**

C Questions d'entretien **101**
C.1 SQL 101
C.2 Modélisation et ingénierie 101
C.3 Risque de crédit 102
C.4 Machine Learning et MLOps 102

Partie 1

Description du projet

> *« Une banque ne gagne pas d'argent en prêtant : elle en gagne quand on la rembourse. »*

Avant d'écrire la moindre ligne de SQL, il faut comprendre **à quoi sert** le système que tu vas construire, **pour qui**, et **pourquoi** il doit être aussi rigoureux. Cette partie pose le décor : le métier du risque de crédit, le rôle de la base de données dans une banque, et la place des modèles de Machine Learning.

1.1 Le titre du projet

Mise en situation — Ton premier jour dans l'équipe Risque

Tu commences ton stage dans une banque en ligne qui accorde des prêts à la consommation. Aujourd'hui, tout est éparpillé : les dossiers de prêt dans un tableur, les paiements dans un export comptable, et les scores du modèle de Machine Learning dans le notebook d'une data scientist.

Ta responsable te dit : *« Le régulateur nous demande de justifier chaque provision, le comité des risques veut savoir chaque matin combien de prêts sont en retard, et personne ne sait quelle version du modèle a décidé de quel prêt. Il nous faut une vraie base de données. »*

Ta mission : construire cette base, de la première table jusqu'à la détection automatique du défaut.

Le titre officiel de ton projet est :

Un système backend SQL d'entreprise orienté Risk Analytics, Scoring de Crédit et MLOps (Fintech / Banque)

Chaque mot de ce titre correspond à une exigence précise :

Système backend SQL	la base PostgreSQL et toute sa logique : tables, contraintes, vues, fonctions, triggers, procédures. Pas d'interface graphique : c'est le moteur que d'autres applications utilisent.
d'entreprise	des données qui ne peuvent pas être fausses (contraintes), des opérations « tout ou rien » (transactions), une trace de chaque changement (audit), des droits d'accès (sécurité) et des temps de réponse corrects sur des millions de lignes (index).
Risk Analytics	mesurer le risque du portefeuille : retards, prêts en défaut, pertes attendues, provisions.
Scoring de crédit	prédire, avant d'accorder un prêt, la probabilité que l'emprunteur ne rembourse pas.
MLOps	faire vivre les modèles en production : savoir quelle version a décidé quoi, conserver chaque prédiction, comparer le modèle en place à ses concurrents, détecter qu'il se dégrade.
Fintech / Banque	un secteur réglementé : chaque chiffre doit pouvoir être justifié devant un auditeur ou un superviseur.

1.2 Contexte et enjeux métier

1.2.1 Le métier : prêter de l'argent qu'on n'a pas encore récupéré

Imagine cette situation...

Tu prêtes 100 € à chacun de tes 10 amis. Tu sais d'expérience qu'en moyenne **un sur dix** ne te rembourse pas. Et quand quelqu'un ne rembourse pas, tu finis quand même par récupérer un peu plus de la moitié de la somme : tu perds **45 %** de ce qu'il te doit.

Combien t'attends-tu à perdre ? Sur 10 amis, environ 1 ne rembourse pas. Il te devait 100 €, tu en perds 45 %. Ta perte attendue est donc d'environ $10 \times 10\ \% \times 45\ \% \times 100\ € = 45\ €$.

Tu viens de faire, sans le savoir, le calcul central du risque de crédit.

Une banque fait exactement la même chose, mais avec des milliers de clients, des prêts qui durent des années et une obligation légale de mesurer ce risque correctement. Trois grandeurs suffisent à le décrire.

Définition — Les trois paramètres du risque de crédit

- **PD** (*Probability of Default*, probabilité de défaut) : la probabilité que l'emprunteur fasse défaut. C'est ce que prédit ton modèle de scoring.
- **LGD** (*Loss Given Default*, perte en cas de défaut) : la part de la somme due que la banque perd réellement si le client fait défaut, après recouvrement. Dans ce projet : **45 %**, la valeur réglementaire de référence pour une créance non garantie.
- **EAD** (*Exposure At Default*, exposition au moment du défaut) : la somme que le client doit encore au moment où il fait défaut. Pour un prêt amortissable, c'est le **capital restant dû** : il baisse à chaque mensualité payée.

Leur produit donne la **perte attendue** (*Expected Loss*) :

$$\boxed{EL = PD \times LGD \times EAD}$$

Calcul à la main — la perte attendue d'un prêt

Un client emprunte 10 000 €. Ton modèle estime sa probabilité de défaut à 5 %. La LGD vaut 45 %.

1. EAD : au départ, le client doit tout le capital, soit **10 000 €**.
2. Perte si défaut : $45\ \% \times 10\,000 = 4\,500$ €.
3. Perte attendue : $5\ \% \times 4\,500 =$ **225** €.

Ces 225 € ne sont pas une perte certaine : c'est la perte **moyenne** sur un grand nombre de prêts semblables. La banque doit la couvrir par le taux d'intérêt qu'elle facture et par des provisions.

1.2.2 Les trois questions de l'équipe Risque

Tout le travail d'une équipe Risque de crédit répond à trois questions, posées à trois moments de la vie d'un prêt. Ton projet apporte une réponse SQL à chacune.

Question	Moment	Réponse dans ton projet
Qui *va* faire défaut ?	avant l'octroi	le **score** du modèle (PD), stocké dans `predictions_scoring_ml` avec son explication SHAP.
Qui *est en train* de faire défaut ?	pendant la vie du prêt	le **retard de paiement** (DPD), calculé à partir de `echeancier_remboursements`.
Combien allons-nous perdre ?	chaque fin de mois	l'**exposition** (EAD) et les **provisions** (ECL), calculées et historisées dans `provisions_mensuelles`.

1.2.3 Le cadre réglementaire, en deux idées

Tu n'as pas besoin d'être juriste, mais deux textes structurent tout le projet. Les citer correctement en entretien fait une vraie différence.

Définition — Le défaut (règlement européen CRR, article 178)

Un emprunteur est **en défaut** lorsqu'il a un retard de paiement de **plus de 90 jours** sur une obligation significative, ou lorsque la banque estime qu'il ne paiera probablement pas (*unlikeliness to pay*). C'est la règle que ton trigger appliquera automatiquement.

Définition — Les provisions IFRS 9

Depuis 2018, la norme comptable IFRS 9 oblige les banques à provisionner les **pertes attendues** (*Expected Credit Loss*, ECL) **dès l'octroi** du prêt, sans attendre l'incident. Chaque prêt est classé dans l'un de trois **stages** :

- **Stage 1** : prêt sain. On provisionne la perte attendue sur les **12 prochains mois**.
- **Stage 2** : risque significativement dégradé, présumé à partir de **plus de 30 jours** de retard. On provisionne la perte attendue sur **toute la durée restante**.
- **Stage 3** : prêt en défaut. La perte est considérée comme avérée : $PD = 100\ \%$.

Pourquoi c'est un sujet SQL

Le superviseur et les auditeurs peuvent demander, des mois plus tard : *« Pourquoi avez-vous provisionné 57,21 € sur le prêt 61 au 30 septembre ? »*. Il faut alors retrouver la donnée utilisée, la règle appliquée et la date du calcul. Un tableur modifié à la main en est incapable. Une base de données bien conçue, avec ses contraintes, son historique et ses calculs rejouables, le permet.

1.2.4 Où intervient le Machine Learning

Ton projet de scoring a produit trois modèles, entraînés en R sur l'historique des prêts : une **grille de score** (des points par caractéristique), une **régression logistique**, et un **XGBoost monotone**. Sur l'échantillon de test, le XGBoost atteint une AUC de **0,923**, contre 0,876 pour la logistique : c'est lui qui décide en production. On l'appelle le **modèle champion** ; les deux autres sont des **challengers**.

Définition — SHAP

Les valeurs SHAP (*SHapley Additive exPlanations*) décomposent une prédiction en **contributions de chaque variable**. Pour un prêt donné, une valeur positive pousse le risque vers le haut, une valeur négative le fait baisser. Ta base conserve, pour chaque prédiction du champion, la variable qui a le plus pesé et sa contribution.

Par exemple, pour le prêt 61, le XGBoost prédit une PD de 1,07 % et la variable la plus influente est la `note_risque`, avec une contribution de −1,29 : sa bonne note (A) a fortement fait baisser le risque estimé. Cette explication est indispensable : le RGPD encadre les décisions automatisées, et les orientations de l'Autorité bancaire européenne sur l'octroi de prêts exigent des modèles explicables. Une banque doit pouvoir dire à un client *pourquoi* son prêt a été refusé.

La base de données est le point de rencontre entre le monde des modèles et celui du métier. La figure 1.1 montre ce cycle, que l'on appelle souvent la **boucle MLOps**.

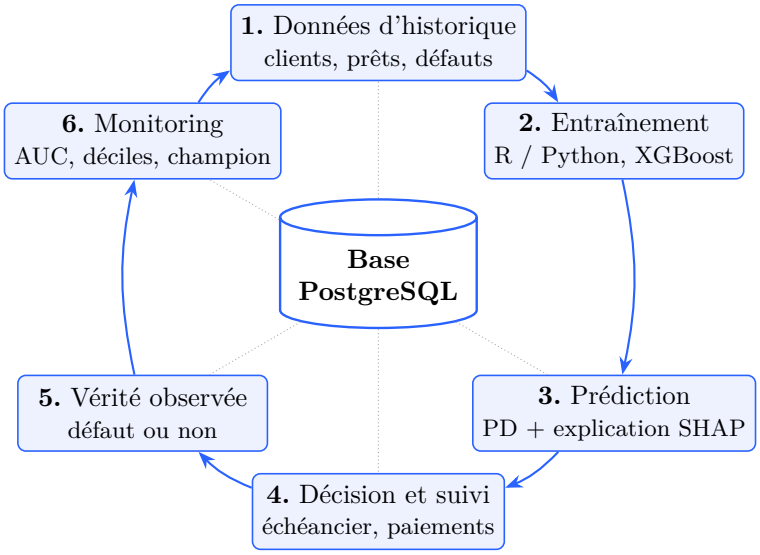


FIGURE 1.1 – La boucle MLOps : la base fournit les données d'entraînement, conserve chaque prédiction, enregistre ce qui se passe réellement, puis permet de mesurer la performance du modèle.

Analogie — le carnet de santé

Pense au prêt comme à un patient. Le **modèle de scoring** est le médecin qui fait un pronostic à l'entrée. La **base de données** est le carnet de santé : on y note le pronostic, qui l'a fait et quand, puis chaque examen (chaque mensualité). Le **suivi du retard** est la prise de température régulière, et le **trigger** est l'alarme qui se déclenche quand la fièvre dépasse un seuil. Sans carnet, impossible de savoir, des années plus tard, si le médecin avait vu juste.

1.2.5 Pourquoi ce projet existe en entreprise

Besoin	Avec des fichiers et un notebook	Avec ton système SQL
Données fiables	un âge de 15 ans ou un paiement en double passent inaperçus	les contraintes les refusent à l'entrée
Source unique	chacun a « sa » version du fichier	une seule base, partagée
Traçabilité	on ne sait plus quel modèle a décidé	chaque prédiction est conservée avec sa version
Détection du défaut	quelqu'un doit y penser	un trigger et un traitement de nuit l'appliquent
Provisions	recalculées à la main, non reproductibles	une procédure rejouable, historisée chaque mois
Volume	le tableur sature vers un million de lignes	1,1 million de mensualités interrogées en quelques millisecondes grâce aux index
Sécurité	le fichier circule par e-mail	des rôles : l'analyste lit, l'application encaisse

1.3 Cas d'usage concret : la vie d'un prêt

Le dossier d'Awa Koné

Awa Koné, 29 ans, locataire, 42 000 € de revenus annuels, demande **12 000 € sur 36 mois** au taux de 9,5 % pour financer ses études. Suivons son prêt de la demande jusqu'au défaut éventuel, et regardons ce que fait la base à chaque étape.

1. **La demande (20/09/2026).** Une fiche client et un dossier de prêt sont créés. Les contraintes vérifient tout de suite que l'âge est plausible, que le montant est positif, que le motif fait partie de la liste autorisée.

2. **Le scoring (24/09/2026).** Le XGBoost estime sa probabilité de défaut à **11,8 %**. Le seuil de décision du modèle est 25,5 % : le prêt est **accordé**. L'explication SHAP indique que la variable la plus influente est le **taux d'effort** (le montant emprunté rapporté au revenu, ici $12\,000/42\,000 \approx 29$ %), qui augmente le risque. La prédiction est enregistrée avec la version du modèle, le seuil et la date.

3. **L'octroi et l'échéancier (25/09/2026).** La base génère **36 mensualités de 384,40 €** à partir du 25/10/2026. Le coût total du crédit est de 1 838,21 € d'intérêts.

4. **Le suivi.** Chaque paiement est enregistré par une procédure qui l'affecte à la mensualité la plus ancienne restant due. Le retard (DPD) se déduit des dates : il n'est jamais saisi à la main.

5. **Le retard.** Si la mensualité du 25/10/2026 n'est pas payée, Awa est **en retard** dès le lendemain. Au-delà de 30 jours, son prêt passe en **stage 2** et la provision augmente.

6. **Le défaut.** Toujours sans paiement, au 31/01/2027 son retard atteint **98 jours** : plus de 90 jours, le prêt passe **en défaut**. Le changement est inscrit dans un journal d'audit, une alerte est envoyée, et la provision est calculée avec une PD de 100 %.

7. **La fin.** Si tout est remboursé, le prêt est **soldé**. S'il est en défaut, il ne redevient pas sain automatiquement : une période probatoire et une décision humaine sont nécessaires.

Ces changements d'état forment le cycle de vie du prêt (figure 1.2). Tu l'implémenteras au chapitre 3.

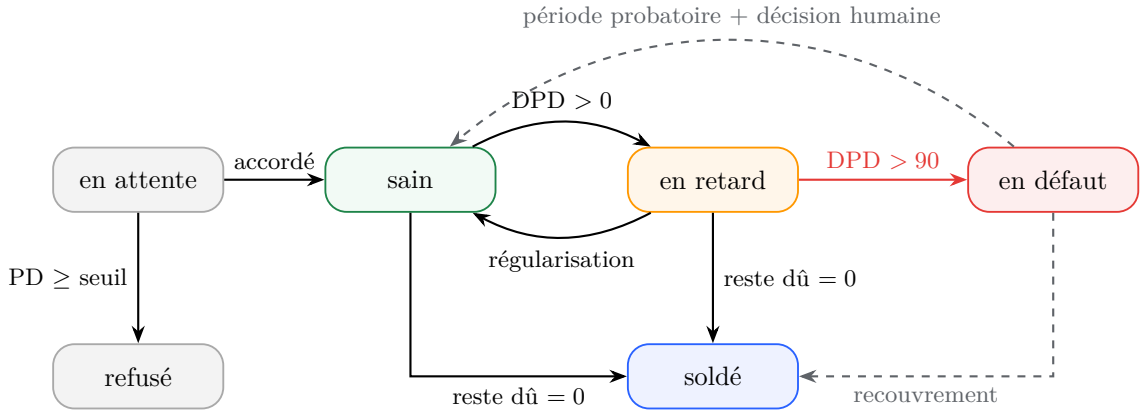


FIGURE 1.2 – Le cycle de vie d'un prêt. Les flèches pleines sont automatiques (trigger ou traitement de nuit) ; les flèches en pointillés demandent une décision humaine.

Le tableau suivant relie chaque étape aux tables et aux mécanismes SQL que tu vas construire.

Étape	Tables	Mécanisme SQL	Où
Demande	`clients`, `demandes_pret`	contraintes `CHECK`, clés, transaction	Ch. 1
Scoring	`predictions_scoring_ml`	registre versionné, contrainte de cohérence	Ch. 1
Échéancier	`echeancier_remboursements`	`generate_series`, CTE, formule d'annuité	Ch. 1
Suivi du retard	vue `v_situation_prets`	agrégats, `FILTER`, fonctions de dates	Ch. 2
Provisions	`provisions_mensuelles`	CTE, `ROLLUP`, procédure rejouable	Ch. 2 et 3
Paiement	`echeancier_remboursements`	procédure stockée, verrou	Ch. 3
Défaut	`demandes_pret`, `journal_statuts`	triggers, traitement de nuit, `pg_notify`	Ch. 3
Monitoring	`predictions_scoring_ml`, `echantillons_modele`	`NTILE`, rangs, AUC en SQL	Ch. 2

Résumé du chapitre

1. Le risque de crédit se résume par $EL = PD \times LGD \times EAD$: probabilité de défaut, perte en cas de défaut, exposition.
2. L'équipe Risque répond à trois questions : qui *va* faire défaut (scoring), qui *est en train* de faire défaut (retards), combien allons-nous perdre (provisions).

3. Deux règles structurent le projet : le **défaut** au-delà de 90 jours de retard (CRR, art. 178) et les **trois stages** de provisionnement IFRS 9 (seuil de 30 jours pour le stage 2).
4. La base de données est le point de rencontre entre les modèles de ML et le métier : elle fournit les données, conserve chaque prédiction et permet de mesurer la performance des modèles.
5. Le cas d'usage suit un prêt de la demande au défaut : chaque étape correspond à une table et à un mécanisme SQL du cours.

Partie 2

Problématique et objectifs

« Un problème bien posé est à moitié résolu. »

Ta responsable t'a donné quatre problèmes à résoudre. Avant de les traduire en SQL, il faut les comprendre parfaitement : ce que mesure chaque indicateur, pourquoi la définition exacte compte, et ce que la banque risque si on se trompe. Cette partie les pose un par un, avec des calculs à la main sur de vrais prêts de ta base.

2.1 Les quatre problèmes métier à résoudre

2.1.1 Problème 1 — Suivre les retards de paiement (DPD)

Imagine cette situation...

Deux clients ont chacun **une** mensualité impayée. Le premier a oublié celle d'hier. Le second n'a pas payé celle d'il y a quatre mois, mais il a payé normalement les trois suivantes.

Sont-ils dans la même situation ? Évidemment pas : le premier a un jour de retard, le second plus de cent. Compter les impayés ne suffit pas : il faut savoir **depuis quand** le client doit de l'argent.

Définition — Days Past Due (DPD), le nombre de jours de retard

À une date d'arrêté t, le DPD d'un prêt est le nombre de jours écoulés depuis **l'échéance la plus ancienne qui est échue et pas entièrement payée**. S'il n'y en a aucune, le DPD vaut 0.

$$DPD(t) = t - \min\{\, d_k \;:\; d_k \leq t \text{ et l'échéance } k \text{ n'est pas soldée à } t \,\}$$

(et $DPD(t) = 0$ si aucune échéance ne remplit ces conditions), où d_k est la date de l'échéance numéro k.

Calcul à la main — le retard du prêt 61 au 30/09/2026

Voici les mensualités du prêt 61 autour de la date d'arrêté (704,41 € par mois) :

N°	Échéance	Montant dû	Payé	Date de paiement	État au 30/09
15	15/05/2026	704,41	704,41	18/05/2026	soldée
16	15/06/2026	704,41	704,41	15/06/2026	soldée
17	15/07/2026	704,41	704,41	03/08/2026	soldée (avec 19 jours de retard)
18	15/08/2026	704,41	0,00	—	**échue, impayée**
19	15/09/2026	704,41	704,41	15/09/2026	soldée
20	15/10/2026	704,41	0,00	—	pas encore échue

1. Échéances échues au 30/09 : les numéros 15 à 19 (la 20e est dans le futur).
2. Parmi elles, non soldées : seulement la **18** (la 19 a été payée le 15/09).
3. La plus ancienne non soldée est donc due depuis le **15/08/2026**.
4. $DPD = 30/09/2026 - 15/08/2026 = 16 + 30 =$ **46 jours**.

Le prêt 61 a 46 jours de retard, bien qu'il ait payé sa dernière mensualité à l'heure.

Piège — Trois fausses définitions du retard

— « **Jours depuis le dernier paiement** » : pour le prêt 61, cela donnerait 15 jours (dernier paiement le 15/09). Faux : l'impayé d'août est masqué par le paiement de septembre.

— « **Nombre d'impayés × 30** » : une mensualité impayée donnerait 30 jours. Faux : les mois n'ont pas tous 30 jours et l'impayé date de 46 jours.

— « **Jours depuis l'impayé le plus récent** » : avec plusieurs impayés, on sous-estimerait gravement le retard. La règle bancaire retient toujours le **plus ancien**.

Pourquoi le DPD n'est pas une colonne de la base

Le retard change **tous les jours**, sans qu'aucun événement ne se produise : un prêt qui a 46 jours de retard aujourd'hui en aura 47 demain. Une colonne `jours_retard` stockée serait donc fausse dès le lendemain, sauf à la recalculer chaque nuit pour tous les prêts. On la **calcule** à partir des dates d'échéance et des paiements, dans une vue. C'est un principe général de conception : *une donnée calculée et stockée finit toujours par devenir incohérente.*

Pour piloter un portefeuille, on regroupe les prêts par **tranche de retard** (*buckets*) : à jour, 1 à 30 jours, 31 à 60, 61 à 90, plus de 90 jours. On en déduit deux indicateurs suivis par tous les comités des risques.

Définition — PAR30 et PAR90 (Portfolio At Risk)

Le PARx est la part de l'encours (le capital restant dû) portée par les prêts qui ont **plus de x jours** de retard :

$$PAR_x = \frac{\text{encours des prêts dont } DPD > x}{\text{encours total}}$$

On le calcule en **montant**, pas en nombre de prêts : un gros prêt en retard pèse plus qu'un petit.

2.1.2 Problème 2 — Calculer l'exposition (EAD) et les provisions

Pour un prêt amortissable, l'**exposition** (EAD) est le capital que le client doit encore. Il diminue à chaque mensualité : une mensualité contient une part de capital et une part d'intérêts, et seule la part de capital réduit la dette. Si une mensualité n'est payée qu'en partie, on considère que le paiement couvre capital et intérêts au prorata.

La provision IFRS 9 d'un prêt est sa perte attendue, avec une probabilité de défaut dont **l'horizon dépend du stage** :

Stage	Condition (dans ce projet)	Horizon de la PD	PD utilisée
1	$DPD \leq 30$	12 prochains mois	PD_{12}
2	$30 < DPD \leq 90$	toute la durée restante	PD_{restante}
3	$DPD > 90$ (défaut)	—	100 %

$$\boxed{ECL = PD_{\text{stage}} \times LGD \times EAD}$$

2.1.2.1 De la PD du modèle à la PD sur 12 mois : la dérivation

Ton modèle prédit une PD sur **toute la durée** du prêt (la variable cible de ton jeu de données indique si le prêt a fait défaut pendant sa vie). Or le stage 1 demande une PD sur 12 mois. Comment passer de l'une à l'autre ?

Analogie — la pièce truquée

Imagine qu'à chaque fin de mois, on lance une pièce truquée qui tombe sur « défaut » avec une petite probabilité h, toujours la même. Le prêt survit s'il évite « défaut » à chaque lancer. Plus le prêt dure, plus il y a de lancers, plus le risque cumulé augmente.

On fait donc l'hypothèse d'un **risque mensuel constant** h. Pour un prêt de n mois :

1. Le prêt ne fait jamais défaut s'il survit aux n mois : $P(\text{survie}) = (1-h)^n$.
2. Cette probabilité vaut $1 - PD$, où PD est la PD du modèle sur la durée totale :

$$(1-h)^n = 1 - PD \quad \Longrightarrow \quad 1 - h = (1-PD)^{1/n}$$

3. La probabilité de faire défaut au cours des m prochains mois est donc :

$$PD_m = 1 - (1-h)^m = 1 - \Big((1-PD)^{1/n}\Big)^m$$

$$\boxed{PD_m = 1 - (1-PD)^{m/n}}$$

Pour le stage 1, $m = \min(\text{mois restants}, 12)$; pour le stage 2, $m =$ mois restants.

Une hypothèse simplificatrice, à assumer en entretien

Les banques utilisent des « structures par terme » de PD estimées sur leur historique : le risque n'est pas constant (il est souvent plus élevé dans les premiers mois). L'hypothèse du risque constant est simple, transparente et suffisante pour ce projet. Savoir dire *pourquoi* c'est une approximation vaut mieux qu'un modèle compliqué mal compris.

Calcul à la main — les provisions de trois prêts au 30/09/2026

LGD = 45 % pour tous.

— **Prêt 67** — 3 jours de retard : **stage 1**. EAD = 16 680,72 €, PD du modèle 0,0973 %, durée 36 mois, 19 mois restants, donc $m = \min(19, 12) = 12$.

$$PD_{12} = 1 - (1 - 0{,}000973)^{12/36} = 0{,}0324\ \%$$

$$ECL = 0{,}000324 \times 0{,}45 \times 16\,680{,}72 = \mathbf{2{,}43}\ €$$

— **Prêt 61** — 46 jours de retard : **stage 2**. EAD = 19 578,01 €, PD 1,0725 %, durée 48

mois, 29 mois restants.

$$PD_{29} = 1 - (1 - 0{,}010725)^{29/48} = 0{,}6494\ \%$$

$$ECL = 0{,}006494 \times 0{,}45 \times 19\,578{,}01 = \mathbf{57{,}21}\ €$$

— **Prêt 9** — 388 jours de retard : **stage 3**, $PD = 100$ %. EAD = 21 712,35 €.

$$ECL = 1 \times 0{,}45 \times 21\,712{,}35 = \mathbf{9\,770{,}56}\ €$$

Le stage pèse bien plus que la note : le prêt 9 avait une bonne note A, mais il est en défaut.

2.1.3 Problème 3 — Historiser les prédictions des modèles

Le scoring ne se fait pas une fois pour toutes. Une banque fait tourner plusieurs modèles, les réentraîne, change leurs seuils. Pour chaque prêt, il faut pouvoir répondre à des questions comme : *quel modèle a décidé ? avec quel seuil ? quand ? pour quelle raison ? les autres modèles auraient-ils décidé autrement ?*

Voici, pour le prêt 61, les trois prédictions enregistrées dans ta base :

Modèle	Champion	PD	Seuil	Décision	Explication principale (SHAP)
`grille_score_v1`	non	3,69 %	24,0 %	accorder	— (623 points)
`logistique_v1`	non	2,66 %	23,5 %	accorder	—
`xgboost_monotone_v2`	**oui**	1,07 %	25,5 %	accorder	`note_risque` : −1,29

Cette table est un **registre** : chaque ligne est une prédiction d'un modèle pour un prêt, et elle ne sera jamais modifiée. Ce registre sert à quatre choses :

1. **L'audit** : retrouver la version, le seuil et la date qui ont conduit à la décision.
2. **L'explication** : répondre à un client refusé, grâce à la variable SHAP principale.
3. **Le monitoring** : comparer, des mois plus tard, la PD prédite au défaut observé, et détecter qu'un modèle se dégrade.
4. **Le champion contre les challengers** : comparer les modèles sur les mêmes prêts avant de changer celui qui décide.

Piège — Écraser le score

La solution naïve consiste à ajouter une colonne `score` dans la table des prêts et à la mettre à jour à chaque nouveau modèle. On perd alors tout l'historique : impossible de savoir quel modèle a accordé le prêt, impossible de comparer deux versions. Une prédiction est un **fait daté** : on l'insère, on ne la met jamais à jour.

Ta base conserve aussi le **découpage apprentissage / test** de ton projet R (table `echantillons_modele`). Mesurer la performance d'un modèle sur les prêts qui ont servi à l'entraîner donnerait des résultats trop beaux : on l'évaluera uniquement sur les 6 482 prêts de test.

2.1.4 Problème 4 — Automatiser les alertes et le passage en défaut

Avec 32 409 prêts, personne ne peut vérifier chaque matin lesquels ont franchi 90 jours de retard. Or un défaut détecté en retard fausse les provisions, les rapports au superviseur et les décisions de recouvrement. Il faut donc que la base applique la règle **elle-même**.

Deux situations très différentes doivent être traitées :

— **Un événement se produit** : un paiement arrive. Le statut du prêt doit être recalculé immédiatement : un client qui régularise redevient « sain ». C'est le rôle d'un **trigger**, une fonction que la base exécute automatiquement quand une ligne change.

— **Rien ne se passe** : le client ne paie plus, et les jours défilent. Le retard franchit 90 jours *sans qu'aucune ligne ne change*. Aucun trigger ne peut se déclencher : il faut un **traitement planifié** (le « batch de nuit ») qui recalcule les statuts de tout le portefeuille.

Règle d'or de l'automatisation

Un trigger réagit à un **événement** (une insertion, une modification, une suppression). Le **passage du temps n'est pas un événement**. Toute règle qui dépend de la date du jour demande un traitement planifié.

Chaque changement de statut doit enfin laisser une trace : l'ancien statut, le nouveau, le retard à ce moment, la date et l'utilisateur. C'est le **journal d'audit**. Et un passage en défaut doit **alerter** le service de recouvrement : la base enverra une notification que toute application abonnée recevra.

Pas de sortie automatique du défaut

Un prêt en défaut qui régularise ne redevient pas sain immédiatement. Les orientations européennes sur la définition du défaut imposent une **période probatoire** (au moins trois mois) avant d'en sortir. Ton système appliquera donc une règle asymétrique : l'entrée en défaut est automatique, la sortie demande une décision humaine.

2.2 Les objectifs techniques

Les quatre problèmes métier se traduisent en quatre objectifs techniques, qui structurent la suite du cours.

Objectif	Concrètement	Notions SQL	Où
Concevoir le schéma relationnel	8 tables reliées, des données qui ne peuvent pas être incohérentes	types, clés, `CHECK`, `UNIQUE`, index, transactions	Partie 3, Chapitre 1
Ingérer des volumes	1,3 million de lignes chargées en une quinzaine de secondes, en une seule transaction	`COPY`, séquences, `ANALYZE`	Tâche 3
Écrire du SQL analytique performant	retards, PAR, provisions, déciles, AUC, matrices de transition, générations	vues, CTE, `FILTER`, fonctions de fenêtre, `ROLLUP`	Chapitre 2
Automatiser la logique métier	recalcul des statuts, journal, alertes, encaissement, clôture mensuelle	fonctions PL/pgSQL, triggers, procédures, rôles	Chapitre 3

2.2.1 L'architecture d'ensemble

La figure 2.1 montre comment les pièces s'assemblent. Les données sources et les sorties de tes modèles R sont chargées par un script Python. La base contient les données *et* la logique métier. Plusieurs utilisateurs s'y connectent, chacun avec des droits différents.

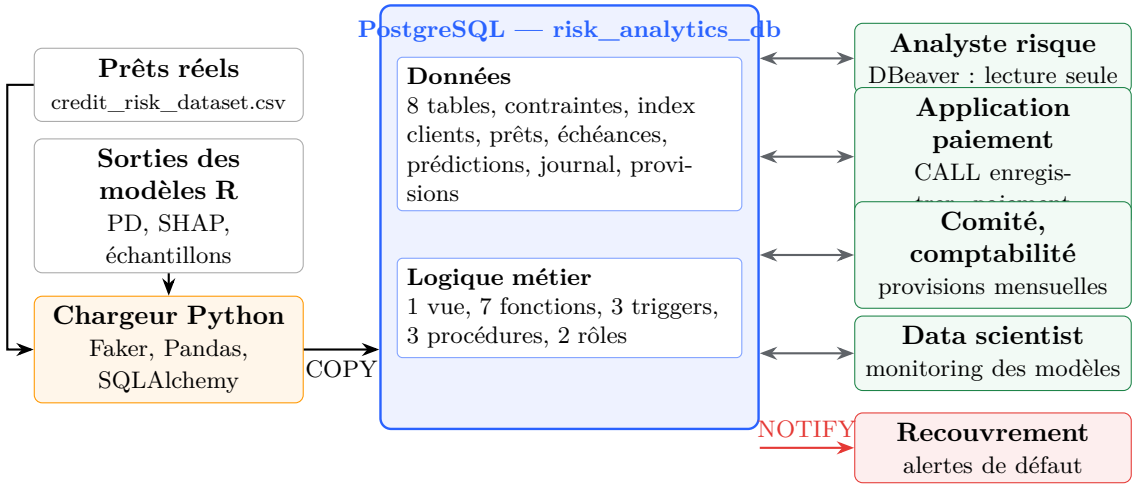


FIGURE 2.1 – L'architecture du système : des sources au pilotage du risque.

2.2.2 Les critères de réussite

Comment sauras-tu que ton projet fonctionne ? Grâce à des **chiffres de contrôle**. Si tu retrouves les valeurs ci-dessous, ton système est correct.

Contrôle	Valeur attendue	Où
Prêts, mensualités et prédictions chargés	32 409 / 1 107 936 / 97 227	Tâche 3
Prêts par statut après le premier traitement	7 088 en défaut, 933 en retard, 13 480 sains, 10 909 soldés	Chapitre 3
Défauts détectés par ta logique *contre* défauts réels	7 088 contre 7 088, **0 désaccord**	Chapitre 3
PAR30 et PAR90	41,90 % et 41,04 %	Chapitre 2
Provisions totales au 30/09/2026	environ 25,63 millions d'euros (19,13 % de l'EAD)	Chapitres 2 et 3
AUC et Gini du champion sur l'échantillon de test	0,923 et 0,847	Chapitre 2
Retard d'un prêt, calculé à la demande	en moins d'une milliseconde	Chapitre 1

Résumé du chapitre

1. Le **DPD** se mesure depuis l'échéance impayée **la plus ancienne** ; il se calcule, il ne se stocke pas. Les tranches de retard donnent le PAR30 et le PAR90, en montant.
2. L'**EAD** est le capital restant dû. La provision vaut $PD \times LGD \times EAD$, avec une PD à 12 mois (stage 1), sur la durée restante (stage 2) ou égale à 100 % (stage 3) : $PD_m = 1 - (1 - PD)^{m/n}$.
3. Les **prédictions** sont des faits datés : on les insère dans un registre, on ne les modifie jamais. Le registre sert à l'audit, à l'explication, au monitoring et à la comparaison des modèles.

4. Un **trigger** réagit aux événements, un **traitement planifié** gère le passage du temps. L'entrée en défaut est automatique, la sortie est humaine.
5. Quatre objectifs techniques : schéma, ingestion, SQL analytique, automatisation.

Partie 3

Architecture de la base et DDL

« On ne construit pas une maison en commençant par le toit, ni une base en commençant par les requêtes. »

Imagine cette situation...

Un architecte ne commence jamais par poser des briques. Il dessine d'abord le plan : les pièces, les portes qui les relient, ce qui porte le poids de la maison. Une erreur sur le plan coûte un coup de gomme ; la même erreur une fois le mur construit coûte un chantier.

Pour une base de données, c'est pareil. Le **modèle conceptuel** est le croquis, le **modèle relationnel** est le plan coté, et le **code DDL** est la construction.

3.1 Du besoin au modèle conceptuel

On part des phrases du métier et on repère les **entités** (les « choses » dont on garde la trace) et les **associations** entre elles :

— un **client** demande un ou plusieurs **prêts** ; chaque prêt appartient à un seul client ;

— un prêt accordé comporte plusieurs **échéances** ; un prêt refusé n'en a aucune ;

— un prêt est évalué par un ou plusieurs **modèles**, chaque évaluation est une **prédiction** ;

— un prêt appartient à l'échantillon d'apprentissage ou à l'échantillon de test du modèle ;

— chaque changement de statut d'un prêt est **journalisé**, et chaque fin de mois produit une **provision** par prêt.

La figure 3.1 traduit ces phrases en **modèle conceptuel de données** (MCD, notation Merise). Les **cardinalités** se lisent ainsi : « un client demande au minimum 1 et au maximum n prêts ; un prêt est demandé par exactement 1 client ».

3.1.1 Le passage au modèle relationnel

Une seule règle suffit pour passer du croquis au plan :

Règle de passage

Une association « un à plusieurs » devient une **clé étrangère** placée **du côté « plusieurs »**. Un prêt a plusieurs échéances : c'est donc l'échéance qui porte la colonne `pret_id`, et non le prêt qui porterait une liste d'échéances.

On obtient le **modèle relationnel** de la figure 3.2 : chaque entité devient une table, chaque association devient une clé étrangère. On y ajoute une table technique, `parametres_risque`, qui centralise les paramètres métier.

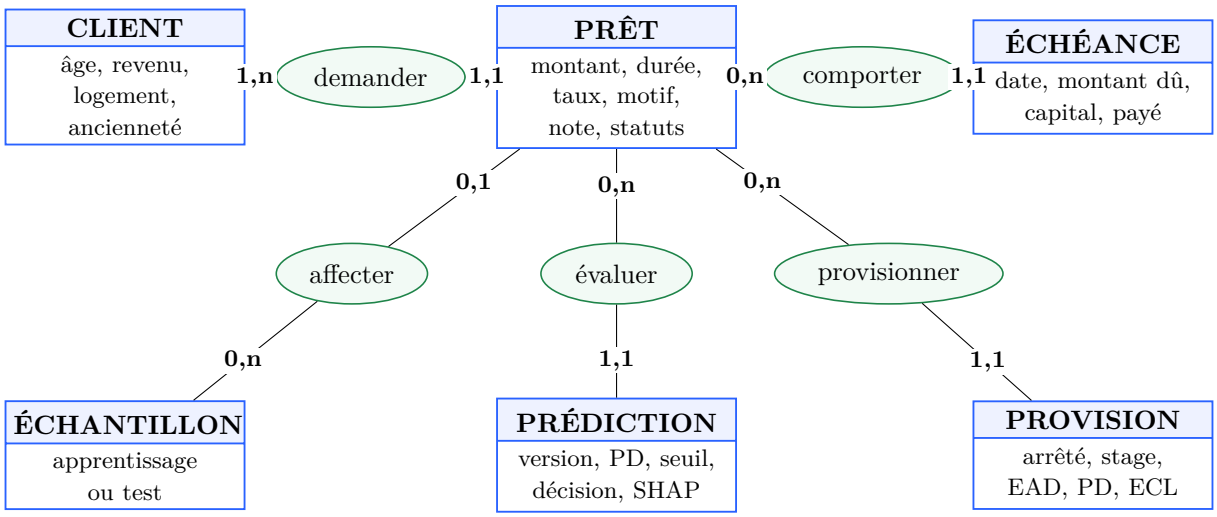


FIGURE 3.1 – Modèle conceptuel de données (simplifié). Le journal des statuts suit le même schéma que les provisions : un prêt en a de 0 à n.

3.2 Les choix de conception, et pourquoi

Un recruteur ne te demandera pas de réciter le schéma. Il te demandera **pourquoi** tu l'as conçu ainsi. Voici les huit décisions qui font de cette base une base « d'entreprise ».

1. NUMERIC pour l'argent, jamais FLOAT

Un `FLOAT` stocke les nombres en binaire et ne représente pas exactement 0,10. Dans PostgreSQL :

```
SELECT 0.1::float8 + 0.2::float8 = 0.3::float8 AS float_egal,     -- false
       0.1::numeric + 0.2::numeric = 0.3       AS numeric_egal,   -- true
       (0.1::float8 + 0.2::float8)::text       AS float_valeur;   -- 0.30000000000000004
```

Sur un million de mensualités, ces erreurs s'accumulent et les totaux comptables ne tombent plus juste. `NUMERIC(12,2)` calcule en décimal exact, au centime.

2. Le retard n'est pas stocké

Le DPD change chaque jour sans événement (partie 2). Il est calculé dans une vue à partir des dates. Même raisonnement pour l'encours : on le déduit des paiements.

3. ON DELETE RESTRICT sur toutes les clés étrangères

Supprimer un client ne doit **jamais** effacer silencieusement ses prêts et leur historique : les banques ont des obligations de conservation. Avec `RESTRICT`, la suppression est refusée tant que des prêts y font référence. `CASCADE` serait dangereux ici.

4. Des identifiants générés par la base

`GENERATED BY DEFAULT AS IDENTITY` pour les clients et les prêts : la base attribue un numéro, mais on peut fournir le sien au chargement (on garde le numéro de ligne du fichier source, ce

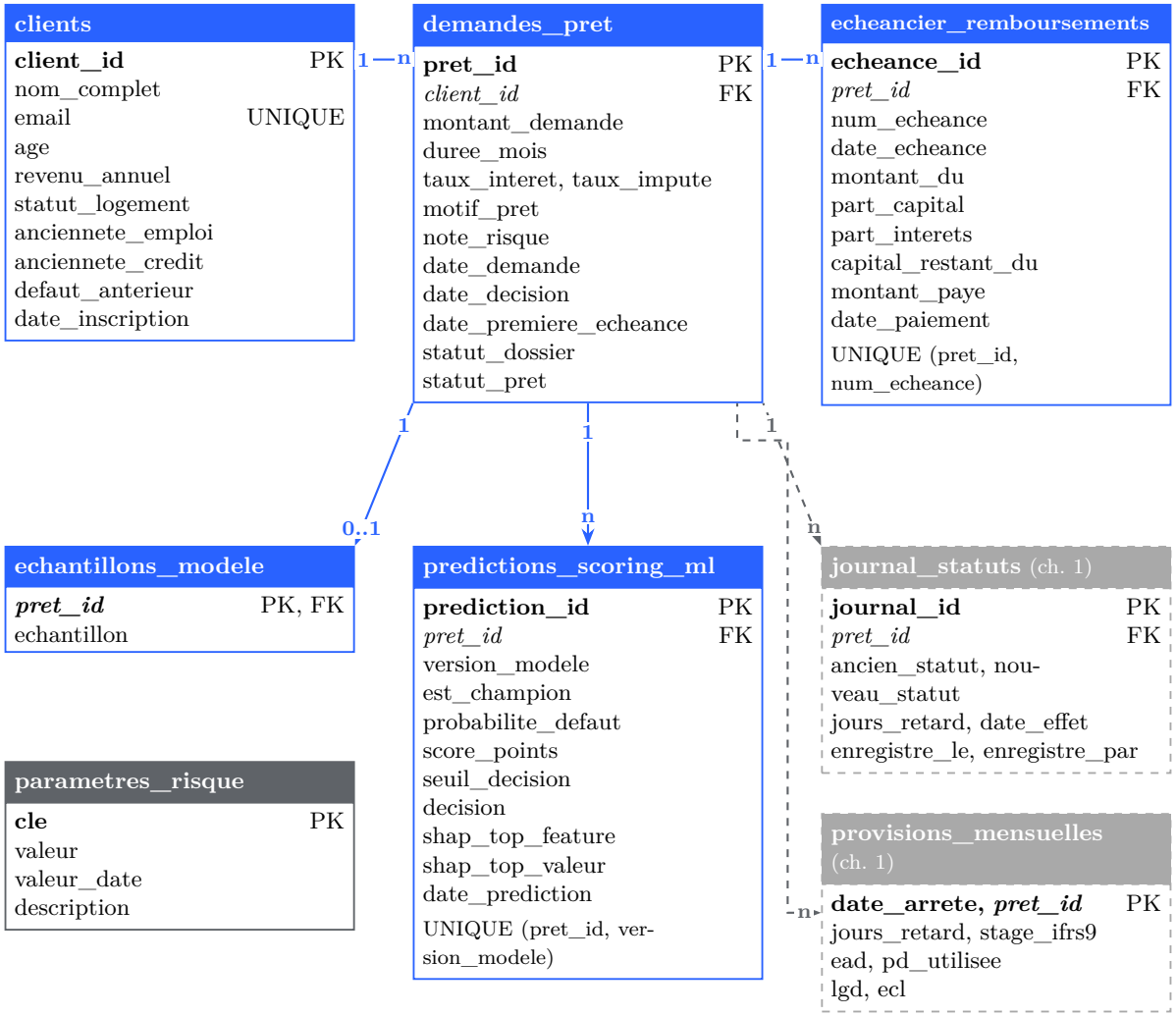


FIGURE 3.2 – Modèle relationnel de `risk_analytics_db`. En gras : clé primaire ; en italique : clé étrangère. Chaque flèche part de la table référencée et pointe vers la table qui porte la clé étrangère (le côté « n »). En pointillés : les deux tables que tu créeras dans l'exercice 1.3, qui référencent toutes deux `demandes_pret`.

qui relie chaque prêt à tes sorties R). `GENERATED ALWAYS` pour les échéances et les prédictions : personne ne doit choisir ces numéros à la main.

5. Une table de paramètres, source unique de vérité

La LGD (45 %), les seuils de 30 et 90 jours et la date de référence sont stockés dans `parametres_risque`, pas écrits en dur dans dix requêtes. Si le seuil change, on modifie une ligne. Une contrainte impose qu'un paramètre ait soit une valeur numérique, soit une date, jamais les deux.

6. Des contraintes entre colonnes

Certaines règles portent sur plusieurs colonnes d'une même ligne. Par exemple, une décision doit être cohérente avec la PD et le seuil : `(decision = 'refuser') = (probabilite_defaut >= seuil_decision)`. L'égalité entre deux conditions exprime une équivalence (« si et seulement si »). On verra cette astuce en détail au chapitre 1.

7. Un registre versionné des prédictions

Plusieurs prédictions par prêt (une par version de modèle), jamais de mise à jour, un drapeau `est_champion` pour le modèle qui décide, le seuil enregistré avec la prédiction.

8. Deux statuts distincts

`statut_dossier` décrit la **décision** (en attente, accordé, refusé) ; `statut_pret` décrit le **remboursement** (sain, en retard, en défaut, soldé). Le second n'a de sens que si le premier vaut « accordé » : une contrainte l'impose.

3.3 Dictionnaire de données

Pour chaque table : les colonnes, leur type PostgreSQL, leurs contraintes et leur signification.

`clients`

Le référentiel des emprunteurs. 32 409 lignes (+ Awa Koné).

Colonne	Type	Contraintes	Description
`client_id`	BIGINT	PK, identité	identifiant de l'emprunteur
`nom_complet`	VARCHAR(100)	NOT NULL	nom fictif (Faker)
`email`	VARCHAR(120)	NOT NULL, UNIQUE	adresse fictive `@exemple.fr`
`age`	SMALLINT	NOT NULL, entre 18 et 100	âge à la demande
`revenu_annuel`	NUMERIC(12,2)	NOT NULL, > 0	revenu annuel en euros
`statut_logement`	VARCHAR(20)	NOT NULL, liste fermée	locataire, proprietaire, credit_immobilier, autre
`anciennete_emploi`	NUMERIC(4,1)	≥ 0, $\leq$ âge $- 14$	années dans l'emploi actuel ; NULL si inconnue (887 clients)
`anciennete_credit`	SMALLINT	NOT NULL, ≥ 0	années d'historique de crédit
`defaut_anterieur`	BOOLEAN	NOT NULL	un défaut a déjà été enregistré par le passé
`date_inscription`	DATE	NOT NULL	entrée dans la banque

demandes_pret

Le dossier de crédit, la décision et le statut de remboursement.

Colonne	Type	Contraintes	Description
pret_id	BIGINT	PK, identité	numéro du prêt (= ligne du fichier source)
client_id	BIGINT	NOT NULL, FK → clients, RESTRICT	l'emprunteur
montant_demande	NUMERIC(12,2)	NOT NULL, > 0	capital emprunté
duree_mois	SMALLINT	NOT NULL, entre 6 et 120	durée du prêt (12 à 60 mois dans les données)
taux_interet	NUMERIC(5,2)	NOT NULL, $0 < t < 40$	taux annuel en %
taux_impute	BOOLEAN	NOT NULL, défaut false	taux absent de la source, remplacé par la médiane (3 094 prêts)
motif_pret	VARCHAR(25)	NOT NULL, liste fermée	etudes, sante, creation_entreprise, personnel, travaux, regroupement_dettes
note_risque	CHAR(1)	NOT NULL, de A à G	note interne (A = meilleur risque)
date_demande	DATE	NOT NULL	dépôt du dossier
date_decision	DATE	$\geq$ date_demande	décision d'octroi
date_premiere_echeance	DATE		première mensualité
statut_dossier	VARCHAR(12)	NOT NULL, défaut en_attente	en_attente, accorde, refuse
statut_pret	VARCHAR(12)	non NULL $\Leftrightarrow$ accordé	sain, en_retard, en_defaut, solde

echeancier_remboursements

Une ligne par mensualité, avec le suivi du paiement. 1 107 936 lignes.

Colonne	Type	Contraintes	Description
echeance_id	BIGINT	PK, identité ALWAYS	identifiant technique
pret_id	BIGINT	NOT NULL, FK → demandes_pret	le prêt
num_echeance	SMALLINT	NOT NULL, > 0 ; UNIQUE avec pret_id	rang de la mensualité (1 à n)
date_echeance	DATE	NOT NULL	date à laquelle la mensualité est due
montant_du	NUMERIC(12,2)	NOT NULL, > 0	mensualité = capital + intérêts
part_capital	NUMERIC(12,2)	NOT NULL, ≥ 0	part qui rembourse la dette
part_interets	NUMERIC(12,2)	NOT NULL, ≥ 0	rémunération de la banque
capital_restant_du	NUMERIC(12,2)	NOT NULL, ≥ 0	dette restante *après* cette échéance
montant_paye	NUMERIC(12,2)	NOT NULL, défaut 0, $\leq$ montant_du	somme déjà versée
date_paiement	DATE	non NULL $\Leftrightarrow$ entièrement payée	date du paiement complet

predictions_scoring_ml

Le registre des prédictions : 3 modèles × 32 409 prêts.

Colonne	Type	Contraintes	Description
prediction_id	BIGINT	PK, identité ALWAYS	identifiant technique
pret_id	BIGINT	NOT NULL, FK	le prêt évalué
version_modele	VARCHAR(40)	NOT NULL ; UNIQUE avec pret_id	grille_score_v1, logistique_v1, xgboost_monotone_v2
est_champion	BOOLEAN	NOT NULL, défaut false	vrai pour le modèle qui décide en production
probabilite_defaut	NUMERIC(7,6)	NOT NULL, entre 0 et 1	PD prédite
score_points	SMALLINT		points de la grille de score (grille uniquement)
seuil_decision	NUMERIC(4,3)	NOT NULL, entre 0 et 1	seuil de refus du modèle
decision	VARCHAR(10)	NOT NULL ; refuser $\Leftrightarrow$ PD $\geq$ seuil	accorder ou refuser
shap_top_feature	VARCHAR(50)		variable SHAP la plus influente (champion uniquement)
shap_top_valeur	NUMERIC(8,4)		sa contribution (log-odds)
date_prediction	TIMESTAMP	NOT NULL	date et heure du calcul

echantillons_modele

Le découpage du projet R : 25 927 prêts d'apprentissage, 6 482 de test.

Colonne	Type	Contraintes	Description
pret_id	BIGINT	PK, FK $\rightarrow$ demandes_pret	le prêt
echantillon	VARCHAR(13)	NOT NULL, apprentissage ou test	échantillon d'appartenance

parametres_risque

Les paramètres métier (4 lignes).

Colonne	Type	Contraintes	Description
cle	VARCHAR(40)	PK	lgd, seuil_defaut_jours, seuil_stage2_jours, date_reference
valeur	NUMERIC	une seule des deux	valeur numérique (0,45 ; 90 ; 30)
valeur_date	DATE	une seule des deux	valeur de type date (30/09/2026)
description	TEXT	NOT NULL	signification et source réglementaire

Les tables journal_statuts et provisions_mensuelles sont spécifiées dans l'exercice 1.3 : tu les concevras toi-même.

3.4 Les index B-Tree

Un index est une structure annexe qui permet de retrouver des lignes sans lire toute la table (on verra son fonctionnement au chapitre 1). PostgreSQL crée automatiquement un index pour chaque clé primaire et chaque contrainte UNIQUE, mais **pas pour les clés étrangères**.

Index	Colonnes	À quoi il sert
Créés automatiquement		
clés primaires (*_pkey)	l'identifiant de chaque table	retrouver une ligne par son identifiant
clients_email_key	email	garantir l'unicité de l'adresse
une_echeance_par_numero	(pret_id, num_echeance)	unicité, et toutes les échéances d'un prêt (33 Mo)
une_prediction_par_version	(pret_id, version_modele)	unicité, et les prédictions d'un prêt
Créés par le script		
idx_prets_client	client_id	jointure client → prêts (clé étrangère)
idx_prets_statut	statut_pret	filtrer un statut rare (933 prêts en retard) ; pour un statut fréquent, lire la table est plus rapide
idx_echeances_impayees	(pret_id, date_echeance) **partiel** : WHERE montant_paye < montant_du	trouver l'impayé le plus ancien d'un prêt : le calcul du retard. 13 Mo seulement, car il ne contient que les 442 459 échéances non soldées

3.5 Le code DDL complet

Le script ci-dessous crée toute la base. Il est fourni dans sql/01_schema.sql. Lis-le en entier en t'appuyant sur le dictionnaire : chaque commentaire justifie un choix. Il commence par supprimer les tables existantes, ce qui permet de le relancer pour repartir de zéro.

```
-- ============================================================================
-- Risk Analytics — schéma de la base (PostgreSQL 15 ou plus)
-- Exécution : psql -d risk_analytics_db -f sql/01_schema.sql   (ou dans DBeaver)
--
-- Choix de conception :
--  * ON DELETE RESTRICT : supprimer un client ne doit JAMAIS effacer silencieusement
--    l'historique de ses prêts (obligation de conservation des données bancaires).
--  * Le retard (DPD) n'est PAS stocké : il se déduit des dates et des paiements.
--    Une donnée calculée et stockée finit toujours par devenir incohérente.
--  * NUMERIC pour l'argent : un FLOAT ne représente pas exactement 0,10 €.
-- ============================================================================
```

```sql
DROP TABLE IF EXISTS provisions_mensuelles, journal_statuts, echantillons_modele,
                     predictions_scoring_ml, echeancier_remboursements, demandes_pret,
                     clients, parametres_risque CASCADE;

-- ============================================================================
-- 0. Paramètres métier : une seule source de vérité, modifiable sans toucher au code
-- ============================================================================
CREATE TABLE parametres_risque (
    cle         VARCHAR(40)  PRIMARY KEY,
    valeur      NUMERIC,
    valeur_date DATE,
    description TEXT         NOT NULL,
    CONSTRAINT une_seule_valeur CHECK ((valeur IS NULL) <> (valeur_date IS NULL))
);
INSERT INTO parametres_risque (cle, valeur, valeur_date, description) VALUES
    ('lgd',               0.45, NULL, 'Perte en cas de défaut (Bâle, approche fondation, créance non garantie)'),
    ('seuil_defaut_jours', 90,   NULL, 'Défaut si le retard dépasse ce nombre de jours (« plus de 90 jours », art.
     ↪ 178 CRR)'),
    ('seuil_stage2_jours', 30,   NULL, 'Stage 2 si le retard dépasse ce nombre de jours (« plus de 30 jours », IFRS 9
     ↪  §5.5.11)'),
    -- En production, on utiliserait CURRENT_DATE ; ici, une date figée rend les résultats reproductibles.
    ('date_reference',     NULL, DATE '2026-09-30', 'Date d''arrêté des calculs (« aujourd''hui » dans la base)');

-- ============================================================================
-- 1. Clients : le référentiel des emprunteurs
-- ============================================================================
CREATE TABLE clients (
    client_id          BIGINT        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nom_complet        VARCHAR(100)  NOT NULL,
    email              VARCHAR(120)  NOT NULL UNIQUE,
    age                SMALLINT      NOT NULL CHECK (age BETWEEN 18 AND 100),
    revenu_annuel      NUMERIC(12,2) NOT NULL CHECK (revenu_annuel > 0),
    statut_logement    VARCHAR(20)   NOT NULL
                       CHECK (statut_logement IN ('locataire', 'proprietaire', 'credit_immobilier', 'autre')),
    anciennete_emploi  NUMERIC(4,1)  CHECK (anciennete_emploi >= 0),   -- NULL = inconnue
    anciennete_credit  SMALLINT      NOT NULL CHECK (anciennete_credit >= 0),
    defaut_anterieur   BOOLEAN       NOT NULL,
    date_inscription   DATE          NOT NULL,
    CONSTRAINT anciennete_coherente CHECK (anciennete_emploi IS NULL OR anciennete_emploi <= age - 14)
);

-- ============================================================================
-- 2. Demandes de prêt : le dossier, sa décision et son statut de remboursement
-- ============================================================================
CREATE TABLE demandes_pret (
    pret_id               BIGINT        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    client_id             BIGINT        NOT NULL REFERENCES clients (client_id) ON DELETE RESTRICT,
    montant_demande       NUMERIC(12,2) NOT NULL CHECK (montant_demande > 0),
    duree_mois            SMALLINT      NOT NULL CHECK (duree_mois BETWEEN 6 AND 120),
    taux_interet          NUMERIC(5,2)  NOT NULL CHECK (taux_interet > 0 AND taux_interet < 40),
    taux_impute           BOOLEAN       NOT NULL DEFAULT false,   -- taux absent de la source, remplacé par la
     ↪ médiane
    motif_pret            VARCHAR(25)   NOT NULL
                          CHECK (motif_pret IN ('etudes', 'sante', 'creation_entreprise', 'personnel',
                                                'travaux', 'regroupement_dettes')),
    note_risque           CHAR(1)       NOT NULL CHECK (note_risque BETWEEN 'A' AND 'G'),
    date_demande          DATE          NOT NULL,
    date_decision         DATE,
    date_premiere_echeance DATE,
    statut_dossier        VARCHAR(12)   NOT NULL DEFAULT 'en_attente'
                          CHECK (statut_dossier IN ('en_attente', 'accorde', 'refuse')),
    -- statut de remboursement : n'a de sens que pour un prêt accordé
    statut_pret           VARCHAR(12)   CHECK (statut_pret IN ('sain', 'en_retard', 'en_defaut', 'solde')),
    CONSTRAINT decision_apres_demande CHECK (date_decision >= date_demande),
    CONSTRAINT statut_si_accorde      CHECK ((statut_dossier = 'accorde') = (statut_pret IS NOT NULL))
);

-- ============================================================================
-- 3. Échéancier : une ligne par mensualité, avec le suivi des paiements
-- ============================================================================
CREATE TABLE echeancier_remboursements (
    echeance_id        BIGINT        GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    pret_id            BIGINT        NOT NULL REFERENCES demandes_pret (pret_id) ON DELETE RESTRICT,
    num_echeance       SMALLINT      NOT NULL CHECK (num_echeance > 0),
```

```
    date_echeance      DATE          NOT NULL,
    montant_du         NUMERIC(12,2) NOT NULL CHECK (montant_du > 0),
    part_capital       NUMERIC(12,2) NOT NULL CHECK (part_capital >= 0),
    part_interets      NUMERIC(12,2) NOT NULL CHECK (part_interets >= 0),
    capital_restant_du NUMERIC(12,2) NOT NULL CHECK (capital_restant_du >= 0),  -- après cette échéance
    montant_paye       NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (montant_paye >= 0),
    date_paiement      DATE,                                  -- date du paiement complet, NULL sinon
    CONSTRAINT une_echeance_par_numero UNIQUE (pret_id, num_echeance),
    CONSTRAINT paiement_borne          CHECK (montant_paye <= montant_du),
    CONSTRAINT date_si_solde           CHECK ((date_paiement IS NOT NULL) = (montant_paye = montant_du))
);

-- ------------------------------------------------------------------------------
-- 4. Prédictions des modèles : un registre versionné (plusieurs modèles par prêt)
-- ------------------------------------------------------------------------------
CREATE TABLE predictions_scoring_ml (
    prediction_id      BIGINT        GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    pret_id            BIGINT        NOT NULL REFERENCES demandes_pret (pret_id) ON DELETE RESTRICT,
    version_modele     VARCHAR(40)   NOT NULL,
    est_champion       BOOLEAN       NOT NULL DEFAULT false,   -- le modèle qui décide en production
    probabilite_defaut NUMERIC(7,6)  NOT NULL CHECK (probabilite_defaut BETWEEN 0 AND 1),
    score_points       SMALLINT,                                -- grille de score uniquement
    seuil_decision     NUMERIC(4,3)  NOT NULL CHECK (seuil_decision BETWEEN 0 AND 1),
    decision           VARCHAR(10)   NOT NULL CHECK (decision IN ('accorder', 'refuser')),
    shap_top_feature   VARCHAR(50),                             -- NULL pour les modèles sans SHAP
    shap_top_valeur    NUMERIC(8,4),
    date_prediction    TIMESTAMP     NOT NULL,
    CONSTRAINT une_prediction_par_version UNIQUE (pret_id, version_modele),
    CONSTRAINT decision_coherente CHECK ((decision = 'refuser') = (probabilite_defaut >= seuil_decision))
);

-- Registre du découpage apprentissage / test : indispensable pour évaluer un modèle honnêtement
CREATE TABLE echantillons_modele (
    pret_id     BIGINT      PRIMARY KEY REFERENCES demandes_pret (pret_id) ON DELETE RESTRICT,
    echantillon VARCHAR(13) NOT NULL CHECK (echantillon IN ('apprentissage', 'test'))
);

-- ------------------------------------------------------------------------------
-- Index : PostgreSQL indexe automatiquement les clés primaires et les contraintes UNIQUE,
-- mais PAS les clés étrangères. Les index ci-dessous sont ceux qui servent vraiment.
-- ------------------------------------------------------------------------------
CREATE INDEX idx_prets_client ON demandes_pret (client_id);                -- jointure client → prêts
CREATE INDEX idx_prets_statut ON demandes_pret (statut_pret);              -- filtres de pilotage
-- index PARTIEL : seules les échéances non soldées, celles que l'on cherche pour le retard
CREATE INDEX idx_echeances_impayees
    ON echeancier_remboursements (pret_id, date_echeance)
    WHERE montant_paye < montant_du;
-- (pret_id, num_echeance) et (pret_id, version_modele) sont déjà indexés par leurs contraintes UNIQUE.
```

Comment l'exécuter et ce que tu dois obtenir

Dans un terminal, depuis le dossier du projet :

```
psql -d risk_analytics_db -v ON_ERROR_STOP=1 -f sql/01_schema.sql
```

L'option `ON_ERROR_STOP=1` arrête le script à la première erreur au lieu de continuer. Dans DBeaver : ouvre le fichier, puis *Exécuter le script* (`Alt+X`). La commande `\dt` de psql doit lister 6 tables : `clients`, `demandes_pret`, `echantillons_modele`, `echeancier_remboursements`, `parametres_risque`, `predictions_scoring_ml`.

Résumé du chapitre

1. On conçoit en trois temps : modèle conceptuel (entités, associations, cardinalités), modèle relationnel (tables, clés), code DDL.
2. Une association « un à plusieurs » devient une clé étrangère du côté « plusieurs ».

3. Les choix clés : `NUMERIC` pour l'argent, retard calculé et non stocké, `ON DELETE RESTRICT`, identités, table de paramètres, contraintes entre colonnes, registre versionné, deux statuts.
4. Les clés primaires et `UNIQUE` sont indexées automatiquement, pas les clés étrangères. Un index partiel n'indexe que les lignes utiles.

Partie 4

Modules de cours

Trois chapitres progressifs. Chacun suit le même plan : **1.** cours et notions importantes, avec les pièges à éviter ; **2.** exercices guidés, corrigés pas à pas ; **3.** exercices à faire seul (corrigés à la fin du cours) ; **4.** synthèse pour les entretiens.

Chaque requête a été exécutée sur ta base : les résultats affichés sont ceux que tu dois retrouver.

Chapitre 1

Modélisation, DDL et contraintes d'intégrité bancaire

« Une donnée fausse qui entre dans la base n'en ressort jamais seule. »

Ce premier chapitre pose les fondations. Tu vas comprendre **comment on découpe** les données en tables, **comment on choisit** les types et les clés, et surtout **comment on empêche** une donnée incohérente d'entrer dans la base. Tu créeras ensuite un dossier de prêt complet, du client jusqu'à son tableau d'amortissement, en une seule transaction.

Tes fichiers pour ce chapitre

— Ton code : `sql/02_chapitre1.sql`. Le corrigé complet : `corriges/chapitre1.sql`.

— Prérequis : les tâches 1 à 3 de la partie 5 (base créée par le schéma, puis chargée par le script Python).

Mise en situation — Le client de 15 ans

Le mois dernier, dans l'ancien tableur, un prêt a été accordé à un client de 15 ans : une faute de frappe, il en avait 51. Le même mois, une mensualité a été saisie deux fois comme payée, et un dossier refusé est apparu dans le rapport des prêts en cours. Personne ne s'en est aperçu pendant trois mois.

Ta responsable est claire : *« Je ne veux pas qu'on corrige ces erreurs. Je veux qu'elles soient impossibles. »*

1.1 Cours et notions importantes

1.1.1 Pourquoi plusieurs tables ? La normalisation

Imagine cette situation...

Imagine toutes les données dans un seul grand tableau, une ligne par mensualité. Pour le prêt d'Awa, 36 lignes répètent 36 fois son nom, son revenu, son adresse e-mail, le montant de son prêt et son taux. Awa change d'adresse e-mail : il faut modifier 36 lignes. Tu en oublies une : la base contient désormais deux adresses pour la même personne. Laquelle est la bonne ?

Ce grand tableau souffre de trois **anomalies** classiques :

— **de mise à jour** : une information répétée doit être modifiée partout, sinon elle devient contradictoire ;

— **d'insertion** : impossible d'enregistrer un client tant qu'il n'a pas de mensualité ;

— **de suppression** : supprimer la dernière mensualité d'un prêt efface aussi le client.

Définition — Normalisation

Normaliser, c'est découper les données en tables de sorte que **chaque fait soit stocké une seule fois, à un seul endroit**. Les trois premières formes normales en donnent la méthode :

1. **1FN** : chaque cellule contient une valeur atomique (pas de liste « 25/10, 25/11, 25/12 » dans une case) ;
2. **2FN** : chaque colonne dépend de **toute** la clé. Dans une table dont la clé est (prêt, numéro d'échéance), le montant du prêt ne dépend que du prêt : il va dans la table des prêts ;
3. **3FN** : aucune colonne ne dépend d'une autre colonne non clé. Le revenu dépend du client, pas du prêt : il va dans la table des clients.

C'est exactement ce découpage que tu as vu dans la partie 3 : `clients`, `demandes_pret`, `echeancier_remboursements`. Chaque table décrit **une seule chose**, et les clés étrangères les relient.

Normaliser la source, dénormaliser pour l'analyse

Pour entraîner un modèle de Machine Learning, on a besoin au contraire d'une seule grande table, une ligne par prêt avec toutes ses variables. On la construit *à partir* de la base normalisée, par des jointures, dans une vue ou un entrepôt de données. La règle : le système qui enregistre les opérations reste normalisé ; les tables d'analyse peuvent être dénormalisées.

1.1.2 Choisir les types de données

Le type d'une colonne est la première contrainte : il refuse tout ce qui n'a pas la bonne forme.

Type	Dans le projet	À savoir
`SMALLINT`, `INTEGER`, `BIGINT`	âge, durée ; identifiants	2, 4 ou 8 octets. Les identifiants en `BIGINT` : on ne veut jamais tomber à court de numéros.
`NUMERIC(p,s)`	montants, taux, PD	décimal exact : p chiffres dont s après la virgule. `NUMERIC(12,2)` va jusqu'à 9 999 999 999,99.
`REAL`, `DOUBLE PRECISION`	jamais pour l'argent	approximatifs : $0{,}1 + 0{,}2 \neq 0{,}3$.
`VARCHAR(n)`, `TEXT`	noms, statuts, descriptions	`VARCHAR(n)` refuse plus de n caractères, `TEXT` n'a pas de limite.
`CHAR(1)`	note de risque	longueur fixe.
`BOOLEAN`	`defaut_anterieur`, `est_champion`	vrai, faux ou NULL.
`DATE`	échéances, paiements	un jour, sans heure.
`TIMESTAMP`	`date_prediction`	date et heure. En production, on préfère souvent `TIMESTAMPTZ`, qui tient compte du fuseau horaire.

Les dates méritent une attention particulière, car tout le calcul du retard repose sur elles.

— `DATE - DATE` donne un **nombre entier de jours** : `DATE '2026-09-30' - DATE '2026-08-15'` vaut **46**. C'est le DPD du prêt 61.

— `DATE + entier` ajoute des jours ; `DATE + INTERVAL '1 month'` ajoute un mois et renvoie un `TIMESTAMP` (d'où les conversions `::date`).

Piège — Ajouter des mois en chaîne

Que vaut le 31 janvier plus un mois ? PostgreSQL renvoie le **28 février** : il s'arrête au dernier jour du mois. Le problème apparaît si l'on enchaîne les ajouts :

```
SELECT DATE '2026-01-31' + INTERVAL '1 month'                                AS plus_1_mois
    ↪ ,   -- 2026-02-28
       (DATE '2026-01-31' + INTERVAL '1 month') + INTERVAL '1 month'         AS
    ↪ chaine_2_mois, -- 2026-03-28
       DATE '2026-01-31' + 2 * INTERVAL '1 month'                            AS
    ↪ direct_2_mois; -- 2026-03-31
```

En partant de chaque échéance pour calculer la suivante, toutes les échéances après février seraient décalées au 28. La bonne méthode : toujours partir de la **première** échéance et ajouter $(k-1)$ mois. C'est ce que fait le corrigé de l'exercice 1.1.

1.1.3 Les clés : identifier et relier

Définition — Clé primaire

Une colonne (ou un groupe de colonnes) qui identifie **chaque ligne de façon unique**. Elle est toujours `NOT NULL` et `UNIQUE`. Une table a une seule clé primaire.

On distingue deux sortes de clés :

— une **clé naturelle** vient du métier : l'adresse e-mail d'un client, par exemple. Mais elle peut changer, et elle n'est pas toujours unique dans la durée ;

— une **clé artificielle** est un numéro sans signification, attribué par la base. C'est le choix du projet : `client_id`, `pret_id`, `echeance_id`. L'e-mail reste unique grâce à une contrainte `UNIQUE`.

La base attribue les numéros grâce aux colonnes **identité** :

```
client_id   BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY  -- numéro automatique, ou
    ↪ fourni
echeance_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY      -- numéro toujours
    ↪ automatique
```

Avec `ALWAYS`, toute tentative d'insérer un numéro à la main est refusée. Avec `BY DEFAULT`, on peut fournir le sien : c'est utile au chargement, pour garder le numéro de ligne du fichier source. L'ancienne syntaxe `SERIAL` fait la même chose, en moins strict.

Définition — Clé étrangère

Une colonne dont chaque valeur doit exister comme clé primaire dans une autre table. Elle garantit l'**intégrité référentielle** : aucune échéance ne peut appartenir à un prêt qui n'existe pas.

Que faire si l'on supprime la ligne référencée ? C'est l'option `ON DELETE` :

Option	Effet si l'on supprime un client qui a des prêts	Usage
`RESTRICT` / `NO ACTION`	la suppression est **refusée**	données à conserver : le choix bancaire
`CASCADE`	les prêts sont supprimés aussi, puis leurs échéances…	données sans valeur propre
`SET NULL`	le `client_id` des prêts devient NULL	lien facultatif

Piège — La clé étrangère n'est pas indexée

PostgreSQL crée un index pour chaque clé primaire et chaque contrainte `UNIQUE`, mais **pas pour les clés étrangères**. Sans index sur `demandes_pret.client_id`, chaque jointure d'un client vers ses prêts relit toute la table des prêts, et chaque suppression d'un client doit parcourir toute la table pour vérifier qu'aucun prêt ne le référence. D'où `idx_prets_client`.

Enfin, une contrainte `UNIQUE` peut porter sur **plusieurs colonnes** : `UNIQUE (pret_id, num_echeance)` interdit deux échéances numéro 5 pour le même prêt, tout en permettant l'échéance numéro 5 de chaque prêt.

1.1.4 NOT NULL et CHECK : les règles métier dans la base

`NOT NULL` interdit l'absence de valeur ; `DEFAULT` fournit une valeur quand on n'en donne pas. La contrainte `CHECK` accepte une ligne seulement si une condition est vraie :

```
age        SMALLINT     NOT NULL CHECK (age BETWEEN 18 AND 100),
note_risque CHAR(1)     NOT NULL CHECK (note_risque BETWEEN 'A' AND 'G'),
motif_pret VARCHAR(25)  NOT NULL CHECK (motif_pret IN ('etudes', 'sante', 'creation_entreprise
    ↪ ',
                                                       'personnel', 'travaux', '
    ↪ regroupement_dettes')),
```

Une contrainte peut porter sur **plusieurs colonnes de la même ligne**. On lui donne alors un nom, qui apparaîtra dans les messages d'erreur :

```
CONSTRAINT paiement_borne         CHECK (montant_paye <= montant_du),
CONSTRAINT decision_apres_demande CHECK (date_decision >= date_demande)
```

1.1.4.1 Deux astuces logiques à connaître

L'équivalence : comparer deux conditions. En SQL, une condition est une valeur booléenne. On peut donc écrire `(A) = (B)` pour dire « A si et seulement si B ». Exemple : un prêt a un statut de remboursement *si et seulement si* il est accordé.

```
CONSTRAINT statut_si_accorde CHECK ((statut_dossier = 'accorde') = (statut_pret IS NOT NULL))
```

`statut_dossier`	`statut_pret`	accordé ?	statut renseigné ?	Ligne acceptée ?
accorde	sain	vrai	vrai	✓ oui
refuse	NULL	faux	faux	✓ oui
accorde	NULL	vrai	faux	non
refuse	sain	faux	vrai	non

La même astuce relie une décision à la PD et au seuil : `(decision = 'refuser') = (probabilite_defaut >= seuil_decision)`.

L'implication : « si A alors B ». Elle s'écrit `NOT A OR B`. Par exemple, « si l'ancienneté est connue, elle ne dépasse pas l'âge moins 14 ans » :

```
CONSTRAINT anciennete_coherente CHECK (anciennete_emploi IS NULL OR anciennete_emploi <= age -
    ↪ 14)
```

Piège — Un CHECK laisse passer NULL

Une contrainte `CHECK` refuse une ligne seulement si la condition est **fausse**. Si elle vaut **NULL** (inconnue), la ligne est acceptée. Or toute comparaison avec NULL donne NULL : `SELECT NULL::int > 0` renvoie NULL, pas faux. Donc `CHECK (montant > 0)` accepte un montant NULL ! Pour l'interdire, il faut ajouter `NOT NULL`. C'est voulu pour `date_decision` (un dossier en attente n'a pas encore de décision), pas pour un montant.

Ce qu'un CHECK ne peut pas faire

Un `CHECK` ne voit que la ligne en cours. Il ne peut pas vérifier une règle qui porte sur **plusieurs lignes** : « la somme des parts de capital d'un prêt égale le montant prêté », « un prêt a exactement autant d'échéances que de mois ». Pour ces règles, on écrit des requêtes de contrôle (exercice 1.4) ou des triggers (chapitre 3).

1.1.5 Les transactions : tout ou rien

Imagine cette situation...

Tu fais un virement de 500 € de ton compte vers celui d'un ami. La banque doit faire deux opérations : retirer 500 € de ton compte, ajouter 500 € au sien. Si le serveur tombe en panne entre les deux, l'argent a disparu. Inacceptable : soit les deux opérations ont lieu, soit aucune.

Définition — Transaction et propriétés ACID

Une transaction est un groupe d'instructions qui forment un tout. PostgreSQL garantit quatre propriétés :

- **Atomicité** : tout est appliqué, ou rien ;
- **Cohérence** : la base passe d'un état valide à un état valide (toutes les contraintes sont respectées) ;
- **Isolation** : les transactions simultanées ne voient pas le travail inachevé des autres ;
- **Durabilité** : une fois validée, une transaction survit à une panne.

```
BEGIN;                -- début de la transaction
INSERT INTO ...;      -- plusieurs instructions
UPDATE ...;
COMMIT;               -- tout est validé d'un coup (ou ROLLBACK : tout est annulé)
```

Sans `BEGIN`, chaque instruction est sa propre transaction (mode *autocommit*, celui de psql et de DBeaver par défaut). Deux usages reviendront tout au long du cours :

— **créer des données liées** (un client, son prêt, ses échéances) : si une étape échoue, rien n'est enregistré ;

— **tester sans risque** : `BEGIN; ... ; ROLLBACK;` exécute un scénario puis annule tout.

Piège — La transaction « interrompue »

Si une instruction échoue au milieu d'une transaction, PostgreSQL refuse toutes les suivantes avec le message *current transaction is aborted, commands ignored until end of transaction block*. Il faut faire `ROLLBACK` pour repartir. Dans DBeaver, vérifie aussi le mode de validation (bouton *Auto-Commit* de la barre d'outils) : en mode manuel, une transaction oubliée garde des verrous et bloque les autres utilisateurs.

1.1.6 Insérer des données liées : les CTE qui modifient

Pour créer un prêt, il faut connaître le `client_id` que la base vient d'attribuer. La clause `RETURNING` renvoie les lignes insérées, et une **CTE** (`WITH`, détaillée au chapitre 2) permet de les réutiliser dans l'instruction suivante :

```
WITH nouveau_client AS (
    INSERT INTO clients (...) VALUES (...)
    RETURNING client_id                      -- le numéro attribué par la base
)
INSERT INTO demandes_pret (client_id, ...)
SELECT client_id, ... FROM nouveau_client;   -- on le réutilise sans le connaître à l'avance
```

Deux règles à connaître : une CTE qui modifie des données **s'exécute toujours**, même si personne ne lit son résultat ; et toutes les parties de l'instruction voient la base **telle qu'elle était au début**. Une CTE ne voit pas les lignes insérées par une autre CTE, sauf à travers son `RETURNING`.

1.1.7 Le tableau d'amortissement : la dérivation complète

Hands-on — La mensualité d'Awa

Awa emprunte $C = 12\,000$ € sur $n = 36$ mois au taux annuel de 9,5 %. Elle veut payer **la même somme chaque mois**. Quelle est cette mensualité M ?

L'intuition. Chaque mois, la banque facture des intérêts sur ce qui reste dû : le **capital restant dû** (CRD) multiplié par le taux mensuel r. La mensualité M paie d'abord ces intérêts ; le reste rembourse le capital. Au début, le CRD est élevé, donc les intérêts aussi : la part de capital est petite. Puis le CRD baisse, les intérêts aussi, et la part de capital grandit (figure 1.1).

La dérivation. Le taux mensuel est $r = \dfrac{\text{taux annuel}}{12 \times 100}$. Notons CRD_k le capital restant dû après k mensualités.

1. Au départ, $CRD_0 = C$.
2. Chaque mois, le CRD produit des intérêts puis la mensualité est payée :

$$CRD_k = CRD_{k-1}\,(1 + r) - M$$

3. En déroulant cette relation k fois, on obtient une somme géométrique :

$$CRD_k = C(1+r)^k - M\left[(1+r)^{k-1} + \cdots + (1+r) + 1\right] = C(1+r)^k - M\,\frac{(1+r)^k - 1}{r}$$

4. Le prêt est remboursé après n mois : $CRD_n = 0$. On isole M :

$$M\,\frac{(1+r)^n - 1}{r} = C(1+r)^n \quad \Longrightarrow \quad \boxed{M = \frac{C\,r}{1-(1+r)^{-n}}}$$

Calcul à la main — la mensualité d'Awa

1. Taux mensuel : $r = 9{,}5/1200 = 0{,}0079167$.
2. $(1+r)^{36} = 1{,}328271$, donc $(1+r)^{-36} = 0{,}752859$ et $1-(1+r)^{-36} = 0{,}247141$.
3. $C \times r = 12\,000 \times 0{,}0079167 = 95{,}00$.
4. $M = 95{,}00/0{,}247141 = 384{,}395$, arrondi au centime : **384,40 €**.

Premier mois : intérêts = 12 000 × r = 95,00 €, capital remboursé = 384,40 − 95,00 = 289,40 €, nouveau CRD = 11 710,60 €. Coût total du crédit : 1 838,21 €.

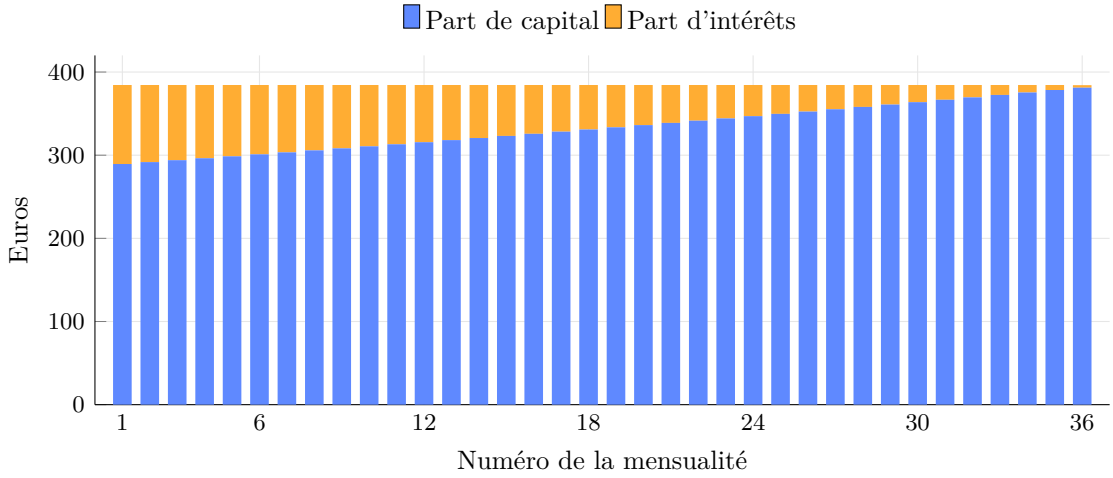


FIGURE 1.1 – Le tableau d'amortissement d'Awa, tel que ta base le génère : la mensualité reste à 384,40 €, mais sa composition change. La dernière (384,21 €) solde exactement le capital restant.

Piège — Les arrondis au centime

La mensualité est arrondie au centime (384,40 au lieu de 384,395…). Si l'on arrondit ensuite *chaque* colonne indépendamment (intérêts, capital, CRD), le CRD d'un mois ne vaut plus exactement le CRD du mois précédent moins la part de capital : des écarts d'un centime apparaissent. La bonne méthode : arrondir les CRD, puis définir la part de capital comme la **différence** de deux CRD arrondis, et ajuster la dernière mensualité pour solder exactement le prêt. Les contrôles de l'exercice 1.4 vérifient tout cela.

1.1.8 Générer des lignes : generate_series

Comment produire 36 lignes à partir d'un seul prêt ? La fonction `generate_series(a, b)` renvoie une table d'entiers de a à b :

```sql
SELECT p.pret_id, k AS num_echeance,
       (p.date_premiere_echeance + (k - 1) * INTERVAL '1 month')::date AS date_echeance
FROM demandes_pret p
CROSS JOIN generate_series(1, p.duree_mois) AS k      -- k = 1, 2, ..., duree_mois
WHERE p.pret_id = 32582;
```

Le `CROSS JOIN` associe chaque prêt à chaque valeur de k : un prêt de 36 mois produit 36 lignes. La fonction peut utiliser les colonnes du prêt (`p.duree_mois`) : PostgreSQL l'évalue pour chaque prêt.

1.1.9 Les index B-Tree : retrouver une ligne parmi un million

Analogie — l'annuaire téléphonique

Pour trouver le numéro de « Koné » dans un annuaire, tu ne lis pas toutes les pages : tu l'ouvres au milieu, tu vois que tu es à « M », tu reviens en arrière, et en quelques gestes tu es à la bonne page. C'est possible parce que l'annuaire est **trié**. Un index est un annuaire de la table : une copie triée d'une ou plusieurs colonnes, avec pour chaque valeur l'adresse de la ligne.

L'index B-Tree (*balanced tree*, l'index par défaut de PostgreSQL) range les valeurs dans un arbre équilibré. Chaque nœud contient des centaines de valeurs : trois ou quatre niveaux suffisent pour plus d'un million de lignes (figure 1.2).

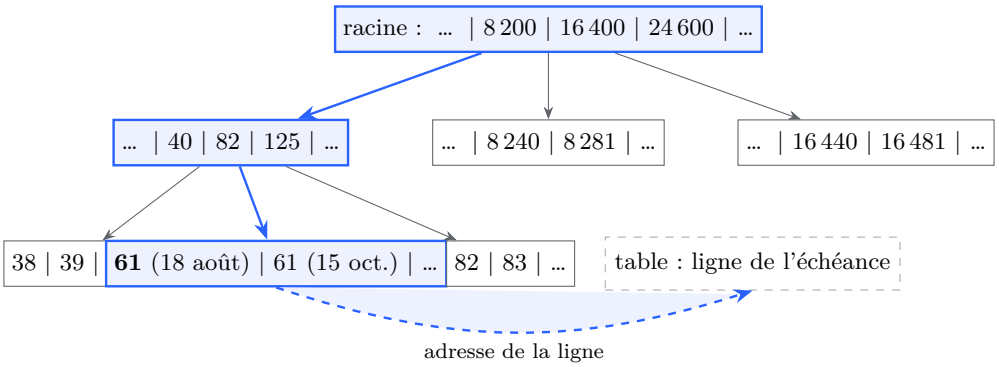


FIGURE 1.2 – Recherche de l'impayé le plus ancien du prêt 61 dans l'index partiel `idx_echeances_impayees (pret_id, date_echeance)` : trois nœuds lus au lieu de 1,1 million de lignes.

1.1.9.1 Ce que ça change, mesuré sur ta base

La commande `EXPLAIN ANALYZE` exécute une requête et affiche le **plan** choisi par PostgreSQL avec les temps réels. Voici le calcul du retard du prêt 61 :

```
EXPLAIN ANALYZE
SELECT coalesce(DATE '2026-09-30' - min(date_echeance), 0)
FROM echeancier_remboursements
WHERE pret_id = 61 AND date_echeance <= DATE '2026-09-30' AND montant_paye < montant_du;
```

Avec l'index partiel : PostgreSQL descend l'arbre et lit une seule entrée.

```
->  Index Only Scan using idx_echeances_impayees on echeancier_remboursements
      Index Cond: ((pret_id = 61) AND (date_echeance <= '2026-09-30'::date))
Execution Time: 0.318 ms
```

Sans index (supprimés le temps du test, dans une transaction annulée) : PostgreSQL lit toute la table, en parallèle sur trois processus.

```
->  Parallel Seq Scan on echeancier_remboursements
      Filter: ((date_echeance <= '2026-09-30'::date) AND (montant_paye < montant_du) AND (pret_id = 61))
      Rows Removed by Filter: 369324
```

```
Execution Time: 113.944 ms
```

Soit un facteur **350**. Pour un prêt, 114 ms paraissent acceptables. Mais au chapitre 3, une fonction calculera le retard des 32 410 prêts un par un : avec l'index, 0,25 seconde ; sans index, plus de 20 minutes.

1.1.9.2 Les règles à connaître

— **Index composite et préfixe gauche.** L'index `(pret_id, date_echeance)` est trié par prêt, puis par date à l'intérieur de chaque prêt, comme un annuaire trié par nom puis prénom. Il sert pour `WHERE pret_id = 61` et pour `WHERE pret_id = 61 AND date_echeance <= ...`, mais **pas** pour `WHERE date_echeance <= ...` seul.

— **Index partiel.** `WHERE montant_paye < montant_du` n'indexe que les échéances non soldées : 13 Mo au lieu de 33 Mo pour un index complet, et il reste à jour automatiquement (une échéance payée en sort d'elle-même).

— **Un index a un coût.** Chaque `INSERT` ou `UPDATE` doit aussi mettre à jour les index, et ils occupent de la place. On n'indexe que les colonnes réellement utilisées pour filtrer ou joindre.

— **`ANALYZE`** met à jour les statistiques que PostgreSQL utilise pour choisir son plan : à lancer après un gros chargement (le chargeur le fait).

Piège — Une fonction sur la colonne indexée

`WHERE date_trunc('month', date_echeance) = DATE '2026-09-01'` ne peut pas utiliser un index sur `date_echeance` : l'index est trié sur la date, pas sur le résultat de la fonction. Écris plutôt un intervalle : `WHERE date_echeance >= DATE '2026-09-01' AND date_echeance < DATE '2026-10-01'`.

1.2 Exercices guidés

Exercice 1.1 — Ouvrir un nouveau dossier de prêt complet, en une transaction

Crée le dossier d'Awa Koné, en **une seule transaction** :

1. la cliente : Awa Koné, `awa.kone@exemple.fr`, 29 ans, 42 000 € de revenu, locataire, 4 ans d'ancienneté dans l'emploi, 6 ans d'historique de crédit, aucun défaut antérieur, inscrite le 15/09/2026 ;
2. son prêt : 12 000 € sur 36 mois à 9,5 %, motif études, note B, demandé le 20/09/2026, accordé le 25/09/2026, première échéance le 25/10/2026, statut « sain » ;
3. la prédiction du modèle champion `xgboost_monotone_v2` qui a justifié la décision : PD 0,118, seuil 0,255, décision « accorder », variable SHAP principale `taux_effort` (0,912), calculée le 24/09/2026 à 14 h 30 ;
4. ses 36 échéances, à mensualité constante.

Aucun identifiant ne doit être écrit en dur : utilise ceux que la base attribue. Vérifie ensuite le nombre d'échéances, le capital total remboursé et le coût du crédit.

Correction de l'exercice 1.1

Étape 1 — Ouvrir la transaction. `BEGIN` : si une seule des quatre insertions échoue, aucune ne sera enregistrée.

Étape 2 — La cliente. Une CTE insère la cliente et renvoie son `client_id`.

Étape 3 — Le prêt. La CTE suivante lit ce `client_id` dans `nouveau_client`. Son `RETURNING` renvoie tout ce dont l'échéancier aura besoin, sous des noms courts : le capital c, la durée n, le taux mensuel r et la date de première échéance.

Étape 4 — La prédiction. Une CTE insère le score. Personne ne la lit ensuite : elle s'exécute quand même.

Étape 5 — Les mensualités. `generate_series(1, n)` produit les 36 numéros ; la mensualité est calculée par la formule d'annuité et arrondie au centime ; chaque date part de la première échéance (voir le piège des mois).

Étape 6 — Le capital restant dû. Pour chaque k, on calcule CRD_{k-1} et CRD_k par la formule fermée, arrondis au centime.

Étape 7 — L'insertion. Pour les 35 premières échéances : montant $= M$, capital $= CRD_{k-1} - CRD_k$, intérêts $= M -$ capital. La dernière solde exactement le capital restant : capital $= CRD_{35}$, intérêts $= CRD_{35} \times r$.

```sql
BEGIN;

WITH nouveau_client AS (
    INSERT INTO clients (nom_complet, email, age, revenu_annuel, statut_logement,
                         anciennete_emploi, anciennete_credit, defaut_anterieur, date_inscription)
    VALUES ('Awa Koné', 'awa.kone@exemple.fr', 29, 42000, 'locataire', 4, 6, false, DATE '2026-09-15')
    RETURNING client_id
),
nouveau_pret AS (
    INSERT INTO demandes_pret (client_id, montant_demande, duree_mois, taux_interet, motif_pret, note_risque,
                               date_demande, date_decision, date_premiere_echeance, statut_dossier, statut_pret)
    SELECT client_id, 12000, 36, 9.5, 'etudes', 'B',
           DATE '2026-09-20', DATE '2026-09-25', DATE '2026-10-25', 'accorde', 'sain'
    FROM nouveau_client
    RETURNING pret_id, montant_demande AS c, duree_mois AS n, taux_interet / 1200 AS r, date_premiere_echeance
),
-- Le score du modèle champion qui a justifié la décision (PD 11,8 % < seuil 25,5 % : accordé).
-- Une CTE qui modifie des données s'exécute même si personne ne la lit ensuite.
score AS (
    INSERT INTO predictions_scoring_ml (pret_id, version_modele, est_champion, probabilite_defaut,
                                        seuil_decision, decision, shap_top_feature, shap_top_valeur,
     ↪ date_prediction)
    SELECT pret_id, 'xgboost_monotone_v2', true, 0.118, 0.255, 'accorder', 'taux_effort', 0.912,
           TIMESTAMP '2026-09-24 14:30'
    FROM nouveau_pret
),
-- Mensualité constante : M = C × r / (1 − (1 + r)−^n)
mensualites AS (
    SELECT p.pret_id, p.c, p.r, p.n, k AS num_echeance,
           (p.date_premiere_echeance + (k − 1) * INTERVAL '1 month')::DATE AS date_echeance,
           round(p.c * p.r / (1 − (1 + p.r) ^ (−p.n)), 2)                  AS mensualite
    FROM nouveau_pret p
    CROSS JOIN generate_series(1, p.n) AS k                    -- une ligne par mensualité
),
-- Capital restant dû après k mensualités : CRD(k) = C(1 + r)^k − M((1 + r)^k − 1) / r, arrondi au centime.
-- La part de capital est la DIFFÉRENCE de deux CRD arrondis : le CRD baisse exactement de la part de capital.
calcul AS (
    SELECT pret_id, num_echeance, date_echeance, mensualite, r, n,
           round(c * (1 + r) ^ (num_echeance − 1) − mensualite * ((1 + r) ^ (num_echeance − 1) − 1) / r, 2) AS
     ↪ crd_avant,
           round(c * (1 + r) ^ num_echeance       − mensualite * ((1 + r) ^ num_echeance − 1) / r, 2)       AS
     ↪ crd_apres
    FROM mensualites
)
```

```sql
INSERT INTO echeancier_remboursements (pret_id, num_echeance, date_echeance, montant_du,
                                       part_capital, part_interets, capital_restant_du)
SELECT pret_id, num_echeance, date_echeance,
       -- la dernière échéance solde exactement le capital restant
       CASE WHEN num_echeance = n THEN crd_avant + round(crd_avant * r, 2) ELSE mensualite END,
       CASE WHEN num_echeance = n THEN crd_avant ELSE crd_avant - crd_apres END,
       CASE WHEN num_echeance = n THEN round(crd_avant * r, 2) ELSE mensualite - (crd_avant - crd_apres) END,
       CASE WHEN num_echeance = n THEN 0 ELSE crd_apres END
FROM calcul;

COMMIT;
```

Étape 8 — Vérifier. On retrouve le prêt par l'e-mail de la cliente, sans connaître son numéro. `FILTER` (vu au chapitre 2) isole une échéance dans un agrégat.

```sql
SELECT p.pret_id, count(*) AS nb_echeances, sum(e.part_capital) AS capital_rembourse,
       sum(e.part_interets) AS cout_du_credit,
       max(e.montant_du) FILTER (WHERE e.num_echeance = 1) AS mensualite,
       max(e.montant_du) FILTER (WHERE e.num_echeance = p.duree_mois) AS derniere_echeance
FROM demandes_pret p JOIN echeancier_remboursements e USING (pret_id)
WHERE p.client_id = (SELECT client_id FROM clients WHERE email = 'awa.kone@exemple.fr')
GROUP BY p.pret_id;
```

pret_id	nb_echeances	capital_rembourse	cout_du_credit	mensualite	derniere_echeance
32582	36	12 000,00	1 838,21	384,40	384,21

À retenir. Les identifiants circulent par `RETURNING` au lieu d'être écrits en dur ; la transaction garantit le « tout ou rien » ; l'échéancier est calculé en une seule requête ensembliste, sans boucle.

Exercice 1.2 — Les contraintes refusent les données incohérentes

Écris quatre instructions qui violent chacune une règle, exécute-les **une par une** et lis le message d'erreur :

a) créer un client de 15 ans ;

b) enregistrer sur une échéance future impayée un paiement supérieur au montant dû ;

c) supprimer le client 2, qui a un prêt ;

d) créer un prêt refusé dont le statut de remboursement est « sain ».

Correction de l'exercice 1.2

```sql
-- a) Un client de 15 ans
INSERT INTO clients (nom_complet, email, age, revenu_annuel, statut_logement, anciennete_credit,
                     defaut_anterieur, date_inscription)
VALUES ('Test', 'test@exemple.fr', 15, 20000, 'locataire', 0, false, CURRENT_DATE);

-- b) Payer plus que le montant dû, sur une échéance future encore impayée
UPDATE echeancier_remboursements SET montant_paye = montant_du + 1
WHERE echeance_id = (SELECT min(echeance_id) FROM echeancier_remboursements
                     WHERE montant_paye = 0 AND date_echeance > DATE '2026-09-30');

-- c) Supprimer un client qui a des prêts
DELETE FROM clients WHERE client_id = 2;

-- d) Un prêt refusé avec un statut de remboursement
INSERT INTO demandes_pret (client_id, montant_demande, duree_mois, taux_interet, motif_pret, note_risque,
```

```
                            date_demande, statut_dossier, statut_pret)
VALUES (2, 1000, 12, 10, 'etudes', 'A', DATE '2026-09-01', 'refuse', 'sain');
```

Les quatre messages obtenus :

```
a) ERROR: new row for relation "clients" violates check constraint "clients_age_check"
b) ERROR: new row for relation "echeancier_remboursements" violates check constraint "
    ↪ paiement_borne"
c) ERROR: update or delete on table "clients" violates foreign key constraint
          "demandes_pret_client_id_fkey" on table "demandes_pret"
   DETAIL: Key (client_id)=(2) is still referenced from table "demandes_pret".
d) ERROR: new row for relation "demandes_pret" violates check constraint "statut_si_accorde"
```

Lecture des messages. En a), la contrainte n'a pas de nom explicite : PostgreSQL l'a nommée `clients_age_check`. En b), d) le nom choisi dans le schéma rend l'erreur immédiatement compréhensible : c'est l'intérêt de nommer ses contraintes.

Pourquoi une échéance future en b) ? Sur une échéance déjà payée, le nouveau montant violerait *deux* contraintes : `paiement_borne`, mais aussi `date_si_solde` (une date de paiement alors que le montant payé ne vaut plus le montant dû). PostgreSQL signale alors la première qu'il vérifie, `date_si_solde`, ce qui brouille la démonstration.

Pourquoi la base, et pas l'application ? Une application peut avoir un bug, un analyste peut écrire une requête directement, un script de reprise peut contourner l'interface. La contrainte en base protège **tous** les chemins d'accès.

1.3 Exercices à faire seul

Exercice 1.3 — Les deux tables du chapitre 3

Le chapitre 3 a besoin de deux nouvelles tables. Écris leur `CREATE TABLE`, avec les types et toutes les contraintes qui te paraissent nécessaires.

1. `journal_statuts` — le journal d'audit des changements de statut d'un prêt. Chaque ligne enregistre : un identifiant automatique ; le prêt concerné ; l'ancien statut (qui peut être absent) et le nouveau (obligatoire) ; le nombre de jours de retard au moment du changement (jamais négatif) ; la date d'effet ; l'horodatage de l'enregistrement (par défaut, l'instant présent) ; l'utilisateur qui a fait le changement (par défaut, l'utilisateur connecté). Une ligne n'a de sens que si le statut a réellement changé. Pense à l'index utile.

2. `provisions_mensuelles` — une « photographie » du risque de chaque prêt à chaque date d'arrêté : le retard, le stage IFRS 9 (1, 2 ou 3), l'EAD, la PD utilisée, la LGD et l'ECL. Un prêt ne peut avoir qu'**une ligne par arrêté**.

Indices

- « Ancien différent de nouveau » : que donne `NULL <> 'sain'` ? Cherche l'opérateur `IS DISTINCT FROM`.
- L'utilisateur connecté s'obtient avec `session_user` ; l'instant présent avec `now()`.
- « Une ligne par prêt et par arrêté » : une clé primaire peut porter sur deux colonnes.
- Les clés étrangères : quelle option `ON DELETE` pour un journal d'audit ?

Exercice 1.4 — Contrôles qualité de l'échéancier

Les contraintes vérifient chaque *ligne*. Certaines règles portent sur un prêt *entier* ou sur deux échéances successives : il faut les contrôler par des requêtes. Écris des requêtes qui comptent les anomalies suivantes (le résultat attendu est 0 partout) :

a) par prêt accordé : la somme des parts de capital diffère du montant prêté ; le nombre d'échéances ou le dernier numéro diffère de la durée ; le CRD de la dernière échéance n'est pas nul ; une échéance a un montant dû différent de capital + intérêts ;

b) entre deux échéances successives d'un même prêt : le CRD ne baisse pas exactement de la part de capital ; l'écart entre les deux dates n'est pas compris entre 28 et 31 jours ;

c) un paiement est daté d'avant l'octroi du prêt.

Indices

— a) Un `GROUP BY p.pret_id` sur la jointure prêts–échéances, puis compter les prêts en anomalie. `count(*) FILTER (WHERE condition)` compte les lignes qui vérifient la condition (vu au chapitre 2) ; `sum(CASE WHEN condition THEN 1 ELSE 0 END)` fait la même chose.

— b) Joins la table à elle-même (**auto-jointure**) : l'échéance `prec` et l'échéance `suiv` du même prêt, avec `suiv.num_echeance = prec.num_echeance + 1`.

— Si tu trouves des écarts d'un centime, relis le piège des arrondis.

1.4 Synthèse pour les entretiens

Résumé du chapitre

1. **Normaliser** : chaque fait stocké une seule fois. Les tables d'analyse pour le ML peuvent être dénormalisées, pas le système source.
2. **Types** : `NUMERIC` pour l'argent ; `DATE - DATE` donne des jours ; toujours ajouter des mois à partir de la première échéance.
3. **Clés** : primaire (unique et non nulle), étrangère (intégrité référentielle, `RESTRICT` en banque), `UNIQUE` composite. Les clés étrangères ne sont pas indexées automatiquement.
4. **CHECK** : règles sur une ligne ; équivalence `(A) = (B)`, implication `NOT A OR B` ; un CHECK laisse passer NULL.
5. **Transactions** : ACID, tout ou rien ; `BEGIN ... ROLLBACK` pour tester sans risque.
6. **CTE et RETURNING** pour insérer des données liées sans écrire d'identifiant en dur.
7. **Mensualité** : $M = C\,r/(1-(1+r)^{-n})$; attention aux arrondis au centime.
8. **Index B-Tree** : recherche en quelques nœuds ; préfixe gauche ; index partiel ; mesuré : 0,3 ms contre 114 ms.

Ce que tu sauras dire en entretien

— « J'ai conçu un schéma normalisé de six tables, plus deux tables d'audit et de provisions. Les règles métier sont dans la base : des contraintes `CHECK` nommées, y compris entre colonnes, comme l'équivalence entre une décision de refus et une PD au-dessus du seuil. »

— « J'utilise `NUMERIC` pour les montants, parce qu'un `FLOAT` ne représente pas exactement 0,10 et que les totaux comptables doivent tomber juste au centime. »

— « Toutes les clés étrangères sont en `ON DELETE RESTRICT` : supprimer un client ne doit jamais effacer l'historique de ses prêts. Et je les indexe, car PostgreSQL ne le fait pas. »

— « Le calcul du retard utilise un index partiel sur les seules échéances impayées : 13 Mo au lieu de 33, et une requête en 0,3 ms au lieu de 114. »

Questions fréquentes en entretien

Clé primaire ou contrainte UNIQUE ? Une table a une seule clé primaire, jamais NULL ; elle peut avoir plusieurs contraintes `UNIQUE`, qui acceptent les NULL (plusieurs NULL ne sont pas considérés comme égaux).

DELETE, TRUNCATE ou DROP ? `DELETE` supprime des lignes (avec un `WHERE`, ligne à ligne, triggers déclenchés) ; `TRUNCATE` vide toute la table d'un coup, très vite ; `DROP` supprime la table elle-même.

Pourquoi ne pas tout mettre en CASCADE ? Parce qu'une seule suppression pourrait effacer en chaîne des milliers de lignes d'historique, sans avertissement.

Quand un index ralentit-il ? Il ralentit les écritures (chaque insertion met à jour chaque index) et devient inutile si la requête lit une grande partie de la table.

Qu'est-ce qu'une transaction ? Un groupe d'instructions atomique : tout ou rien. Avec les propriétés ACID.

Chapitre 2

SQL analytique avancé : CTE et fonctions de fenêtre

« Les données ne parlent pas d'elles-mêmes : il faut savoir les interroger. »

La base est construite et remplie. Il faut maintenant la faire parler : combien de prêts sont en retard, combien la banque doit provisionner, si le modèle classe toujours bien les clients, comment évoluent les encaissements. Ces questions demandent des outils plus puissants qu'un simple `GROUP BY` : les **CTE**, les **vues**, les **agrégats conditionnels** et surtout les **fonctions de fenêtre**, les notions SQL les plus testées en entretien data.

Tes fichiers pour ce chapitre

— Ton code : `sql/03_chapitre2.sql`. Le corrigé complet : `corriges/chapitre2.sql`.

— Prérequis : le chapitre 1 exécuté. « Aujourd'hui » est la date de référence de la base, le 30/09/2026.

Mise en situation — Le comité des risques de 9 heures

Demain matin, le directeur des risques présente l'état du portefeuille au comité de direction. Il te demande quatre chiffres pour 9 heures :

1. la répartition des prêts par tranche de retard, et la part de l'encours à risque ;
2. le montant des provisions IFRS 9, stage par stage ;
3. la preuve que le modèle de scoring classe toujours correctement les clients ;
4. l'évolution des encaissements sur les douze derniers mois.

Il est 17 heures. Tu as une base, PostgreSQL et ce chapitre.

2.1 Cours et notions importantes

2.1.1 L'ordre d'exécution d'une requête

On écrit une requête dans l'ordre `SELECT ... FROM ... WHERE ...`, mais PostgreSQL l'exécute dans un autre ordre (figure 2.1). Le connaître explique la moitié des erreurs de débutant.

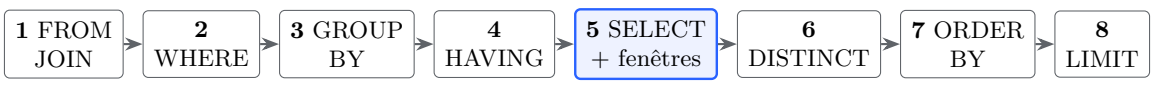


FIGURE 2.1 – L'ordre logique d'exécution. Les fonctions de fenêtre sont calculées à l'étape 5, après le filtrage et le regroupement.

Trois conséquences pratiques :

1. un alias défini dans le `SELECT` n'existe pas encore dans le `WHERE` ;

2. on ne peut pas filtrer sur le résultat d'une fonction de fenêtre dans le `WHERE` (elle n'est pas encore calculée). PostgreSQL n'a pas de clause `QUALIFY` comme d'autres moteurs : on calcule la fenêtre dans une CTE, puis on filtre dans la requête suivante ;
3. une fonction de fenêtre peut s'appliquer à un agrégat : `sum(sum(encours)) OVER ()` est valide, car le `GROUP BY` est déjà fait quand la fenêtre est calculée.

2.1.2 Les CTE : découper une requête en étapes nommées

Analogie — la recette de cuisine

Une recette ne dit pas « mélange les œufs battus avec la farine tamisée et le beurre fondu cuit à feu doux » en une seule phrase. Elle découpe : *1. Fais fondre le beurre. 2. Bats les œufs. 3. Mélange.* Chaque étape a un nom, et la suivante l'utilise. Une CTE fait la même chose pour une requête.

Définition — CTE (Common Table Expression)

Une sous-requête nommée, définie avec `WITH nom AS (...)` avant la requête principale. Chaque CTE peut utiliser les précédentes. Elle n'existe que le temps de la requête.

```sql
WITH impayes AS (                          -- étape 1 : les échéances échues non soldées
    SELECT pret_id, date_echeance
    FROM echeancier_remboursements
    WHERE date_echeance <= DATE '2026-09-30' AND montant_paye < montant_du
),
retards AS (                               -- étape 2 : le retard de chaque prêt
    SELECT pret_id, DATE '2026-09-30' - min(date_echeance) AS jours_retard
    FROM impayes
    GROUP BY pret_id
)
SELECT count(*) FILTER (WHERE jours_retard > 90) AS prets_en_defaut   -- étape 3 : la réponse
FROM retards;                                                         -- 7 088
```

Outil	Durée de vie	Quand l'utiliser
Sous-requête	la requête	un calcul court, utilisé une fois
CTE (`WITH`)	la requête	découper un calcul en étapes lisibles
Vue	permanente (seule la définition est stockée)	une règle réutilisée par beaucoup de requêtes
Vue matérialisée	permanente, résultat stocké	un calcul lourd, rafraîchi périodiquement (`REFRESH MATERIALIZED VIEW`)
Table temporaire	la session	un résultat intermédiaire réutilisé par plusieurs requêtes

Piège — Une CTE n'est pas un résultat stocké

Depuis PostgreSQL 12, une CTE lue une seule fois est **fusionnée** dans la requête principale, comme une sous-requête : l'optimiseur peut y pousser les filtres. Une CTE lue plusieurs fois est calculée une fois. On peut forcer le comportement avec `WITH x AS MATERIALIZED (...)` ou `NOT MATERIALIZED`. Et une CTE ne survit pas à la requête : pour réutiliser un résultat dans plusieurs requêtes, il faut une vue ou une table temporaire.

2.1.3 Les vues : écrire une règle une seule fois

Le calcul du retard sera utilisé par presque toutes les analyses de ce chapitre. Le réécrire à chaque fois, c'est risquer qu'une requête le calcule différemment des autres. Une **vue** enregistre la requête sous un nom ; on l'interroge ensuite comme une table :

```sql
CREATE OR REPLACE VIEW v_situation_prets AS
SELECT ... ;                                          -- la règle, écrite une fois

SELECT count(*) FROM v_situation_prets WHERE en_defaut;   -- utilisée partout
```

La vue ne stocke aucune donnée : elle est recalculée à chaque interrogation, donc toujours à jour.

2.1.4 Les agrégats conditionnels : FILTER

Pour compter ou sommer seulement certaines lignes d'un groupe, la clause `FILTER` s'ajoute à n'importe quel agrégat :

```sql
SELECT count(*)                                        AS nb_prets,
       count(*)     FILTER (WHERE statut_pret = 'en_defaut') AS nb_defauts,
       sum(montant_demande) FILTER (WHERE note_risque = 'A') AS montant_note_a
FROM demandes_pret;
```

C'est l'équivalent, plus lisible, de `sum(CASE WHEN condition THEN x END)`. On l'utilisera aussi pour lire les paramètres en une ligne : `max(valeur) FILTER (WHERE cle = 'lgd')`.

2.1.5 Les fonctions de fenêtre

Imagine cette situation...

Au marathon de Clermont, chaque coureur reçoit deux classements : son rang au général et son rang dans sa catégorie d'âge. Pour les calculer, on ne « regroupe » pas les coureurs en une ligne par catégorie : chaque coureur garde **sa** ligne, avec son temps, et on lui **ajoute** une information calculée en regardant les autres coureurs de sa catégorie.

Définition — Fonction de fenêtre

Une fonction calculée pour chaque ligne à partir d'un ensemble de lignes liées à elle, la **fenêtre**, **sans regrouper les lignes**. Elle se reconnaît au mot-clé `OVER`.

La différence avec `GROUP BY` est fondamentale :

`GROUP BY note_risque`

note	count(*)
A	3
B	2

5 prêts → 2 lignes

`count(*) OVER (PARTITION BY note_risque)`

prêt	note	nb dans la note
101	A	3
102	A	3
103	A	3
104	B	2
105	B	2

5 prêts → 5 lignes

2.1.5.1 L'anatomie d'une fenêtre

```
fonction(...) OVER (
    PARTITION BY colonnes      -- découpe en groupes indépendants (facultatif)
    ORDER BY colonnes          -- ordonne les lignes dans chaque groupe (facultatif)
    ROWS BETWEEN ... AND ...   -- le cadre : quelles lignes autour de la ligne courante (
    ↪ facultatif)
)
```

Famille	Fonctions	Usage dans le projet
Rangs	`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE(n)`	déciles de risque, plus grosses expositions
Décalage	`LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`	évolution d'un mois sur l'autre, matrice de transition
Agrégats fenêtrés	`SUM`, `AVG`, `COUNT`, `MIN`, `MAX OVER (...)`	cumuls, moyennes mobiles, parts du total

2.1.5.2 Les rangs : ROW_NUMBER, RANK, DENSE_RANK

Classons cinq prêts par PD décroissante (exemple illustratif, avec un ex æquo) :

prêt	PD	`ROW_NUMBER()`	`RANK()`	`DENSE_RANK()`
P1	0,30	1	1	1
P2	0,25	2	2	2
P3	0,25	3	2	2
P4	0,10	4	**4**	**3**
P5	0,05	5	5	4

— `ROW_NUMBER` numérote sans jamais répéter ;

— `RANK` donne le même rang aux ex æquo, puis **saute** des rangs (1, 2, 2, 4) : c'est le classement sportif ;

— `DENSE_RANK` donne le même rang aux ex æquo **sans sauter** (1, 2, 2, 3).

Piège — ROW_NUMBER et les ex æquo

Entre P2 et P3, qui reçoit le numéro 2 ? `ROW_NUMBER` en choisit un au hasard, et le choix peut changer d'une exécution à l'autre. Pour un résultat reproductible, ajoute toujours un critère de départage : `ORDER BY probabilite_defaut DESC, pret_id`. Question d'entretien classique : « les trois plus gros prêts en défaut de chaque note » se fait avec `RANK` (les ex æquo sont tous gardés) ou `ROW_NUMBER` (exactement trois lignes), selon ce que veut le métier.

2.1.5.3 NTILE : découper en groupes de même taille

`NTILE(n)` répartit les lignes ordonnées en n groupes aussi égaux que possible, numérotés de 1 à n.

Calcul à la main — NTILE(4) sur 10 prêts

$10 = 4 \times 2 + 2$: chaque groupe reçoit 2 prêts, et les 2 prêts restants vont aux **premiers** groupes. Les tailles sont donc 3, 3, 2, 2.

Sur ta base, les déciles de l'échantillon de test : $6\,482 = 10 \times 648 + 2$, donc les déciles 1 et 2 contiennent 649 prêts et les huit autres 648. C'est exactement ce que tu obtiendras dans l'exercice 2.3.

2.1.5.4 LAG et LEAD : regarder la ligne d'avant ou d'après

`LAG(x)` renvoie la valeur de `x` sur la ligne **précédente** de la fenêtre, `LEAD(x)` sur la ligne **suivante**. Exemple réel : le retard de paiement du prêt 61, échéance par échéance.

```sql
SELECT num_echeance, date_echeance, date_paiement,
       date_paiement - date_echeance                                  AS retard,
       LAG(date_paiement - date_echeance) OVER (ORDER BY num_echeance)  AS retard_precedent
FROM echeancier_remboursements
WHERE pret_id = 61 AND num_echeance BETWEEN 13 AND 19
ORDER BY num_echeance;
```

num_echeance	date_echeance	date_paiement	retard	retard_precedent
13	15/03/2026	13/03/2026	−2	NULL
14	15/04/2026	14/04/2026	−1	−2
15	15/05/2026	18/05/2026	3	−1
16	15/06/2026	15/06/2026	0	3
17	15/07/2026	03/08/2026	19	0
18	15/08/2026	NULL	NULL	19
19	15/09/2026	15/09/2026	0	NULL

Trois remarques : la première ligne n'a pas de précédent (NULL) ; un retard négatif est un paiement en avance ; et l'échéance 18, impayée, a un retard NULL, que la ligne 19 hérite comme « précédent ». `LAG(x, 1, 0)` remplace l'absence de ligne précédente par 0, mais pas une valeur NULL existante.

2.1.5.5 Les agrégats fenêtrés et le cadre

Avec `ORDER BY`, un agrégat fenêtré devient **cumulatif**. Le **cadre** (`ROWS BETWEEN`) précise exactement quelles lignes entrent dans le calcul (figure 2.2).

```sql
sum(encaisse) OVER (ORDER BY mois)                                  -- cumul depuis le
    ↪ début
avg(encaisse) OVER (ORDER BY mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) -- moyenne mobile
    ↪ 3 mois
sum(encaisse) OVER ()                                               -- total de toutes
    ↪ les lignes
```

Piège — Le cadre par défaut est RANGE, pas ROWS

Avec un `ORDER BY` et sans cadre explicite, PostgreSQL utilise `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. `RANGE` inclut les lignes **ex æquo** avec la ligne courante : si deux lignes ont la même date, le « cumul » de la première contient déjà la seconde. Pour un cumul ligne à ligne, écris `ROWS` explicitement. Même cause, autre symptôme : `LAST_VALUE(x) OVER (ORDER BY ...)` renvoie la ligne courante, pas la dernière, sauf à écrire `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

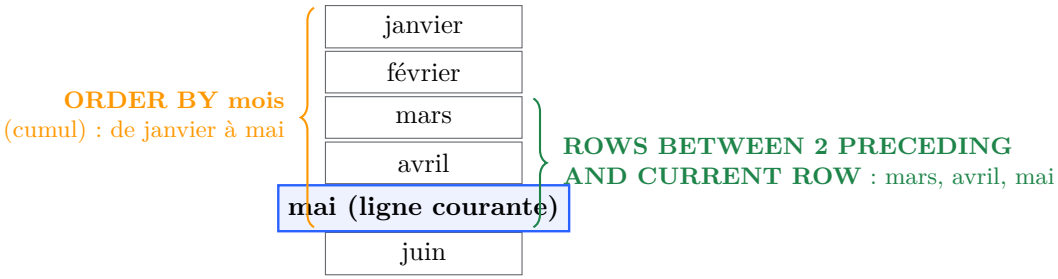


FIGURE 2.2 – Deux cadres de fenêtre pour la ligne de mai.

Quand plusieurs fonctions partagent la même fenêtre, la clause `WINDOW` évite de la répéter :

```
SELECT LAG(retard, 1) OVER w, LAG(retard, 2) OVER w
FROM ...
WINDOW w AS (PARTITION BY pret_id ORDER BY num_echeance);
```

2.1.6 Les sous-totaux : ROLLUP

`GROUP BY ROLLUP (stage)` produit une ligne par stage **plus une ligne de total**, où la colonne `stage` vaut NULL. On l'affiche avec `coalesce(stage::text, 'Total')`. Avec plusieurs colonnes, `ROLLUP (a, b)` donne les sous-totaux par `a` puis le total général ; la fonction `GROUPING(a)` permet de distinguer un vrai NULL d'une ligne de sous-total.

2.1.7 Les indicateurs du risque, traduits en SQL

2.1.7.1 Le retard et l'encours de chaque prêt

La définition du DPD (partie 2) se traduit directement :

```
coalesce(date_ref - min(e.date_echeance)
          FILTER (WHERE e.date_echeance <= date_ref AND e.montant_paye < e.montant_du), 0) AS
    ↪ jours_retard
```

Le `min` cherche l'échéance la plus ancienne parmi celles qui sont échues et non soldées ; s'il n'y en a aucune, il vaut NULL, et `coalesce` le remplace par 0. L'encours (capital non remboursé) tient compte des paiements partiels au prorata : $\sum \text{part_capital} \times (1 - \text{montant_payé}/\text{montant_dû})$.

2.1.7.2 L'AUC et le Gini, sans bibliothèque

Définition — AUC

L'AUC (aire sous la courbe ROC) est la probabilité qu'un prêt en défaut, tiré au hasard, ait une PD **plus élevée** qu'un prêt sain tiré au hasard. 0,5 = hasard, 1 = classement parfait. Le **Gini** vaut $2 \times AUC - 1$.

Cette définition donne une formule calculable en SQL, celle de la statistique de Mann–Whitney. On range tous les prêts par PD croissante (rang 1 = la plus petite PD). Avec n_1 défauts et n_0 prêts sains :

$$AUC = \frac{\sum \text{rangs des défauts} - \dfrac{n_1(n_1+1)}{2}}{n_1 \times n_0}$$

Pourquoi ? La somme des rangs des défauts compte, pour chaque défaut, le nombre de prêts classés en dessous de lui (plus lui-même). En retirant $n_1(n_1+1)/2$, on enlève les défauts classés sous d'autres défauts : il reste exactement le nombre de paires (défaut, sain) bien ordonnées, que l'on divise par le nombre total de paires $n_1 n_0$.

Calcul à la main — l'AUC de cinq prêts

Deux défauts (D1 : PD 0,9 ; D2 : PD 0,4) et trois prêts sains (S1 : 0,6 ; S2 : 0,3 ; S3 : 0,1).

1. **Par les paires.** 6 paires (défaut, sain). D1 dépasse S1, S2, S3 : 3 paires bien ordonnées. D2 dépasse S2 et S3, pas S1 : 2 paires. $AUC = 5/6 = 0{,}833$.
2. **Par les rangs.** Par PD croissante : S3 = 1, S2 = 2, D2 = 3, S1 = 4, D1 = 5. Somme des rangs des défauts : $3 + 5 = 8$. $AUC = (8 - 2 \times 3/2)/(2 \times 3) = 5/6 = 0{,}833$.
3. Gini $= 2 \times 0{,}833 - 1 = 0{,}667$.

En cas d'ex æquo, chaque prêt reçoit le **rang moyen** de son groupe.

2.1.7.3 La courbe de gains, la matrice de transition, l'analyse par génération

— **Courbe de gains** : on classe les prêts en déciles de PD décroissante et on mesure la part cumulée des défauts « captés » par les premiers déciles. Si refuser les 10 % les plus risqués évite 45 % des défauts, le modèle est très discriminant.

— **Matrice de transition** (*roll rates*) : pour chaque tranche de retard en fin de mois, la répartition des prêts dans les tranches le mois suivant. Elle montre la dynamique du risque : quelle part des prêts à 31–60 jours se régularise, quelle part s'aggrave.

— **Analyse par génération** (*vintage*) : on regroupe les prêts par trimestre d'octroi et on suit leur taux de défaut cumulé après 6, 12, 18, 24 mois d'ancienneté (*months on book*, MOB). Une génération qui fait défaut plus vite que les précédentes signale un relâchement de la politique d'octroi.

Piège — Comparer des générations qui n'ont pas le même âge

Une génération accordée il y a 3 mois n'a pas encore eu le temps de faire défaut : son taux de défaut « après 12 mois » n'existe pas. Si on le calcule quand même, les jeunes générations paraissent excellentes. Pour chaque horizon de h mois, on ne garde que les prêts **observés pendant au moins h mois**.

2.1.8 Performance : filtrer avant de joindre

Une requête analytique sur 1,1 million de mensualités peut prendre des secondes ou des minutes selon la façon dont elle est écrite. La règle la plus rentable : **réduire les données avant de les joindre**. Dans la matrice de transition (exercice 2.6), on joint chaque fin de mois à chaque prêt et à ses échéances : en ne gardant d'abord que les échéances qui *peuvent* être impayées pendant la période étudiée, la requête passe de 7 à 3 secondes. `EXPLAIN ANALYZE` permet de vérifier l'effet de chaque réécriture.

2.2 Exercices guidés

Exercice 2.1 — La situation de chaque prêt à la date d'arrêté

a) Crée la vue `v_situation_prets` : pour chaque prêt accordé avant la date de référence, son retard (DPD), son encours, ses impayés échus, le nombre de mois restants et un booléen `en_defaut` (retard supérieur au seuil). Les paramètres sont lus dans `parametres_risque`.

b) Donne la répartition des prêts **non soldés** par tranche de retard (à jour, 1–30, 31–60, 61–90, plus de 90 jours) : nombre, encours, part de l'encours total.

c) Calcule le PAR30 et le PAR90.

Correction de l'exercice 2.1

a) La vue. Une première CTE lit les paramètres en une ligne (agrégats `FILTER`). La seconde joint les prêts à leurs échéances et calcule tout par prêt. Le `CROSS JOIN` ajoute la ligne des paramètres à chaque ligne : c'est le moyen simple de rendre une constante disponible partout.

```
CREATE OR REPLACE VIEW v_situation_prets AS
WITH arrete AS (
    SELECT max(valeur_date) FILTER (WHERE cle = 'date_reference')  AS date_ref,
           max(valeur)      FILTER (WHERE cle = 'seuil_defaut_jours') AS seuil_defaut
    FROM parametres_risque
),
calcul AS (
    SELECT p.pret_id, p.client_id, p.note_risque, p.motif_pret, p.duree_mois, p.date_decision,
           a.date_ref, a.seuil_defaut,
           -- DPD : jours écoulés depuis la plus ancienne échéance échue et non soldée (0 si tout est à jour)
           coalesce(a.date_ref - min(e.date_echeance)
                    FILTER (WHERE e.date_echeance <= a.date_ref AND e.montant_paye < e.montant_du), 0) AS
     ↪ jours_retard,
           -- encours : capital pas encore remboursé (un paiement partiel couvre capital et intérêts au prorata)
           round(sum(e.part_capital * (1 - e.montant_paye / e.montant_du)), 2)                          AS
     ↪ encours,
           coalesce(sum(e.montant_du - e.montant_paye) FILTER (WHERE e.date_echeance <= a.date_ref), 0) AS
     ↪ impayes,
           count(*) FILTER (WHERE e.date_echeance > a.date_ref)                                         AS
     ↪ mois_restants
    FROM demandes_pret p
    JOIN echeancier_remboursements e USING (pret_id)
    CROSS JOIN arrete a
    WHERE p.statut_dossier = 'accorde' AND p.date_decision <= a.date_ref
    GROUP BY p.pret_id, a.date_ref, a.seuil_defaut    -- p.pret_id est la clé : les autres colonnes de p en
     ↪ dépendent
)
SELECT pret_id, client_id, note_risque, motif_pret, duree_mois, date_decision, date_ref,
       jours_retard, encours, impayes, mois_restants,
       jours_retard > seuil_defaut AS en_defaut       -- « plus de 90 jours » (art. 178 CRR)
FROM calcul;
```

Remarque sur le GROUP BY. On sélectionne `p.note_risque`, `p.duree_mois`…sans les mettre dans le `GROUP BY`. C'est permis parce que `p.pret_id`, clé primaire de `demandes_pret`, y figure : PostgreSQL sait que les autres colonnes de `p` en dépendent.

b) Les tranches. La CTE classe chaque prêt dans une tranche (les numéros devant les libellés donnent l'ordre de tri). `sum(sum(encours)) OVER ()` calcule le total de toutes les tranches sans perdre le détail : c'est une fonction de fenêtre appliquée au résultat du `GROUP BY`.

```
WITH tranches AS (
    SELECT encours,
           CASE WHEN jours_retard = 0  THEN '1. À jour'
```

```sql
                WHEN jours_retard <= 30 THEN '2. 1 à 30 jours'
                WHEN jours_retard <= 60 THEN '3. 31 à 60 jours'
                WHEN jours_retard <= 90 THEN '4. 61 à 90 jours'
                ELSE                         '5. Plus de 90 jours' END AS tranche
    FROM v_situation_prets
    WHERE encours > 0                                -- les prêts soldés ne portent plus de risque
)
SELECT tranche, count(*) AS nb_prets, sum(encours) AS encours,
       -- SUM(SUM(...)) OVER () : le total de toutes les tranches, sans perdre le détail
       round(100 * sum(encours) / sum(sum(encours)) OVER (), 2) AS part_encours_pct
FROM tranches
GROUP BY tranche
ORDER BY tranche;
```

c) PAR30 et PAR90.

```sql
SELECT sum(encours)                                                              AS encours_total,
       round(100 * sum(encours) FILTER (WHERE jours_retard > 30) / sum(encours), 2) AS par30_pct,
       round(100 * sum(encours) FILTER (WHERE en_defaut)         / sum(encours), 2) AS par90_pct
FROM v_situation_prets
WHERE encours > 0;
```

Ce que tu dois obtenir

Tranche	Prêts	Encours (€)	Part de l'encours
1. À jour	13 480	73 884 266,09	55,15 %
2. 1 à 30 jours	714	3 948 710,84	2,95 %
3. 31 à 60 jours	201	1 081 975,60	0,81 %
4. 61 à 90 jours	18	69 932,39	0,05 %
5. Plus de 90 jours	7 088	54 978 554,44	41,04 %

Encours total : 133 963 439,36 € PAR30 : **41,90 %** PAR90 : **41,04 %**

Les 10 909 prêts soldés (encours nul) sont exclus : 13 480+714+201+18+7 088 = 21 501 prêts vivants.

Un PAR90 de 41 % : réaliste ?

Non : dans une banque de crédit à la consommation, le PAR90 se mesure plutôt en quelques pour cent. Deux raisons à cet écart, qu'il faut savoir expliquer. D'abord, le jeu de données contient 21,9 % de défauts, bien plus qu'un portefeuille réel. Ensuite, un prêt en défaut cesse de payer et garde tout son encours, alors que les prêts sains s'amortissent et 10 909 sont déjà soldés : dans l'encours *restant*, les défauts pèsent donc de plus en plus lourd. Le calcul est juste ; c'est le portefeuille qui est atypique.

Exercice 2.2 — Provisions IFRS 9 : stage, EAD, PD, LGD et perte attendue

Pour chaque prêt non soldé, détermine son stage (seuils lus dans les paramètres), son EAD (l'encours), la PD du modèle champion, la PD utilisée selon le stage ($PD_m = 1 - (1 - PD)^{m/n}$) et l'ECL. Présente le nombre de prêts, l'EAD, la PD moyenne, l'ECL et le taux de couverture (ECL / EAD) par stage, avec une ligne de total. Vérifie enfin qu'aucun prêt n'est oublié faute de score.

Correction de l'exercice 2.2

Trois CTE, trois étapes : les paramètres ; le stage de chaque prêt ; la PD utilisée. La requête finale agrège avec `ROLLUP`.

```sql
WITH parametres AS (
    SELECT max(valeur) FILTER (WHERE cle = 'lgd')                AS lgd,
           max(valeur) FILTER (WHERE cle = 'seuil_defaut_jours') AS seuil_defaut,
           max(valeur) FILTER (WHERE cle = 'seuil_stage2_jours') AS seuil_stage2
    FROM parametres_risque
),
stages AS (
    SELECT s.pret_id, s.encours AS ead, s.duree_mois,
           greatest(s.mois_restants, 1) AS mois_restants,       -- un prêt échu mais impayé : au moins 1 mois
           ps.probabilite_defaut        AS pd_vie_entiere,
           CASE WHEN s.jours_retard > prm.seuil_defaut THEN 3   -- défaut avéré
                WHEN s.jours_retard > prm.seuil_stage2 THEN 2   -- dégradation significative du risque
                ELSE 1 END AS stage
    FROM v_situation_prets s
    JOIN predictions_scoring_ml ps ON ps.pret_id = s.pret_id AND ps.est_champion
    CROSS JOIN parametres prm
    WHERE s.encours > 0
),
pertes AS (
    -- La PD du modèle porte sur toute la durée du prêt. Avec un risque mensuel constant,
    -- la probabilité de défaut sur m mois vaut 1 - (1 - PD)^(m / durée).
    SELECT st.*,
           CASE stage
               WHEN 3 THEN 1                                                                  -- défaut : PD = 1
               WHEN 2 THEN 1 - (1 - pd_vie_entiere) ^ (mois_restants::numeric / duree_mois)   -- vie restante
               ELSE        1 - (1 - pd_vie_entiere) ^ (least(mois_restants, 12)::numeric / duree_mois)  -- 12
    ↪ mois
           END AS pd_utilisee
    FROM stages st
)
SELECT coalesce(stage::text, 'Total')                    AS stage_ifrs9,
       count(*)                                           AS nb_prets,
       sum(ead)                                           AS ead,
       round(100 * avg(pd_utilisee), 2)                   AS pd_moyenne_pct,
       round(sum(pd_utilisee * prm.lgd * ead), 2)         AS ecl,
       round(100 * sum(pd_utilisee * prm.lgd * ead) / sum(ead), 2) AS taux_couverture_pct
FROM pertes
CROSS JOIN parametres prm
GROUP BY ROLLUP (stage)                                   -- ROLLUP ajoute la ligne de total
ORDER BY stage;

-- Contrôle qualité : un prêt accordé sans score du modèle champion disparaîtrait du calcul (jointure interne)
SELECT count(*) AS prets_sans_score
FROM v_situation_prets s
WHERE NOT EXISTS (SELECT 1 FROM predictions_scoring_ml ps WHERE ps.pret_id = s.pret_id AND ps.est_champion);
```

Deux détails qui comptent. `mois_restants::numeric` : sans conversion, la division de deux entiers serait entière ($12/36 = 0$). `greatest(mois_restants, 1)` : un prêt dont toutes les échéances sont passées mais impayées a 0 mois restant, ce qui annulerait sa PD.

Ce que tu dois obtenir

Stage	Prêts	EAD (€)	PD moyenne	ECL (€)	Couverture
1	14 194	77 832 976,93	2,20 %	856 830,45	1,10 %
2	219	1 151 907,99	3,80 %	29 949,20	2,60 %
3	7 088	54 978 554,44	100,00 %	24 740 349,50	45,00 %
Total	21 501	133 963 439,36	34,45 %	**25 627 129,15**	19,13 %

Prêts sans score : **0**.

Lecture. Le stage 3 représente 96,5 % des provisions ; son taux de couverture vaut exactement la LGD (45 %), puisque sa PD vaut 100 %. Les 14 194 prêts du stage 1 (58 % de l'EAD) ne coûtent que 1,10 % de leur encours. Le stage 2 compte 219 prêts : les 201 + 18 prêts des tranches 31–60 et 61–90 jours de l'exercice précédent.

Exercice 2.3 — Le modèle classe-t-il bien ? Déciles de risque sur l'échantillon test

Sur les prêts de l'échantillon **test**, découpe les prédictions du modèle champion en 10 déciles de PD décroissante (décile 1 = les plus risqués). Pour chaque décile : nombre de prêts, PD moyenne prédite, taux de défaut observé, nombre de défauts et part cumulée des défauts captés.

Correction de l'exercice 2.3

```sql
WITH test AS (
    SELECT ps.probabilite_defaut AS pd, s.en_defaut::int AS defaut
    FROM predictions_scoring_ml ps
    JOIN echantillons_modele em USING (pret_id)
    JOIN v_situation_prets s   USING (pret_id)
    WHERE ps.est_champion AND em.echantillon = 'test'    -- jamais sur l'apprentissage : résultats trop beaux
),
deciles AS (
    SELECT pd, defaut, NTILE(10) OVER (ORDER BY pd DESC) AS decile   -- décile 1 = les 10 % les plus risqués
    FROM test
)
SELECT decile,
       count(*)                    AS nb_prets,
       round(100 * avg(pd), 2)     AS pd_predite_pct,
       round(100 * avg(defaut), 2) AS taux_defaut_observe_pct,
       sum(defaut)                 AS nb_defauts,
       -- courbe de gains : part cumulée des défauts captés en refusant les déciles 1 à k
       round(100.0 * sum(sum(defaut)) OVER (ORDER BY decile) / sum(sum(defaut)) OVER (), 1) AS
    ↪ defauts_captes_cumules_pct
FROM deciles
GROUP BY decile
ORDER BY decile;
```

Les astuces. `en_defaut::int` convertit le booléen en 0 ou 1 : sa moyenne est un taux, sa somme un nombre de défauts. La dernière colonne empile deux niveaux : `sum(defaut)` agrège chaque décile, puis `sum(...) OVER (ORDER BY decile)` cumule les déciles.

Ce que tu dois obtenir

Décile	Prêts	PD prédite	Défaut observé	Défauts	Défauts captés (cumul)
1	649	97,55 %	100,00 %	649	45,8 %
2	649	67,59 %	64,41 %	418	75,2 %
3	648	20,06 %	15,28 %	99	82,2 %
4	648	10,97 %	12,65 %	82	88,0 %
5	648	7,47 %	10,03 %	65	92,6 %
6	648	5,23 %	5,56 %	36	95,1 %
7	648	3,66 %	5,86 %	38	97,8 %
8	648	2,31 %	3,55 %	23	99,4 %
9	648	1,10 %	1,08 %	7	99,9 %
10	648	0,24 %	0,15 %	1	100,0 %

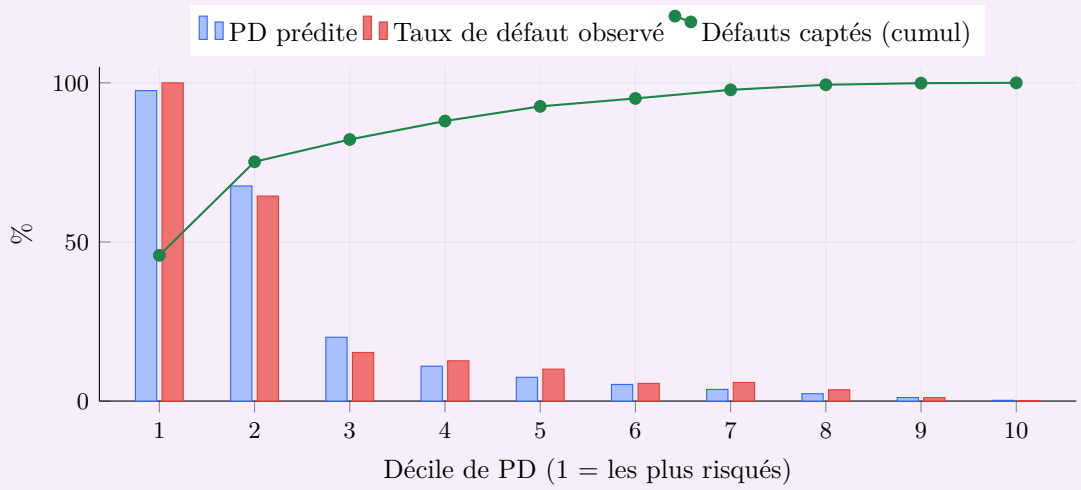


FIGURE 2.3 – Déciles du modèle champion sur les 6 482 prêts de test : PD prédite et défaut observé sont proches décile par décile, et les deux premiers déciles concentrent 75 % des défauts.

Lecture. Deux qualités distinctes apparaissent. La **discrimination** : le taux de défaut baisse régulièrement du décile 1 au décile 10, et refuser les 10 % les plus risqués éviterait 45,8 % des défauts. La **calibration** : dans chaque décile, la PD prédite est proche du taux observé (67,6 % contre 64,4 % dans le décile 2), ce qui autorise à utiliser la PD comme une vraie probabilité, par exemple dans le calcul des provisions.

Exercice 2.4 — Encaissements mensuels : cumul, moyenne mobile et évolution

Calcule les sommes encaissées chaque mois depuis octobre 2025 (d'après la date de paiement), puis pour chaque mois : le cumul depuis octobre, la moyenne mobile sur trois mois et l'évolution en pourcentage par rapport au mois précédent.

Correction de l'exercice 2.4

```sql
WITH mensuel AS (
    SELECT date_trunc('month', date_paiement)::date AS mois, sum(montant_paye) AS encaisse
    FROM echeancier_remboursements
    WHERE date_paiement >= DATE '2025-10-01'
    GROUP BY 1
)
SELECT mois, encaisse,
       sum(encaisse) OVER (ORDER BY mois)                                          AS cumul_depuis_octobre,
       round(avg(encaisse) OVER (ORDER BY mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS
    ↪ moyenne_mobile_3_mois,
       round(100 * (encaisse / LAG(encaisse) OVER (ORDER BY mois) - 1), 1)         AS evolution_pct
FROM mensuel
ORDER BY mois;
```

Pourquoi `date_paiement IS NOT NULL` n'est pas écrit ? Le filtre `>= DATE '2025-10-01'` élimine déjà les NULL : une comparaison avec NULL n'est jamais vraie. Le cumul utilise ici le cadre par défaut (`RANGE`) sans risque, car chaque mois n'apparaît qu'une fois.

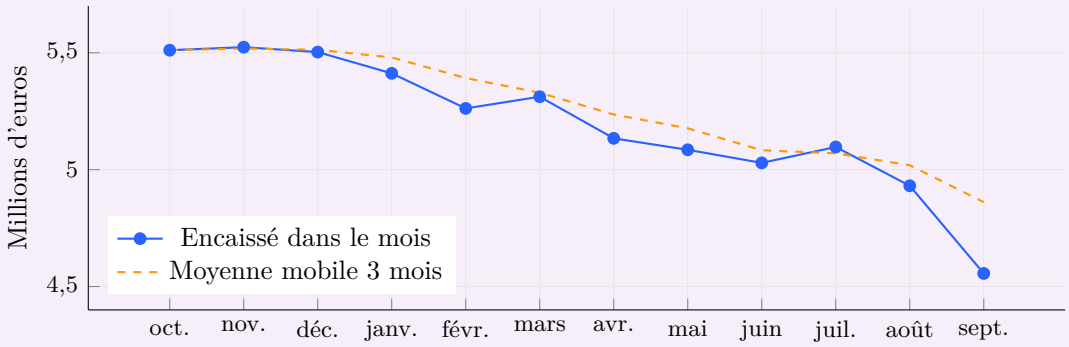


FIGURE 2.4 – Encaissements mensuels d'octobre 2025 à septembre 2026 (62,36 millions d'euros au total).

Lecture. Les encaissements baissent lentement : le portefeuille s'amortit, les prêts en défaut ne paient plus, et les nouveaux prêts se raréfient (les dernières demandes du jeu simulé datent de fin juin 2026). La chute de septembre ($-7{,}6$ %) est en partie un **effet de bord** : les mensualités de fin septembre payées avec quelques jours de retard ne seront encaissées qu'en octobre, après la date d'arrêté. Réflexe à avoir en entretien : *la dernière période d'une série est souvent incomplète.*

2.3 Exercices à faire seul

Exercice 2.5 — Champion contre challengers : AUC, Gini, rappel et taux de refus

Sur l'échantillon test, calcule pour **chacun des trois modèles** : l'AUC (formule des rangs, avec rangs moyens en cas d'ex æquo), le Gini, le rappel (part des défauts que le modèle aurait refusés), le taux de refus et le taux de défaut parmi les prêts qu'il aurait accordés. Le champion mérite-t-il sa place ?

Indices

- Le rang par modèle : `RANK() OVER (PARTITION BY version_modele ORDER BY pd)`.
- Le rang moyen d'un groupe d'ex æquo de taille t dont le premier rang est R vaut $R + (t-1)/2$; t s'obtient avec `count(*) OVER (PARTITION BY version_modele, pd)`.
- Puis un `GROUP BY version_modele` avec des agrégats `FILTER`. Attention aux divisions entières.

Exercice 2.6 — La matrice de transition entre fins de mois, avec LEAD

Pour chaque fin de mois de septembre 2025 à septembre 2026, calcule le retard de chaque prêt **à cette date** (une échéance est impayée à la date D si elle est échue à D et n'était pas soldée à D). Classe les prêts en tranches, puis donne, pour chaque tranche de départ, la répartition (en %) des prêts dans les tranches **le mois suivant**. Écarte les prêts terminés et soldés.

Indices

— Les fins de mois : `generate_series` sur les premiers jours de mois, puis `+ INTERVAL '1 month - 1 day'`.

— « Non soldée à D » : `date_paiement IS NULL OR date_paiement > D`.

— Un prêt sans impayé n'a aucune échéance à joindre : il faut un `LEFT JOIN` pour le garder (avec un retard de 0).

— La tranche du mois suivant : `LEAD(tranche) OVER (PARTITION BY pret_id ORDER BY date_arrete)`.

— Performance : filtre les échéances inutiles *avant* la jointure.

Exercice 2.7 — L'analyse par génération (vintage)

Regroupe les prêts accordés par trimestre d'octroi. Pour chaque génération, calcule le taux de défaut cumulé après 6, 12, 18 et 24 mois (MOB). La date d'entrée en défaut d'un prêt est la date d'échéance de sa première mensualité restée impayée plus de 90 jours, plus 91 jours. Ajoute l'écart du MOB 12 avec la génération précédente. Que remarques-tu sur les générations récentes ?

Indices

— Une échéance a été impayée plus de 90 jours si `coalesce(date_paiement, date_ref) - date_echeance > 90`.

— `date_trunc('quarter', date_decision)` donne le trimestre.

— Les horizons : `CROSS JOIN (VALUES (6), (12), (18), (24)) AS h (mois)`, en ne gardant que les prêts observés au moins `h.mois` mois (relis le piège des générations).

Exercice 2.8 — Classements et signal précoce

a) Les trois plus grosses expositions en défaut de chaque note de risque, avec le nom du client et la part de chacune dans l'encours en défaut de sa note.

b) Le **signal précoce** : les 20 prêts encore à jour ou presque (retard actuel ≤ 30 jours, encours non nul) dont le retard de paiement **augmente** sur chacune de leurs trois dernières échéances payées, la dernière étant payée en retard. Trie par retard décroissant puis par encours.

Indices

— a) `RANK() OVER (PARTITION BY note_risque ORDER BY encours DESC)` dans une CTE, puis `WHERE rang <= 3` dans la requête suivante.

— b) `LAG(retard, 1)` et `LAG(retard, 2)` sur une même fenêtre (clause `WINDOW`), et `ROW_NUMBER() OVER (PARTITION BY pret_id ORDER BY num_echeance DESC) = 1` pour la dernière échéance payée.

2.4 Synthèse pour les entretiens

Résumé du chapitre

1. **Ordre d'exécution** : FROM, WHERE, GROUP BY, HAVING, SELECT (fenêtres), DISTINCT, ORDER BY, LIMIT. On filtre une fenêtre dans une requête englobante (pas de `QUALIFY` dans PostgreSQL).
2. **CTE** pour découper, **vue** pour réutiliser une règle, **FILTER** pour les agrégats conditionnels.
3. **Fenêtres** : elles gardent les lignes. `PARTITION BY` découpe, `ORDER BY` ordonne, le cadre (`ROWS`) délimite.
4. **Rangs** : `ROW_NUMBER` (unique), `RANK` (saute), `DENSE_RANK` (ne saute pas), `NTILE` (groupes égaux, les premiers reçoivent le reste).
5. **LAG / LEAD** : ligne précédente / suivante ; NULL au bord.
6. **Cadre par défaut** : `RANGE` avec les ex æquo ; écrire `ROWS` pour un cumul ligne à ligne.
7. **Indicateurs** : DPD, PAR, ECL par stage avec `ROLLUP`, déciles et courbe de gains, AUC par les rangs, matrice de transition, générations observées assez longtemps.

Ce que tu sauras dire en entretien

— « Le retard est calculé dans une vue unique, `v_situation_prets` : toutes les analyses utilisent la même règle, celle de l'article 178 du CRR. »

— « Les provisions IFRS 9 tiennent en une requête de trois CTE : 25,6 millions d'euros, dont 96,5 % en stage 3. Le `ROLLUP` donne le total, et un contrôle vérifie qu'aucun prêt n'est sans score. »

— « J'ai calculé l'AUC directement en SQL par la statistique de Mann–Whitney, avec des rangs moyens pour les ex æquo : 0,923 pour le XGBoost contre 0,876 pour la logistique, sur l'échantillon test uniquement. »

— « Les déciles montrent à la fois la discrimination, 75 % des défauts dans les deux premiers déciles, et la calibration : la PD prédite suit le taux observé. »

Questions fréquentes en entretien

Différence entre WHERE et HAVING ? `WHERE` filtre les lignes avant le regroupement, `HAVING` filtre les groupes après.

RANK, DENSE_RANK, ROW_NUMBER ? Voir le tableau des rangs : le traitement des ex æquo les distingue.

Comment obtenir le 2e plus gros prêt de chaque client ? `DENSE_RANK() OVER (PARTITION BY client_id ORDER BY montant DESC)` dans une CTE, puis `WHERE rang = 2`.

Comment calculer une évolution d'un mois sur l'autre ? `LAG(valeur) OVER (ORDER BY mois)`, puis `valeur / LAG(...) - 1`.

Une CTE est-elle plus rapide qu'une sous-requête ? Pas en soi : depuis PostgreSQL 12, elles sont généralement optimisées de la même façon. La CTE gagne en lisibilité.

Chapitre 3

Automatisation métier : triggers et procédures stockées

« Ce qui doit être fait à chaque fois ne doit dépendre de la mémoire de personne. »

Jusqu'ici, la base *stocke* et *répond*. Dans ce chapitre, elle *agit* : elle refuse qu'on modifie un contrat, recalcule le statut d'un prêt dès qu'un paiement arrive, inscrit chaque changement dans un journal, prévient le recouvrement d'un passage en défaut, encaisse les paiements et calcule les provisions de fin de mois. C'est la partie « automatisation » de ton projet, celle qui fait la différence entre un exercice de requêtes et un vrai système.

Tes fichiers pour ce chapitre

- Ton code : `sql/04_chapitre3.sql`. Le corrigé complet : `corriges/chapitre3.sql`.
- Prérequis : chapitres 1 et 2 exécutés (tables `journal_statuts` et `provisions_mensuelles`, vue `v_situation_prets`).

Mise en situation — Le week-end où personne n'est là

Awa Koné n'a payé aucune mensualité depuis le 25 octobre. Le dimanche 24 janvier 2027, son retard atteint 91 jours : elle est en défaut au sens réglementaire. Personne n'est au bureau.

Lundi matin, ta responsable veut que le prêt soit déjà **passé en défaut**, que ce changement soit **tracé** (qui, quand, avec quel retard), que le service de recouvrement ait été **alerté**, et que la provision de fin de mois en tienne compte. Et pendant ce temps, l'application d'encaissement doit continuer à enregistrer les paiements des autres clients, sans jamais pouvoir effacer un paiement ni modifier un contrat.

3.1 Cours et notions importantes

3.1.1 Où placer la logique métier ?

Une règle métier peut vivre dans l'application (Python, Java…) ou dans la base. Les deux choix se défendent ; en entretien, on attend que tu saches les comparer.

	Dans la base (fonctions, triggers)	Dans l'application
Contournement	impossible : tous les accès passent par la base	une requête directe ou un script contourne la règle
Cohérence	une seule implémentation	une par application qui écrit dans la base
Performance	au plus près des données, ensembliste	allers-retours réseau
Tests, versions	plus difficile (outils dédiés)	outils habituels du développeur
Lisibilité	logique « cachée » dans la base	visible dans le code

La règle pratique

On met dans la base ce qui **ne doit jamais être contourné** : l'intégrité des données, la traçabilité, les règles réglementaires simples (le passage en défaut). On laisse dans l'application l'interface, l'orchestration et les calculs complexes (l'entraînement d'un modèle de ML, par exemple).

3.1.2 Les fonctions : SQL ou PL/pgSQL

Une fonction enregistre un calcul sous un nom, avec des paramètres. PostgreSQL en propose deux langages principaux :

— **SQL** : le corps est une simple requête. Idéal pour une formule.

— **PL/pgSQL** : un vrai langage procédural, avec variables, conditions, boucles et gestion des erreurs.

```sql
CREATE OR REPLACE FUNCTION fn_date_reference() RETURNS DATE
LANGUAGE sql STABLE AS $$
    SELECT valeur_date FROM parametres_risque WHERE cle = 'date_reference'
$$;
```

Le corps est encadré par `$$ ... $$` (*dollar quoting*) : tout ce qui est entre les deux est du texte, sans avoir à doubler les apostrophes. Un bloc PL/pgSQL a toujours la même structure :

```sql
CREATE OR REPLACE FUNCTION fn_parametre(p_cle TEXT) RETURNS NUMERIC
LANGUAGE plpgsql STABLE AS $$
DECLARE                                  -- 1. les variables
    v_valeur NUMERIC;
BEGIN                                    -- 2. les instructions
    SELECT valeur INTO STRICT v_valeur FROM parametres_risque WHERE cle = p_cle;
    RETURN v_valeur;
EXCEPTION WHEN NO_DATA_FOUND THEN        -- 3. la gestion des erreurs (facultative)
    RAISE EXCEPTION 'Paramètre inconnu : %', p_cle;
END $$;
```

`INTO STRICT` exige exactement une ligne : zéro ligne lève `NO_DATA_FOUND`, plusieurs lignes `TOO_MANY_ROWS`. Sans `STRICT`, une clé mal orthographiée donnerait silencieusement NULL, et tout calcul qui en dépend deviendrait NULL sans que personne ne s'en aperçoive. `RAISE EXCEPTION` annule l'instruction en cours avec un message ; les `\%` sont remplacés par les valeurs qui suivent.

3.1.2.1 La volatilité : promettre quelque chose à l'optimiseur

Catégorie	Promesse	Dans le projet
`IMMUTABLE`	mêmes arguments ⇒ même résultat, pour toujours ; ne lit pas la base	`fn_pd_ifrs9` (une formule)
`STABLE`	même résultat pendant toute une instruction ; peut lire la base	`fn_parametre`, `fn_jours_retard`
`VOLATILE`	aucune promesse (par défaut) ; peut modifier la base	`random()`, les fonctions de trigger

Ces promesses permettent à PostgreSQL de calculer une fois au lieu de mille. Mentir (déclarer `IMMUTABLE` une fonction qui lit une table) peut donner des résultats faux, par exemple dans un index.

3.1.2.2 Ensembliste ou ligne à ligne

Le retard d'**un** prêt s'écrit naturellement comme une fonction, `fn_jours_retard(pret, date)`. Mais l'appeler pour 32 410 prêts, c'est exécuter 32 410 requêtes. Grâce à l'index partiel, chacune prend quelques microsecondes : 0,25 seconde au total. Sans l'index, chacune relirait 1,1 million de lignes : plus de 20 minutes. La version **ensembliste**, `fn_situation_prets(date)`, calcule tout le portefeuille en une seule requête et renvoie une table (`RETURNS TABLE`).

Piège — La fonction SQL « dépliée »

PostgreSQL recopie le corps d'une fonction SQL simple dans la requête qui l'appelle (*inlining*). C'est souvent un gain, mais si l'argument est lui-même un appel de fonction, comme `fn_situation_prets(fn_date_reference())`, cet argument peut se retrouver recalculé pour chacune des 1,1 million de lignes parcourues : 3,5 secondes au lieu de 0,2. Écrite en PL/pgSQL, la fonction reçoit son argument déjà calculé, une seule fois.

3.1.3 Les triggers

Analogie — le détecteur de fumée

Un détecteur de fumée ne fait rien tant qu'il n'y a pas de fumée. Il n'a pas besoin qu'on pense à lui : dès que l'événement se produit, il réagit, toujours de la même façon. Un trigger est un détecteur installé sur une table.

Définition — Trigger (déclencheur)

Une fonction que PostgreSQL exécute **automatiquement** quand un événement (`INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`) touche une table. La fonction est déclarée `RETURNS trigger` ; le trigger l'attache à la table et précise quand l'appeler.

```
CREATE TRIGGER trg_journal_statut
    AFTER UPDATE OF statut_pret ON demandes_pret        -- quand : après la modification de
    ↪ cette colonne
    FOR EACH ROW                                        -- pour chaque ligne modifiée
    WHEN (OLD.statut_pret IS DISTINCT FROM NEW.statut_pret)   -- seulement si le statut change
    ↪  vraiment
    EXECUTE FUNCTION fn_trg_journal_statut();           -- quoi faire
```

Choix	Options	Ce que ça change
Moment	`BEFORE`, `AFTER`	`BEFORE` peut refuser ou modifier la ligne avant écriture ; `AFTER` voit le résultat final et réagit
Événement	`INSERT`, `UPDATE [OF col]`, `DELETE`	`UPDATE OF` limite le trigger à certaines colonnes
Niveau	`FOR EACH ROW`, `FOR EACH STATEMENT`	une fois par ligne touchée, ou une fois par instruction
Condition	`WHEN (...)`	la fonction n'est même pas appelée si la condition est fausse

Dans une fonction de trigger par ligne, deux variables décrivent la ligne : `OLD` (avant) et `NEW` (après). La valeur renvoyée a un sens précis :

— dans un trigger `BEFORE` par ligne, `RETURN NEW` fait écrire la ligne `NEW` (éventuellement modifiée

par la fonction) ; `RETURN NULL` annule silencieusement l'opération sur cette ligne ; `RAISE EXCEPTION` annule toute l'instruction avec une erreur ;

— dans un trigger `AFTER`, la valeur renvoyée est ignorée : on écrit `RETURN NULL`.

Un trigger `AFTER ... FOR EACH STATEMENT` peut recevoir **toutes** les lignes modifiées par l'instruction, sous forme de tables : les **tables de transition**.

```
AFTER UPDATE ON echeancier_remboursements
REFERENCING OLD TABLE AS anciennes NEW TABLE AS nouvelles   -- toutes les lignes avant / après
FOR EACH STATEMENT EXECUTE FUNCTION ...
```

Une instruction qui enregistre 1 000 paiements déclenche alors **un** recalcul ensembliste au lieu de 1 000.

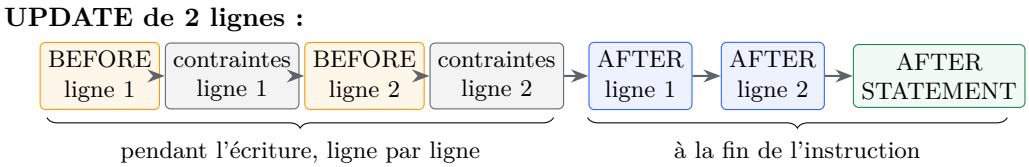


FIGURE 3.1 – L'ordre d'exécution des triggers pour une instruction qui modifie deux lignes. Si plusieurs triggers ont le même moment, ils s'exécutent dans l'ordre alphabétique de leur nom.

Piège — Les triggers en cascade

Un trigger qui modifie une autre table peut en déclencher d'autres : dans ton projet, un paiement déclenche le recalcul du statut, qui déclenche le journal. C'est voulu. Mais un trigger qui modifie **sa propre table** se redéclenche lui-même, à l'infini. Et une erreur dans n'importe quel trigger annule toute l'instruction d'origine. Un trigger est aussi **invisible** pour qui lit une requête : documente-les.

3.1.4 Les procédures stockées

	`FUNCTION`	`PROCEDURE`
Appel	dans une requête : `SELECT f(x)`	instruction seule : `CALL p(x)`
Résultat	renvoie une valeur ou une table	ne renvoie rien (ou des paramètres `INOUT`)
Transactions	ne peut pas valider	peut faire `COMMIT` entre deux étapes (si elle n'est pas appelée dans un bloc `BEGIN`)
Usage typique	un calcul	une opération métier : encaisser, clôturer

Une procédure PL/pgSQL utilise les mêmes briques qu'une fonction, plus quelques outils fréquents :

```
FOR ech IN SELECT ... ORDER BY num_echeance LOOP    -- parcourir le résultat d'une requête
    EXIT WHEN v_restant = 0;                        -- sortir de la boucle
    ...
END LOOP;
GET DIAGNOSTICS v_nb = ROW_COUNT;                   -- nombre de lignes touchées par la
    ↪ dernière instruction
RAISE NOTICE '% statut(s) mis à jour', v_nb;        -- un message d'information (pas une erreur
    ↪ )
```

3.1.4.1 Deux paiements en même temps : le verrou

Imagine cette situation…

Le prêt 61 a une échéance impayée de 704,41 €. Au même instant, deux virements arrivent : 704,41 € par l'application mobile, et 704,41 € par le guichet. Les deux procédures lisent « échéance 18 impayée », et les deux l'affectent à l'échéance 18. Résultat : un paiement est perdu, ou la contrainte `paiement_borne` fait échouer l'un des deux au hasard.

La solution est de **verrouiller le prêt** au début de la procédure : `SELECT ... FROM demandes_pret WHERE pret_id = p FOR UPDATE`. Le second paiement attend que le premier soit validé, puis voit l'échéance 18 déjà payée et passe à la 19. Les paiements du même prêt sont traités l'un après l'autre ; ceux des autres prêts continuent en parallèle.

3.1.5 Le temps qui passe : le traitement de nuit

Comme on l'a vu dans la partie 2, **le passage du temps n'est pas un événement**. Le retard d'Awa franchit 90 jours un dimanche sans qu'aucune ligne ne change : aucun trigger ne peut le détecter. Il faut une procédure, `actualiser_statuts()`, qui recalcule les statuts de tout le portefeuille, et un **ordonnanceur** qui la lance chaque nuit. Sur un serveur, c'est une ligne de `cron` :

```
0 2 * * *  psql -d risk_analytics_db -c "CALL actualiser_statuts();"
```

ou l'extension `pg_cron`, qui planifie directement dans PostgreSQL. Les deux mécanismes partagent la même règle de statut, écrite une seule fois dans une fonction.

3.1.6 Rejouer sans dupliquer : l'idempotence

Une clôture mensuelle peut échouer à mi-parcours, ou devoir être relancée après une correction. Elle doit donner le même résultat qu'on la lance une ou trois fois : elle doit être **idempotente**. L'instruction `INSERT ... ON CONFLICT` (l'« upsert ») le permet :

```
INSERT INTO provisions_mensuelles (date_arrete, pret_id, ...) SELECT ...
ON CONFLICT (date_arrete, pret_id) DO UPDATE             -- la ligne existe déjà : on la
    ↪ remplace
    SET ecl = EXCLUDED.ecl, ...;                          -- EXCLUDED = la ligne qu'on voulait
    ↪ insérer
```

3.1.7 Alerter : LISTEN et NOTIFY

PostgreSQL peut envoyer des messages aux programmes connectés. Un programme s'abonne à un canal avec `LISTEN alertes_defaut` ; une fonction envoie un message avec `pg_notify('alertes_defaut', '...')`. Le message n'est délivré qu'au **moment du COMMIT** : si la transaction est annulée, aucune fausse alerte n'est envoyée. Voici ce que psql affiche quand le prêt d'Awa passe en défaut :

```
Asynchronous notification "alertes_defaut" with payload "{"pret_id" : 32582, "jours_retard" : 98}"
received from server process with PID 87868.
```

Dans une application Python, la boucle `for n in conn.notifies(): ...` de psycopg reçoit ces messages et peut, par exemple, créer une tâche pour le service de recouvrement.

3.1.8 La sécurité : le moindre privilège

Chaque utilisateur ne doit avoir que les droits dont il a besoin. PostgreSQL gère des **rôles** et des droits (`GRANT`, `REVOKE`) :

— l'analyste risque **lit** tout, ne modifie rien : `GRANT SELECT` ;

— l'application d'encaissement n'écrit **jamais** directement dans les tables : elle n'a le droit que d'appeler `enregistrer_paiement`. La procédure est déclarée `SECURITY DEFINER` : elle s'exécute avec les droits de son propriétaire, et fait donc ce que l'application ne peut pas faire elle-même, mais seulement de la façon prévue.

Piège — SECURITY DEFINER sans search_path

Une fonction `SECURITY DEFINER` cherche les tables et fonctions qu'elle utilise dans le `search_path` de l'utilisateur qui l'appelle. Un utilisateur malveillant pourrait créer une fausse table homonyme dans son propre schéma et détourner la procédure, qui s'exécute avec des droits élevés. Il faut toujours figer le chemin : `SET search_path = public, pg_temp`.

3.1.9 La règle du statut

Toute l'automatisation repose sur une règle, évaluée **dans cet ordre** (la première condition vraie l'emporte) :

Ordre	Condition	Nouveau statut	Justification
1	reste dû $= 0$	`solde`	tout est remboursé
2	statut actuel = en défaut	`en_defaut`	pas de sortie automatique (période probatoire)
3	DPD > 90	`en_defaut`	article 178 du CRR
4	DPD > 0	`en_retard`	au moins une échéance impayée
5	sinon	`sain`	

3.2 Exercices guidés

Exercice 3.1 — Des fonctions pour ne jamais réécrire une règle métier

Écris les fonctions suivantes, puis teste-les :

a) `fn_parametre(cle)` (erreur explicite si la clé n'existe pas) et `fn_date_reference()` ;

b) `fn_jours_retard(pret, date)` : le retard d'un prêt à une date quelconque, passée ou actuelle ;

c) `fn_situation_prets(date)` : pour tout le portefeuille à cette date, le retard, l'encours, le reste dû et les mois restants ;

d) `fn_stage_ifrs9(jours_retard)` et `fn_pd_ifrs9(stage, pd, mois_restants, duree)`.

Tests : le retard du prêt 3 ; la concordance entre `fn_jours_retard` et `fn_situation_prets` ; la concordance entre `fn_situation_prets` et la vue du chapitre 2.

Correction de l'exercice 3.1

a) et b) Le retard « à une date quelconque » demande une définition plus générale que celle de la vue : une échéance payée *après* la date D était impayée *à* la date D. D'où la condition `date_paiement IS NULL OR date_paiement > p_date`. C'est elle qui permettra, au chapitre suivant, de reconstituer le passé (provisions des mois précédents, matrice de transition).

```sql
-- a) Lire un paramètre : une erreur explicite vaut mieux qu'un NULL silencieux
CREATE OR REPLACE FUNCTION fn_parametre(p_cle TEXT) RETURNS NUMERIC
LANGUAGE plpgsql STABLE AS $$
DECLARE
    v_valeur NUMERIC;
BEGIN
    SELECT valeur INTO STRICT v_valeur FROM parametres_risque WHERE cle = p_cle;
    RETURN v_valeur;
EXCEPTION WHEN NO_DATA_FOUND THEN
    RAISE EXCEPTION 'Paramètre inconnu : %', p_cle;
END $$;

CREATE OR REPLACE FUNCTION fn_date_reference() RETURNS DATE
LANGUAGE sql STABLE AS $$
    SELECT valeur_date FROM parametres_risque WHERE cle = 'date_reference'
$$;

-- b) Le retard d'UN prêt à une date quelconque (passée ou actuelle)
CREATE OR REPLACE FUNCTION fn_jours_retard(p_pret_id BIGINT, p_date DATE) RETURNS INTEGER
LANGUAGE sql STABLE AS $$
    SELECT coalesce(p_date - min(e.date_echeance), 0)
    FROM echeancier_remboursements e
    WHERE e.pret_id = p_pret_id
      AND e.date_echeance <= p_date
      AND (e.date_paiement IS NULL OR e.date_paiement > p_date)    -- pas encore soldée à cette date
$$;
```

c) La version ensembliste, en PL/pgSQL pour éviter le piège du dépliage. `RETURNS TABLE` déclare les colonnes de sortie ; attention, leurs noms deviennent des variables dans la fonction : on préfixe donc toutes les colonnes des tables (`e.pret_id`) pour éviter toute ambiguïté.

```sql
CREATE OR REPLACE FUNCTION fn_situation_prets(p_date DATE)
RETURNS TABLE (pret_id BIGINT, jours_retard INTEGER, encours NUMERIC, reste_du NUMERIC, mois_restants INTEGER)
LANGUAGE plpgsql STABLE AS $$
BEGIN
    RETURN QUERY
    SELECT e.pret_id,
           coalesce(p_date - min(e.date_echeance)
                    FILTER (WHERE e.date_echeance <= p_date
                              AND (e.date_paiement IS NULL OR e.date_paiement > p_date)), 0),
           -- capital non remboursé à la date : impayé partiel au prorata, soldé après la date = encore dû
           round(sum(e.part_capital * CASE WHEN e.date_paiement IS NULL THEN 1 - e.montant_paye / e.montant_du
                                           WHEN e.date_paiement > p_date THEN 1
                                           ELSE 0 END), 2),
           sum(e.montant_du - e.montant_paye),
           (count(*) FILTER (WHERE e.date_echeance > p_date))::int
    FROM echeancier_remboursements e
    JOIN demandes_pret d ON d.pret_id = e.pret_id
    WHERE d.statut_dossier = 'accorde' AND d.date_decision <= p_date
    GROUP BY e.pret_id;                    -- colonnes préfixées (e., d.) : pas de conflit avec les noms en
     ↪ sortie
END $$;
```

d) Les règles IFRS 9 du chapitre 2, enfin nommées et réutilisables :

```sql
CREATE OR REPLACE FUNCTION fn_stage_ifrs9(p_jours_retard INTEGER) RETURNS SMALLINT
LANGUAGE sql STABLE AS $$
    SELECT (CASE WHEN p_jours_retard > fn_parametre('seuil_defaut_jours') THEN 3
                 WHEN p_jours_retard > fn_parametre('seuil_stage2_jours') THEN 2
                 ELSE 1 END)::smallint
```

```
$$;

CREATE OR REPLACE FUNCTION fn_pd_ifrs9(p_stage SMALLINT, p_pd_vie NUMERIC, p_mois_restants INTEGER, p_duree
    ↪ INTEGER)
RETURNS NUMERIC LANGUAGE sql IMMUTABLE AS $$
    SELECT CASE p_stage
               WHEN 3 THEN 1
               WHEN 2 THEN 1 - (1 - p_pd_vie) ^ (greatest(p_mois_restants, 1)::numeric / p_duree)
               ELSE        1 - (1 - p_pd_vie) ^ (least(greatest(p_mois_restants, 1), 12)::numeric / p_duree)
           END
$$;
```

`fn_stage_ifrs9` est `STABLE` (elle lit les paramètres), `fn_pd_ifrs9` est `IMMUTABLE` (une pure formule).

Les tests. Une fonction non testée n'existe pas. On vérifie que les deux versions du calcul donnent la même chose, et qu'elles concordent avec la vue du chapitre 2 :

```
SELECT fn_jours_retard(3, fn_date_reference()) AS retard_pret_3;

SELECT count(*) AS desaccords_fonction_vs_ensembliste          -- attendu : 0
FROM fn_situation_prets(fn_date_reference()) s
WHERE s.jours_retard <> fn_jours_retard(s.pret_id, fn_date_reference());

SELECT count(*) AS desaccords_vue_vs_fonction                  -- attendu : 0
FROM v_situation_prets v
JOIN fn_situation_prets(fn_date_reference()) s USING (pret_id)
WHERE (v.jours_retard, v.encours) IS DISTINCT FROM (s.jours_retard, s.encours);
```

Ce que tu dois obtenir

Retard du prêt 3 : **1 170 jours**. Désaccords : **0** et **0**.

La comparaison `(a, b) IS DISTINCT FROM (c, d)` compare deux couples d'un coup, en traitant correctement les NULL.

Exercice 3.2 — Trois triggers : protéger, recalculer, tracer

a) **Protéger** l'échéancier : un `UPDATE` ne peut pas modifier les termes du contrat (numéro, date, montants, CRD) ni diminuer un montant payé. Une échéance qui devient entièrement payée sans date de paiement reçoit la date de référence.

b) Écrire la **règle du statut** dans une fonction `fn_regle_statut(statut_actuel, jours_retard, reste_du)`.

c) **Recalculer** : après chaque instruction qui modifie des montants payés, recalculer le statut des prêts concernés, en un seul passage.

d) **Tracer** : chaque changement de `statut_pret` est inscrit dans `journal_statuts`, et un passage en défaut envoie une notification sur le canal `alertes_defaut`.

Correction de l'exercice 3.2

a) Le gardien : BEFORE UPDATE, par ligne. Seul un trigger `BEFORE` peut refuser une modification ou compléter la ligne avant son écriture. La comparaison de deux couples par `IS DISTINCT FROM` détecte n'importe quel changement des termes du contrat, y compris vers NULL.

```
CREATE OR REPLACE FUNCTION fn_trg_proteger_echeance() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
```

```
    IF (NEW.pret_id, NEW.num_echeance, NEW.date_echeance, NEW.montant_du,
        NEW.part_capital, NEW.part_interets, NEW.capital_restant_du)
       IS DISTINCT FROM
       (OLD.pret_id, OLD.num_echeance, OLD.date_echeance, OLD.montant_du,
        OLD.part_capital, OLD.part_interets, OLD.capital_restant_du) THEN
        RAISE EXCEPTION 'Échéance % : les termes du contrat ne se modifient pas', OLD.echeance_id
            USING HINT = 'Un changement de contrat passe par un avenant et un nouvel échéancier.';
    END IF;
    IF NEW.montant_paye < OLD.montant_paye THEN
        RAISE EXCEPTION 'Échéance % : un paiement ne s''efface pas (% → %)',
                        OLD.echeance_id, OLD.montant_paye, NEW.montant_paye
            USING HINT = 'Enregistrer une écriture de régularisation.';
    END IF;
    IF NEW.montant_paye = NEW.montant_du AND NEW.date_paiement IS NULL THEN
        NEW.date_paiement := fn_date_reference();      -- échéance soldée : la date est complétée
    END IF;
    RETURN NEW;                                       -- BEFORE : la ligne renvoyée est celle qui sera écrite
END $$;

DROP TRIGGER IF EXISTS trg_proteger_echeance ON echeancier_remboursements;
CREATE TRIGGER trg_proteger_echeance
    BEFORE UPDATE ON echeancier_remboursements
    FOR EACH ROW EXECUTE FUNCTION fn_trg_proteger_echeance();
```

Test, avec deux instructions qui doivent échouer :

```
UPDATE echeancier_remboursements SET montant_du = 1   WHERE echeance_id = 1;
UPDATE echeancier_remboursements SET montant_paye = 0 WHERE echeance_id = 1;
```

```
ERROR:  Échéance 1 : les termes du contrat ne se modifient pas
HINT:  Un changement de contrat passe par un avenant et un nouvel échéancier.
ERROR:  Échéance 1 : un paiement ne s'efface pas (88.45 → 0.00)
HINT:  Enregistrer une écriture de régularisation.
```

b) La règle, écrite une fois. Elle sera utilisée par le trigger *et* par le traitement de nuit : impossible qu'ils divergent.

```
CREATE OR REPLACE FUNCTION fn_regle_statut(p_statut_actuel VARCHAR, p_jours_retard INTEGER, p_reste_du NUMERIC)
RETURNS VARCHAR LANGUAGE sql STABLE AS $$
    SELECT CASE
               WHEN p_reste_du = 0                                    THEN 'solde'
               -- pas de sortie automatique du défaut : période probatoire et décision humaine
               WHEN p_statut_actuel = 'en_defaut'                     THEN 'en_defaut'
               WHEN p_jours_retard > fn_parametre('seuil_defaut_jours') THEN 'en_defaut'
               WHEN p_jours_retard > 0                                THEN 'en_retard'
               ELSE                                                        'sain'
           END
$$;
```

c) Le recalcul : AFTER UPDATE, par instruction, avec tables de transition. La fonction joint `anciennes` et `nouvelles` pour ne garder que les prêts dont un montant payé a changé, puis met à jour leur statut. La condition `IS DISTINCT FROM` évite d'écrire (et donc de journaliser) un statut inchangé.

```
CREATE OR REPLACE FUNCTION fn_trg_statut_apres_paiement() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    WITH prets_payes AS (
        SELECT DISTINCT n.pret_id
        FROM nouvelles n
        JOIN anciennes a USING (echeance_id)
        WHERE n.montant_paye <> a.montant_paye
    ),
    recalcul AS (
        SELECT d.pret_id,
               fn_regle_statut(d.statut_pret,
```

```
                            fn_jours_retard(d.pret_id, fn_date_reference()),
                            (SELECT sum(e.montant_du - e.montant_paye)
                             FROM echeancier_remboursements e WHERE e.pret_id = d.pret_id)) AS statut
        FROM demandes_pret d
        JOIN prets_payes USING (pret_id)
    )
    UPDATE demandes_pret d
    SET statut_pret = r.statut
    FROM recalcul r
    WHERE d.pret_id = r.pret_id
      AND d.statut_pret IS DISTINCT FROM r.statut;      -- n'écrit (et ne journalise) que les vrais changements
    RETURN NULL;                                        -- AFTER : la valeur renvoyée est ignorée
END $$;

DROP TRIGGER IF EXISTS trg_statut_apres_paiement ON echeancier_remboursements;
CREATE TRIGGER trg_statut_apres_paiement
    AFTER UPDATE ON echeancier_remboursements
    REFERENCING OLD TABLE AS anciennes NEW TABLE AS nouvelles
    FOR EACH STATEMENT EXECUTE FUNCTION fn_trg_statut_apres_paiement();
```

d) Le journal et l'alerte : AFTER UPDATE OF, par ligne, avec WHEN. `session_user` et `now()` sont remplis par les valeurs par défaut de la table.

```
CREATE OR REPLACE FUNCTION fn_trg_journal_statut() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_retard INTEGER := fn_jours_retard(NEW.pret_id, fn_date_reference());
BEGIN
    INSERT INTO journal_statuts (pret_id, ancien_statut, nouveau_statut, jours_retard, date_effet)
    VALUES (NEW.pret_id, OLD.statut_pret, NEW.statut_pret, v_retard, fn_date_reference());
    IF NEW.statut_pret = 'en_defaut' THEN
        -- alerte : tout programme abonné (LISTEN alertes_defaut) la reçoit quand la transaction est validée
        PERFORM pg_notify('alertes_defaut',
                          json_build_object('pret_id', NEW.pret_id, 'jours_retard', v_retard)::text);
    END IF;
    RETURN NULL;
END $$;

DROP TRIGGER IF EXISTS trg_journal_statut ON demandes_pret;
CREATE TRIGGER trg_journal_statut
    AFTER UPDATE OF statut_pret ON demandes_pret
    FOR EACH ROW
    WHEN (OLD.statut_pret IS DISTINCT FROM NEW.statut_pret)    -- la fonction n'est même pas appelée sinon
    EXECUTE FUNCTION fn_trg_journal_statut();
```

La chaîne complète. Un paiement modifie `echeancier_remboursements` → le gardien le valide → le recalcul met à jour `demandes_pret` → le journal enregistre le changement et, en cas de défaut, alerte. Trois triggers, trois responsabilités.

Exercice 3.3 — Deux procédures : l'encaissement et le traitement de nuit

a) `enregistrer_paiement(pret, montant, date)` : vérifie le montant et le prêt, verrouille le prêt, refuse un paiement supérieur au reste dû, puis affecte le paiement aux échéances impayées **de la plus ancienne à la plus récente**.

b) `actualiser_statuts(pret)` : recalcule le statut d'un prêt, ou de tout le portefeuille si aucun prêt n'est donné. Lance-la une première fois : tous les prêts chargés sont « sain ».

c) Dans une transaction annulée à la fin, joue trois scénarios : le prêt 67 règle tout son impayé (916,55 €) ; le prêt 61 paie 300 € ; le prêt 9 règle tous ses impayés (9 422,92 €). Observe les statuts et le journal.

Correction de l'exercice 3.3

a) L'encaissement. Les contrôles d'abord, le verrou ensuite, puis une boucle sur les échéances impayées dans l'ordre. Chaque `UPDATE` passe par le gardien ; le recalcul du statut se fait automatiquement, après chaque instruction.

```
CREATE OR REPLACE PROCEDURE enregistrer_paiement(p_pret_id BIGINT, p_montant NUMERIC, p_date DATE DEFAULT NULL)
LANGUAGE plpgsql AS $$
DECLARE
    v_date     DATE    := coalesce(p_date, fn_date_reference());
    v_restant  NUMERIC := p_montant;
    v_dossier  VARCHAR;
    v_du_total NUMERIC;
    v_part     NUMERIC;
    ech        RECORD;
BEGIN
    IF p_montant IS NULL OR p_montant <= 0 THEN
        RAISE EXCEPTION 'Montant invalide : %', p_montant;
    END IF;
    -- verrou sur le prêt : deux paiements simultanés du même prêt passent l'un après l'autre
    SELECT statut_dossier INTO v_dossier FROM demandes_pret WHERE pret_id = p_pret_id FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Prêt % inconnu', p_pret_id;
    ELSIF v_dossier <> 'accorde' THEN
        RAISE EXCEPTION 'Le prêt % n''est pas accordé', p_pret_id;
    END IF;
    SELECT sum(montant_du - montant_paye) INTO v_du_total
    FROM echeancier_remboursements WHERE pret_id = p_pret_id;
    IF p_montant > v_du_total THEN
        RAISE EXCEPTION 'Paiement de % € supérieur au reste dû du prêt % (% €)', p_montant, p_pret_id,
     ↪ v_du_total;
    END IF;

    FOR ech IN
        SELECT echeance_id, montant_du - montant_paye AS manquant
        FROM echeancier_remboursements
        WHERE pret_id = p_pret_id AND montant_paye < montant_du
        ORDER BY num_echeance                          -- la plus ancienne d'abord
    LOOP
        EXIT WHEN v_restant = 0;
        v_part := least(v_restant, ech.manquant);
        UPDATE echeancier_remboursements
        SET montant_paye  = montant_paye + v_part,
            date_paiement = CASE WHEN montant_paye + v_part = montant_du THEN v_date END
        WHERE echeance_id = ech.echeance_id;
        v_restant := v_restant - v_part;
    END LOOP;
    RAISE NOTICE 'Prêt % : % € encaissés le %', p_pret_id, p_montant, v_date;
END $$;
```

b) Le traitement de nuit. Une seule instruction ensembliste, qui réutilise les deux fonctions de l'exercice précédent : `fn_situation_prets` et `fn_regle_statut`. Grâce au paramètre `DEFAULT NULL`, la même procédure traite un prêt ou tout le portefeuille.

```
CREATE OR REPLACE PROCEDURE actualiser_statuts(p_pret_id BIGINT DEFAULT NULL)   -- NULL = tout le portefeuille
LANGUAGE plpgsql AS $$
DECLARE
    v_nb INTEGER;
BEGIN
    WITH recalcul AS (
        SELECT d.pret_id, fn_regle_statut(d.statut_pret, s.jours_retard, s.reste_du) AS statut
        FROM demandes_pret d
        JOIN fn_situation_prets(fn_date_reference()) s USING (pret_id)
        WHERE p_pret_id IS NULL OR d.pret_id = p_pret_id
    )
    UPDATE demandes_pret d
    SET statut_pret = r.statut
    FROM recalcul r
    WHERE d.pret_id = r.pret_id AND d.statut_pret IS DISTINCT FROM r.statut;
    GET DIAGNOSTICS v_nb = ROW_COUNT;
```

```
    RAISE NOTICE '% statut(s) mis à jour au %', v_nb, fn_date_reference();
END $$;

-- Premier passage : tous les prêts chargés sont « sain », le traitement leur donne leur vrai statut
CALL actualiser_statuts();
SELECT statut_pret, count(*) AS nb_prets FROM demandes_pret GROUP BY statut_pret ORDER BY statut_pret;
```

Ce que tu dois obtenir

`NOTICE: 18930 statut(s) mis à jour au 2026-09-30` : 7 088 `en_defaut`, 933 `en_retard`, 13 480 `sain`, 10 909 `solde`. Le journal contient désormais 18 930 lignes.

c) Les scénarios. `BEGIN ... ROLLBACK` : on observe tout, puis la base revient à son état initial.

```
BEGIN;
CALL enregistrer_paiement(67, 916.55);     -- 1. prêt en retard de 3 jours : il règle tout son impayé
CALL enregistrer_paiement(61, 300);        -- 2. prêt en retard de 46 jours : paiement partiel
CALL enregistrer_paiement(9, 9422.92);     -- 3. prêt en défaut (21 mois restants) : il règle tous ses impayés
SELECT d.pret_id, d.statut_pret, fn_jours_retard(d.pret_id, fn_date_reference()) AS jours_retard
FROM demandes_pret d WHERE d.pret_id IN (9, 61, 67) ORDER BY d.pret_id;
SELECT pret_id, ancien_statut, nouveau_statut, jours_retard, enregistre_par
FROM journal_statuts WHERE enregistre_le = now() ORDER BY journal_id;    -- now() : heure de début de la
    ↪ transaction
ROLLBACK;
```

Ce que tu dois obtenir

Prêt	Paiement	Statut après	Retard après
67	916,55 € : l'échéance du 27/09 est soldée	**sain** (était en retard)	0
61	300 € sur l'échéance de 704,41 € du 15/08	`en_retard` (inchangé)	46
9	9 422,92 € : les 13 échéances impayées	`en_defaut` (inchangé)	0

Le journal contient une seule nouvelle ligne : prêt 67, `en_retard` → `sain`.

Lecture. Le prêt 61 reste en retard : un paiement partiel ne solde pas l'échéance, dont la date ne change pas. Le prêt 9 n'a plus aucun retard, mais reste en défaut : c'est la période probatoire, voulue. Et le filtre `enregistre_le = now()` fonctionne parce que `now()` renvoie l'heure de *début* de la transaction, la même pour toutes les instructions.

3.3 Exercices à faire seul

Exercice 3.4 — Valider l'automatisation contre la réalité, puis faire passer le temps

a) Charge le fichier `donnees/verite_defauts.csv` (le défaut réel de chaque prêt, issu du jeu de données) dans une table temporaire. Les prêts que ta logique a mis « en défaut » sont-ils exactement ceux qui ont fait défaut dans la réalité ? Compte les désaccords.

b) Le prêt d'Awa (première échéance le 25/10/2026) : que se passe-t-il au 31/01/2027 si elle ne paie rien ? Avance la date de référence dans une transaction, regarde son statut avant et après le traitement de nuit, puis le journal. Annule tout à la fin.

Indices

— Dans psql : `\copy table FROM 'fichier.csv' WITH (FORMAT csv, HEADER true)`. Dans DBeaver : clic droit sur la table, *Importer des données*.

— b) Un `UPDATE parametres_risque` suffit à « faire passer le temps » : c'est l'intérêt d'avoir une date de référence paramétrée.

Exercice 3.5 — La clôture mensuelle des provisions, rejouable sans doublon

Écris la procédure `cloture_mensuelle(date_arrete)`. Elle enregistre dans la table `provisions_mensuelles` la provision de chaque prêt non soldé à cette date (retard, stage, EAD, PD utilisée, LGD, ECL). Elle refuse une date postérieure à la date de référence et peut être relancée sans créer de doublon. Produis les 12 arrêtés d'octobre 2025 à septembre 2026, puis l'évolution mensuelle de l'EAD, de l'ECL par stage, de la **dotation nette** (variation de l'ECL d'un mois sur l'autre) et du taux de couverture. Vérifie l'idempotence.

Indices

— Réutilise `fn_situation_prets(date)` : elle sait déjà calculer le passé.

— `CROSS JOIN LATERAL (SELECT ...)` permet de calculer le stage, puis la PD qui en dépend, dans la même requête.

— Un `CALL` ne s'utilise pas dans un `SELECT` : pour boucler sur 12 dates, écris un bloc anonyme `DO $$ ... $$`.

— La dotation nette : `ecl - LAG(ecl) OVER (ORDER BY date_arrete)`.

Exercice 3.6 — Bonus — Sécurité : chacun n'a que les droits dont il a besoin

Crée deux rôles : `analyste_risque` (lecture seule sur toutes les tables) et `application_paiement` (aucun droit sur les tables, seulement l'exécution de `enregistrer_paiement`). Rends la procédure `SECURITY DEFINER` de façon sûre. Vérifie, dans une transaction annulée, que l'application peut encaisser par la procédure mais pas écrire directement dans l'échéancier.

Indices

— `CREATE ROLE ... NOLOGIN` si le rôle n'existe pas (un bloc `DO` avec `IF NOT EXISTS (SELECT FROM pg_roles ...)`).

— `REVOKE ALL ON PROCEDURE ... FROM PUBLIC` : par défaut, tout le monde peut exécuter une procédure.

— Pour tester : `SET LOCAL ROLE application_paiement`, et l'erreur attendue s'appelle `insufficient_privilege`.

3.4 Synthèse pour les entretiens

Résumé du chapitre

1. On place dans la base ce qui ne doit **jamais être contourné** : intégrité, traçabilité, règles réglementaires simples.
2. **Fonctions** : SQL pour une formule, PL/pgSQL pour une logique ; `INTO STRICT` et `RAISE EXCEPTION` pour des erreurs explicites ; volatilité `IMMUTABLE`, `STABLE`, `VOLATILE`.
3. **Triggers** : `BEFORE` pour valider ou compléter, `AFTER` pour réagir ; par ligne ou par instruction (tables de transition) ; `WHEN` pour filtrer ; attention aux cascades.
4. **Procédures** : `CALL`, opérations métier, verrou `FOR UPDATE` contre les accès simultanés.
5. **Le temps n'est pas un événement** : un traitement planifié complète les triggers, avec la même règle.
6. **Idempotence** : `ON CONFLICT DO UPDATE`. **Alertes** : `pg_notify`, délivrées au commit. **Sécurité** : moindre privilège, `SECURITY DEFINER` avec `search_path` figé.

Ce que tu sauras dire en entretien

— « La détection du défaut est automatique et validée : sur 32 409 prêts réels, ma logique trouve 7 088 défauts, exactement les 7 088 du jeu de données, sans aucun désaccord. »

— « J'ai séparé les responsabilités : un trigger `BEFORE` protège le contrat, un trigger `AFTER` par instruction recalcule les statuts en un seul passage grâce aux tables de transition, un troisième journalise et alerte. La règle du statut est une fonction, partagée avec le traitement de nuit. »

— « Un trigger ne voit pas le temps passer : sans paiement, un retard franchit 90 jours sans événement. C'est pourquoi un traitement planifié complète les triggers. »

— « La clôture mensuelle est idempotente grâce à `ON CONFLICT` : on peut la relancer sans doublon. Et l'application d'encaissement n'a aucun droit sur les tables : elle passe par une procédure `SECURITY DEFINER` au `search_path` figé. »

Questions fréquentes en entretien

Fonction ou procédure ? Une fonction calcule et renvoie une valeur, utilisable dans un `SELECT` ; une procédure exécute une opération, s'appelle par `CALL` et peut gérer les transactions.

BEFORE ou AFTER ? `BEFORE` pour valider ou modifier la ligne avant écriture ; `AFTER` pour réagir au résultat (journal, recalcul d'une autre table).

Quels risques avec les triggers ? Logique invisible, cascades, récursion, coût par ligne, et une erreur qui annule toute l'instruction.

Comment éviter qu'un même paiement soit compté deux fois ? Verrouiller la ligne du prêt (`FOR UPDATE`) et contraindre les montants (`paiement_borne`).

Qu'est-ce que l'idempotence ? Une opération qu'on peut relancer sans changer le résultat : ici, une clôture rejouée remplace ses lignes au lieu de les dupliquer.

Partie 5

Feuille de route de réalisation

« Un grand projet n'est qu'une suite de petites tâches terminées. »

Cette partie est ta **liste de contrôle**. Elle reprend le projet de A à Z en six tâches, dans l'ordre où il faut les faire. Pour chaque tâche : l'objectif, les étapes à cocher, le résultat attendu et les pièges fréquents. Imprime-la ou garde-la ouverte à côté de ton éditeur.

Semaine	Programme	Tâches
1	Installer l'environnement, créer la base, la charger ; lire les parties 1 à 3	1, 2, 3
2	Chapitre 1 : cours et exercices ; début du chapitre 2	—
3	Chapitre 2 : requêtes analytiques	4
4	Chapitre 3 : automatisation ; publication	5, 6

Compte 4 à 6 heures par semaine. Rien n'empêche d'aller plus vite, mais ne saute pas les exercices seuls : ce sont eux qui te prépareront aux questions imprévues.

5.1 Tâche 1 — Configurer l'environnement (PostgreSQL et DBeaver)

Objectif : un serveur PostgreSQL qui tourne, une base vide, un outil graphique connecté et un environnement Python prêt. **Durée :** 1 heure.

☐ **PostgreSQL 17.** Sur macOS, avec Homebrew :

```
brew install postgresql@17
brew services start postgresql@17          # démarre le serveur, et à chaque démarrage du
    ↪ Mac
psql --version                             # attendu : psql (PostgreSQL) 17.x
```

Si `psql` est introuvable, ajoute son dossier au `PATH` (Homebrew l'indique à la fin de l'installation). Sur ton Mac, PostgreSQL 17 est déjà installé : vérifie seulement la version.

☐ **La base.**

```
createdb risk_analytics_db
psql -d risk_analytics_db -c "SELECT version();"
```

☐ **DBeaver** (édition Community, gratuite) : *Nouvelle connexion* → PostgreSQL ; hôte `localhost`, port `5432`, base `risk_analytics_db`, utilisateur : ton nom d'utilisateur macOS, mot de passe vide (installation locale). Accepte le téléchargement du pilote, puis *Tester la connexion*.

☐ **Python.** Depuis le dossier du projet :

```
cd ~/Desktop/Projet_SQL_Risk
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt            # numpy, pandas, Faker, SQLAlchemy, psycopg
```

☐ **La connexion du script.** Le chargeur lit l'adresse de la base dans la variable d'environnement `DATABASE_URL`. Sans mot de passe en local, la valeur par défaut suffit. Sinon, définis-la dans le terminal, **jamais dans le code** :

```
# par défaut : postgresql://localhost/risk_analytics_db
export DATABASE_URL="postgresql://moi:secret@localhost/risk_analytics_db"
```

Pièges fréquents

— *role "..." does not exist* : l'utilisateur PostgreSQL n'existe pas. `createuser -s ton_nom`.

— *port 5432 already in use* : un autre PostgreSQL tourne (Postgres.app, une autre version). Arrête-le, ou change de port.

— Dans DBeaver, l'onglet SQL est lié à une base : vérifie en haut de l'éditeur que c'est bien `risk_analytics_db`.

5.2 Tâche 2 — Exécuter le script DDL

Objectif : créer les 6 tables, leurs contraintes et leurs index. **Durée :** 1 heure, lecture comprise.

☐ Lis `sql/01_schema.sql` en entier, avec le dictionnaire de données de la partie 3 à côté. Pour chaque contrainte, demande-toi quelle erreur métier elle empêche.

☐ Exécute-le :

```
psql -d risk_analytics_db -v ON_ERROR_STOP=1 -f sql/01_schema.sql
```

Dans DBeaver : *Exécuter le script* (`Alt+X`), et non *Exécuter l'instruction* (`Ctrl+Entrée`), qui n'exécute que la requête sous le curseur.

☐ Vérifie la structure :

```
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE' ORDER BY 1;     -- 6 tables

SELECT contype, count(*) FROM pg_constraint                                 -- c : CHECK, f
    ↪ : clé étrangère,
WHERE connamespace = 'public'::regnamespace GROUP BY 1 ORDER BY 1;          -- p : clé
    ↪ primaire, u : UNIQUE

SELECT * FROM parametres_risque;                                            -- 4 paramètres
```

☐ Teste une contrainte : l'insertion d'un client de 15 ans doit échouer (exercice 1.2).

Ce que tu dois obtenir

6 tables ; 29 contraintes `CHECK`, 4 clés étrangères, 6 clés primaires, 3 contraintes `UNIQUE` ; 4 paramètres (`lgd` 0,45, `seuil_defaut_jours` 90, `seuil_stage2_jours` 30, `date_reference` 2026-09-30). Les `NOT NULL` ne sont pas comptés : PostgreSQL 17 les stocke à part.

Piège — Le script repart de zéro

Le script commence par `DROP TABLE ... CASCADE` : le relancer **efface toutes les données**, y compris le travail des chapitres. C'est voulu pour reconstruire la base, mais ne le relance pas par erreur au milieu du chapitre 3.

5.3 Tâche 3 — Générer et injecter les données (Python)

Objectif : remplir la base avec 1,3 million de lignes cohérentes. **Durée :** 2 heures pour lire et comprendre le script, 15 secondes pour l'exécuter.

5.3.1 Le script et ses choix

Ton prompt demandait un script **Faker + Pandas + SQLAlchemy** injectant **50 000 lignes** synthétiques. Le script fourni, `python/charger_donnees.py`, utilise bien ces trois bibliothèques, mais va plus loin sur un point essentiel : il part de **tes données réelles**.

Demandé	Dans ton projet	Pourquoi
50 000 lignes générées par Faker	32 409 prêts **réels** et 1,3 million de lignes au total	les prêts, les défauts et les prédictions de tes modèles sont réels : tu peux **valider** ta détection du défaut (0 désaccord sur 7 088), ce qui est impossible avec des données inventées
Faker	noms et e-mails fictifs, en français	aucune donnée personnelle ; graine fixe (2026), donc résultats identiques à chaque exécution
Pandas	nettoyage, échéancier, paiements	calculs vectorisés : 1,1 million de mensualités en moins d'une seconde, sans boucle Python
SQLAlchemy	connexion, transaction unique, `DataFrame.to_sql`	`to_sql` envoie des `INSERT` par défaut (plusieurs minutes) ; avec une méthode `COPY`, une dizaine de secondes

Ce qui est **simulé** (le fichier source ne le contient pas) l'est de façon **cohérente avec le défaut réel** : un prêt qui a fait défaut cesse de payer et cumule plus de 90 jours de retard à la date de référence ; un prêt sain paie, avec de petits retards occasionnels.

5.3.2 Lecture guidée, section par section

1. **Lecture et nettoyage.** Les règles exactes de ton projet R : suppression des doublons, âge au plus 100 ans, ancienneté d'emploi au plus égale à l'âge moins 14. Un `assert` vérifie qu'il reste 32 409 prêts : si les règles divergeaient de celles du projet R, les prédictions ne correspondraient plus aux prêts.

2. **Dates et durées.** Dates de demande tirées au hasard (avant fin octobre 2025 pour les prêts en défaut, pour qu'ils aient le temps de cumuler leurs impayés) ; durée selon le montant (12 à 60 mois) ; taux manquant remplacé par la médiane, avec le drapeau `taux_impute`.

3. **Clients** : Faker pour les noms, e-mails construits sans accent et rendus uniques par le numéro de ligne.

4. **Prêts** : tous accordés (le fichier ne contient que des prêts octroyés), statut « sain » en attendant le traitement de nuit du chapitre 3.

5. **Échéancier vectorisé.** `np.repeat` crée une ligne par mensualité ; la formule d'annuité et le CRD sont calculés sur 1,1 million de lignes d'un coup ; la dernière échéance absorbe les arrondis ; les dates mois par mois sont bornées à la fin du mois (le piège du 31 janvier).

6. **Paiements.** Un prêt en défaut paie k mensualités puis s'arrête, en laissant au moins 5 impayés échus ; 25 % d'entre eux règlent une partie de leur premier impayé. Les autres paiements : 88 % à l'heure, 10 % avec 1 à 20 jours de retard, 2 % avec 31 à 75 jours (incidents régularisés).

7. **Prédictions** : les trois modèles, leurs seuils, le champion ; les points pour la grille seulement, l'explication SHAP pour le champion seulement. Le fichier `verite_defauts.csv` est écrit pour l'exercice 3.4.

8. **Chargement.** Une seule transaction : `TRUNCATE ... RESTART IDENTITY CASCADE`, puis `to_sql` avec la méthode `COPY` pour chaque table, puis le recalage des compteurs d'identité (`setval`) et un `ANALYZE`.

Analogie — INSERT contre COPY

Envoyer un million d'`INSERT`, c'est poster un million de lettres d'une ligne : chacune a son enveloppe, son timbre, son passage au tri. `COPY`, c'est envoyer un seul colis contenant toutes les lignes. Le contenu est le même, le coût n'a rien à voir.

5.3.3 Le script complet

```
"""
Alimente la base risk_analytics_db à partir des données RÉELLES du projet de scoring de crédit.

Réel     : caractéristiques des 32 409 emprunteurs et prêts, défaut observé,
           prédictions des trois modèles (grille de score, logistique, XGBoost) et explications SHAP.
Simulé   : le fichier source ne contient ni noms, ni dates, ni durées, ni détail des paiements.
           Ces éléments sont simulés de façon COHÉRENTE avec le défaut réel : un prêt qui a fait
           défaut cesse de payer (plus de 90 jours de retard à la date de référence), un prêt sain paie.
           Les noms sont fictifs (Faker) ; aucune donnée personnelle réelle.

Usage    : python python/charger_donnees.py          (depuis le dossier du projet)
Connexion: variable d'environnement DATABASE_URL, par défaut postgresql://localhost/risk_analytics_db
           (ne jamais écrire un mot de passe dans le code : export DATABASE_URL=postgresql://user:
    ↪ mdp@hote/base)
Outils   : Faker (noms fictifs), Pandas (préparation), SQLAlchemy (connexion et transaction),
           psycopg 3 (pilote PostgreSQL, utilisé par SQLAlchemy pour le COPY).
"""
import csv
import io
import os
import time
import unicodedata

import numpy as np
import pandas as pd
from faker import Faker
from sqlalchemy import create_engine, text
from sqlalchemy.engine import make_url
```

```
DATABASE_URL = os.environ.get("DATABASE_URL", "postgresql://localhost/risk_analytics_db")
DATE_REFERENCE = pd.Timestamp("2026-09-30")   # « aujourd'hui » figé : les résultats sont reproductibles
GRAINE = 2026
MEDIANE_TAUX = 11.0                           # médiane d'apprentissage du projet R (imputation)

rng = np.random.default_rng(GRAINE)
Faker.seed(GRAINE)
fake = Faker("fr_FR")
debut = time.time()


def etape(message):
    print(f"[{time.time() - debut:5.1f} s] {message}")


# ------------------------------------------------------------------------------
# 1. Lecture et nettoyage : exactement les règles du projet R (32 409 prêts)
# ------------------------------------------------------------------------------
brut = pd.read_csv("donnees/credit_risk_dataset.csv")
brut.insert(0, "id_source", np.arange(1, len(brut) + 1))          # numéro de ligne du fichier
colonnes = [c for c in brut.columns if c != "id_source"]
df = brut.drop_duplicates(subset=colonnes, keep="first")
df = df[df.person_age <= 100]
df = df[df.person_emp_length.isna() | (df.person_emp_length <= df.person_age - 14)].reset_index(drop=
    ↪ True)
assert len(df) == 32409, len(df)
n = len(df)
etape(f"{n} prêts après nettoyage")

LOGEMENT = {"RENT": "locataire", "OWN": "proprietaire", "MORTGAGE": "credit_immobilier", "OTHER": "autre
    ↪ "}
MOTIF = {"EDUCATION": "etudes", "MEDICAL": "sante", "VENTURE": "creation_entreprise", "PERSONAL": "
    ↪ personnel",
         "HOMEIMPROVEMENT": "travaux", "DEBTCONSOLIDATION": "regroupement_dettes"}
defaut = df.loan_status.to_numpy()

# ------------------------------------------------------------------------------
# 2. Dates et durées (simulées). Un prêt en défaut doit avoir assez d'échéances échues
#    pour cumuler plus de 90 jours de retard à la date de référence.
# ------------------------------------------------------------------------------
def dates_uniformes(debut, fin, taille):
    jours = (pd.Timestamp(fin) - pd.Timestamp(debut)).days
    return pd.Timestamp(debut) + pd.to_timedelta(rng.integers(0, jours + 1, taille), unit="D")

date_demande = np.where(defaut == 1,
                        dates_uniformes("2022-01-01", "2025-10-31", n),
                        dates_uniformes("2022-01-01", "2026-06-30", n))
date_demande = pd.to_datetime(date_demande)
date_decision = date_demande + pd.to_timedelta(rng.integers(0, 11, n), unit="D")
premiere_echeance = date_decision + pd.DateOffset(months=1)

montant = df.loan_amnt.to_numpy().astype(float)
duree = np.select([montant < 5000, montant < 15000],
                  [rng.choice([12, 24], n), rng.choice([24, 36, 48], n)],
                  rng.choice([36, 48, 60], n))
taux = df.loan_int_rate.to_numpy()
taux_impute = np.isnan(taux)
taux = np.where(taux_impute, MEDIANE_TAUX, taux)

# ------------------------------------------------------------------------------
# 3. Clients
# ------------------------------------------------------------------------------
def sans_accent(texte):
```

```
    return unicodedata.normalize("NFKD", texte).encode("ascii", "ignore").decode().lower()

noms = [fake.name() for _ in range(n)]
emails = [f"{sans_accent(nom).replace(' ', '.').replace(chr(39), '')}.{i}@exemple.fr"
          for nom, i in zip(noms, df.id_source)]
clients = pd.DataFrame({
    "client_id": df.id_source,
    "nom_complet": noms,
    "email": emails,
    "age": df.person_age,
    "revenu_annuel": df.person_income.astype(float),
    "statut_logement": df.person_home_ownership.map(LOGEMENT),
    "anciennete_emploi": df.person_emp_length,
    "anciennete_credit": df.cb_person_cred_hist_length,
    "defaut_anterieur": df.cb_person_default_on_file.eq("Y"),
    "date_inscription": (date_demande - pd.to_timedelta(rng.integers(0, 1500, n), unit="D")).date,
})

# ------------------------------------------------------------------------------
# 4. Demandes de prêt (toutes accordées : le fichier source ne contient que des prêts octroyés)
# ------------------------------------------------------------------------------
prets = pd.DataFrame({
    "pret_id": df.id_source,
    "client_id": df.id_source,
    "montant_demande": montant,
    "duree_mois": duree,
    "taux_interet": taux,
    "taux_impute": taux_impute,
    "motif_pret": df.loan_intent.map(MOTIF),
    "note_risque": df.loan_grade,
    "date_demande": date_demande.date,
    "date_decision": date_decision.date,
    "date_premiere_echeance": premiere_echeance.date,
    "statut_dossier": "accorde",
    "statut_pret": "sain",          # recalculé à partir des paiements au chapitre 3
})

# ------------------------------------------------------------------------------
# 5. Échéancier à mensualités constantes : M = C × r / (1 − (1 + r)−^n), r = taux annuel / 12
# ------------------------------------------------------------------------------
idx = np.repeat(np.arange(n), duree)                        # une ligne par mensualité
num = np.concatenate([np.arange(1, d + 1) for d in duree])
r = taux[idx] / 1200
C = montant[idx]
nn = duree[idx]
mensualite = np.round(C * r / (1 - (1 + r) ** -nn), 2)
crd_avant = C * (1 + r) ** (num - 1) - mensualite * ((1 + r) ** (num - 1) - 1) / r
interets = np.round(crd_avant * r, 2)
capital = np.round(mensualite - interets, 2)
# Arrondis au centime : la dernière échéance rembourse exactement le capital restant,
# pour que la somme des parts de capital soit égale au montant prêté, au centime près.
derniere = num == nn
cumul_avant = pd.Series(capital).groupby(idx).cumsum().to_numpy() - capital
capital = np.where(derniere, np.round(C - cumul_avant, 2), capital)
montant_du = np.round(capital + interets, 2)
crd_apres = np.round(C - (cumul_avant + capital), 2)
# décalage mois par mois (vectorisé) : même jour du mois que la première échéance, borné à la fin du
    ↪ mois
base = pd.to_datetime(premiere_echeance.to_numpy()[idx])
mois = (base.year * 12 + base.month - 1) + (num - 1)
annee, mois_num = mois // 12, mois % 12 + 1
fin_mois = pd.to_datetime(dict(year=annee, month=mois_num, day=1)) + pd.offsets.MonthEnd(0)
jour = np.minimum(base.day, fin_mois.dt.day)
date_echeance = pd.to_datetime(dict(year=annee, month=mois_num, day=jour))
```

```
etape(f"{len(idx)} mensualités calculées")

# ---------------------------------------------------------------------------
# 6. Paiements jusqu'à la date de référence, cohérents avec le défaut réel
# ---------------------------------------------------------------------------
echue = (date_echeance <= DATE_REFERENCE).to_numpy()
nb_echues = np.bincount(idx, weights=echue, minlength=n).astype(int)
# Prêt en défaut : il paie k échéances puis s'arrête, en laissant au moins 5 échéances impayées.
dernier_paye = np.where(defaut == 1,
                        rng.integers(1, np.maximum(nb_echues - 4, 2)),
                        np.iinfo(np.int32).max)
paye = echue & (num <= dernier_paye[idx])
# Retards ponctuels : 88 % des paiements à l'heure (jusqu'à 3 jours d'avance), 10 % de 1 à 20 jours
# de retard, 2 % de 31 à 75 jours (incidents temporaires, régularisés ensuite : jamais 90 jours).
tirage = rng.random(len(idx))
decalage = np.select([tirage < 0.88, tirage < 0.98],
                     [-rng.integers(0, 4, len(idx)), rng.integers(1, 21, len(idx))],
                     rng.integers(31, 76, len(idx)))
date_paiement = date_echeance + pd.to_timedelta(decalage, unit="D")
paye &= (date_paiement <= DATE_REFERENCE).to_numpy()        # un paiement futur n'a pas encore eu lieu
# 25 % des prêts en défaut ont réglé une partie de leur première échéance impayée.
premiere_impayee = (defaut[idx] == 1) & (num == dernier_paye[idx] + 1) & echue
partiel = premiere_impayee & (rng.random(len(idx)) < 0.25)
montant_paye = np.where(paye, montant_du, np.where(partiel, np.round(montant_du * rng.uniform(0.3, 0.7,
    ↪ len(idx)), 2), 0.0))

echeancier = pd.DataFrame({
    "pret_id": df.id_source.to_numpy()[idx],
    "num_echeance": num,
    "date_echeance": date_echeance.dt.date,
    "montant_du": montant_du,
    "part_capital": capital,
    "part_interets": interets,
    "capital_restant_du": crd_apres,
    "montant_paye": montant_paye,
    "date_paiement": np.where(paye, date_paiement.dt.date, None),
})
etape(f"paiements simulés : {int(paye.sum())} mensualités payées, {int((echue & ~paye).sum())} impayées
    ↪ échues")

# ---------------------------------------------------------------------------
# 7. Prédictions des trois modèles (réelles, issues du projet R) et registre des échantillons
# ---------------------------------------------------------------------------
scores = pd.read_csv("donnees/predictions_modeles.csv")
shap = pd.read_csv("donnees/shap_xgboost.csv")
base_pred = df[["id_source"]].merge(scores, left_on="id_source", right_on="id_pret").merge(shap, on="
    ↪ id_pret")
MODELES = [  # version, colonne de probabilité, seuil (projet R), date d'entraînement, champion
    ("grille_score_v1", "pd_grille", 0.240, "2026-04-20 10:00", False),
    ("logistique_v1", "pd_logistique", 0.235, "2026-04-20 10:00", False),
    ("xgboost_monotone_v2", "pd_xgboost", 0.255, "2026-09-15 16:00", True),
]
morceaux = []
for version, colonne, seuil, date_pred, champion in MODELES:
    p = base_pred[colonne].round(6)
    morceaux.append(pd.DataFrame({
        "pret_id": base_pred.id_pret, "version_modele": version, "est_champion": champion,
        "probabilite_defaut": p, "seuil_decision": seuil,
        "decision": np.where(p >= seuil, "refuser", "accorder"), "date_prediction": date_pred,
    }))
predictions = pd.concat(morceaux, ignore_index=True).merge(
    base_pred[["id_pret", "points_grille", "shap_top_feature", "shap_top_valeur"]].rename(columns={"
    ↪ id_pret": "pret_id"}),
    on="pret_id")
```

```
# Points : seulement pour la grille ; explication SHAP : seulement pour le modèle champion (XGBoost).
predictions["score_points"] = predictions.points_grille.round().astype("Int64").where(
    predictions.version_modele.eq("grille_score_v1"))
for col in ("shap_top_feature", "shap_top_valeur"):
    predictions[col] = predictions[col].where(predictions.est_champion)
predictions = predictions.drop(columns="points_grille")
echantillons = pd.read_csv("donnees/echantillon.csv").rename(columns={"id_pret": "pret_id"})

# Vérité terrain, pour contrôler la simulation (elle ne fait pas partie de la base)
df[["id_source", "loan_status"]].rename(columns={"id_source": "pret_id", "loan_status": "defaut_reel"})
    ↪ \
    .to_csv("donnees/verite_defauts.csv", index=False)

# ------------------------------------------------------------------------
# 8. Chargement : DataFrame.to_sql (Pandas + SQLAlchemy) avec la méthode COPY de PostgreSQL.
#    Par défaut, to_sql envoie des INSERT : plusieurs minutes pour 1,1 million d'échéances.
#    COPY les envoie en un seul flux : environ 10 secondes.
# ------------------------------------------------------------------------
def methode_copy(table, conn, colonnes, lignes):
    """Méthode d'insertion pour to_sql : envoie les lignes avec COPY au lieu d'INSERT."""
    tampon = io.StringIO()
    csv.writer(tampon).writerows(lignes)              # une valeur manquante devient un champ vide
    tampon.seek(0)
    curseur = conn.connection.driver_connection.cursor()   # la connexion psycopg sous SQLAlchemy
    with curseur.copy(f"COPY {table.name} ({', '.join(colonnes)}) FROM STDIN WITH (FORMAT csv, NULL '')"
    ↪ ) as copie:
        copie.write(tampon.read())


def charger(conn, table, frame):
    frame.to_sql(table, conn, if_exists="append", index=False, method=methode_copy, chunksize=200_000)
    etape(f"{table} : {len(frame)} lignes")


# postgresql…:// désigne le pilote psycopg2 pour SQLAlchemy : on précise psycopg (version 3)
moteur = create_engine(make_url(DATABASE_URL).set(drivername="postgresql+psycopg"))
with moteur.begin() as conn:                          # une seule transaction : tout ou rien
    conn.execute(text("TRUNCATE predictions_scoring_ml, echantillons_modele, echeancier_remboursements,
    ↪ "
                      "demandes_pret, clients RESTART IDENTITY CASCADE"))
    charger(conn, "clients", clients)
    charger(conn, "demandes_pret", prets)
    charger(conn, "echeancier_remboursements", echeancier)
    charger(conn, "predictions_scoring_ml", predictions)
    charger(conn, "echantillons_modele", echantillons)
    # Les identifiants ont été fournis explicitement : on recale les compteurs automatiques,
    # sinon le prochain client créé recevrait l'identifiant …1 déjà pris.
    for table, colonne in (("clients", "client_id"), ("demandes_pret", "pret_id")):
        conn.execute(text(f"SELECT setval(pg_get_serial_sequence('{table}', '{colonne}'), (SELECT max({
    ↪ colonne}) FROM {table}))"))
with moteur.connect() as conn:                        # ANALYZE : statistiques à jour pour l'optimiseur
    conn.execute(text("ANALYZE"))
    conn.commit()
etape("chargement terminé")
```

5.3.4 Exécution et contrôle

☐ Lance le script **depuis le dossier du projet** (les chemins `donnees/...` sont relatifs) :

```
source .venv/bin/activate
python python/charger_donnees.py
```

```
[  0.0 s] 32409 prêts après nettoyage
[  0.6 s] 1107936 mensualités calculées
[  1.0 s] paiements simulés : 665512 mensualités payées, 108737 impayées échues
[  1.6 s] clients : 32409 lignes
[  1.9 s] demandes_pret : 32409 lignes
[ 11.4 s] echeancier_remboursements : 1107936 lignes
[ 12.2 s] predictions_scoring_ml : 97227 lignes
[ 12.3 s] echantillons_modele : 32409 lignes
[ 12.9 s] chargement terminé
```

☐ Vérifie les volumes dans la base :

```sql
SELECT 'clients' AS table_, count(*) FROM clients
UNION ALL SELECT 'demandes_pret', count(*) FROM demandes_pret
UNION ALL SELECT 'echeancier_remboursements', count(*) FROM echeancier_remboursements
UNION ALL SELECT 'predictions_scoring_ml', count(*) FROM predictions_scoring_ml
UNION ALL SELECT 'echantillons_modele', count(*) FROM echantillons_modele;
```

☐ Ouvre quelques prêts dans DBeaver (les prêts 9, 61, 67) et compare avec les tableaux de la partie 2.

Pièges fréquents

— *FileNotFoundError : donnees/credit_risk_dataset.csv* : tu n'es pas dans le dossier du projet.

— *No module named psycopg* : l'environnement virtuel n'est pas activé, ou `pip install -r requirements.txt` n'a pas été lancé.

— Le script vide les tables avant de les remplir : le prêt d'Awa (chapitre 1) disparaît. Relance l'exercice 1.1 après un rechargement.

5.4 Tâche 4 — Rédiger les requêtes analytiques métier

Objectif : écrire toi-même les analyses du chapitre 2. **Durée :** une semaine.

☐ Fais les chapitres 1 et 2 dans l'ordre, chacun dans son fichier : `sql/02_chapitre1.sql`, puis `sql/03_chapitre2.sql`. Commence chaque bloc par un commentaire qui dit quelle question métier il traite.

☐ **Calcul du DPD** : la vue `v_situation_prets` (exercice 2.1).

☐ **Analyse de portefeuille** : tranches de retard, PAR30 et PAR90 (2.1), matrice de transition (2.6), analyse par génération (2.7), plus grosses expositions (2.8).

☐ **Provisions** : stages, EAD, ECL avec `ROLLUP` (2.2).

☐ **Monitoring du modèle** : déciles et courbe de gains (2.3), AUC et Gini des trois modèles (2.5).

☐ Compare chaque résultat aux valeurs du cours ; note dans ton journal les écarts et leur cause.

☐ Mesure une requête lourde avec `EXPLAIN ANALYZE` (la matrice de transition), puis améliore-la.

Ce que tu dois obtenir

PAR30 41,90 %, PAR90 41,04 % ; ECL 25 627 129,15 € (19,13 % de l'EAD) ; décile 1 : 45,8 % des défauts captés ; AUC du champion 0,923 (Gini 0,847) contre 0,876 et 0,875 pour les challengers.

5.5 Tâche 5 — Implémenter la détection automatique du défaut

Objectif : la base passe elle-même les prêts en défaut, trace et alerte. **Durée :** une semaine.

☐ Crée les tables `journal_statuts` et `provisions_mensuelles` (exercice 1.3), si ce n'est pas fait.

☐ Dans `sql/04_chapitre3.sql` : les fonctions (3.1), les trois triggers (3.2), les deux procédures (3.3).

☐ Lance le premier `CALL actualiser_statuts();` : 18 930 statuts mis à jour.

☐ Teste chaque mécanisme **dans une transaction annulée** : protection de l'échéancier, scénarios de paiement, journal.

☐ Valide contre la réalité : 7 088 défauts détectés, 0 désaccord (exercice 3.4).

☐ Fais passer le temps au 31/01/2027 et regarde le prêt d'Awa passer en défaut.

☐ Démontre l'alerte avec deux terminaux : `LISTEN alertes_defaut;` dans le premier, le traitement de nuit dans le second.

☐ La clôture mensuelle sur 12 mois (3.5) et, en bonus, les rôles (3.6).

5.6 Tâche 6 — Publier sur GitHub et valoriser le projet

Objectif : un dépôt propre, qu'un recruteur comprend en deux minutes. **Durée :** une demi-journée.

5.6.1 Le dépôt

☐ Une structure claire :

```
Projet_SQL_Risk/├──
 README.md              la vitrine du projet├──
 requirements.txt├──
 executer.sh            reconstruit tout de zéro├──
 sql/                   01_schema.sql, 02_chapitre1.sql, 03_chapitre2.sql, 04_chapitre3.sql├──
 python/                charger_donnees.py├──
 donnees/               sorties de tes modèles (et le jeu de données si sa licence le permet)└──
 docs/                  ce cours, le schéma de la base
```

☐ Un `.gitignore` : `.venv/`, `__pycache__/`, `.DS_Store`, `.env`, `*.log`, et les fichiers temporaires de LaTeX (`*.aux`, `*.log`, `*.toc`…).

☐ **Les données.** Vérifie la licence du *Credit Risk Dataset* sur sa page Kaggle avant de le publier. Si elle ne le permet pas, ne versionne pas `credit_risk_dataset.csv` et indique dans le README où le télécharger.

☐ Un script `executer.sh` qui reconstruit tout, preuve que ton projet est reproductible :

```
#!/bin/sh
```

```
set -e                                        # s'arrête à la première erreur
psql -d risk_analytics_db -v ON_ERROR_STOP=1 -f sql/01_schema.sql
python python/charger_donnees.py
for f in sql/02_chapitre1.sql sql/03_chapitre2.sql sql/04_chapitre3.sql; do
    psql -d risk_analytics_db -v ON_ERROR_STOP=1 -f "$f"
done
```

☐ Des commits qui racontent l'histoire : « Schéma : tables, contraintes et index », « Chargement des données réelles par COPY », « Vue de situation et PAR », « Triggers de détection du défaut »… Jamais de mot de passe dans un commit.

☐ Crée le dépôt et pousse : `git init`, `git add .`, `git commit`, puis `gh repo create` (ou l'interface de GitHub).

5.6.2 Le README

Le README est souvent la seule chose qu'un recruteur lira. Il doit répondre en une page à quatre questions : quel problème, quelle solution, quels résultats, comment l'exécuter.

```
# Risk Analytics & Scoring de crédit — un backend SQL d'entreprise

Système PostgreSQL qui suit 32 409 prêts réels de la demande au défaut : retards (DPD),
provisions IFRS 9, registre des prédictions de 3 modèles de ML, détection automatique du défaut.

## Résultats clés
- Détection du défaut automatisée et validée : 7 088 défauts détectés, 0 désaccord avec la réalité
- Provisions IFRS 9 : 25,6 €M (19,1 % de l'exposition), dont 96,5 % en stage 3
- Monitoring du modèle en SQL : AUC 0,923 (XGBoost) contre 0,876 (logistique)
- 1,3 million de lignes chargées en ~13 s (COPY) ; retard d'un prêt en 0,3 ms (index partiel)

## Architecture        (schéma de la base, figure de la partie 3)
## Ce que montre le projet   (contraintes, CTE, fenêtres, triggers, procédures, rôles)
## Lancer le projet     (prérequis, ./executer.sh)
## Ce que j'ai appris
```

5.6.3 Le portfolio et le CV

☐ Une carte sur ton portfolio : un titre, deux lignes de contexte, trois chiffres, le lien GitHub.

☐ Une ligne de CV, avec des résultats mesurés :

> *Backend SQL de gestion du risque de crédit (PostgreSQL) : 32 000 prêts réels, détection automatique du défaut par triggers (0 écart avec la réalité), provisions IFRS 9 et monitoring de modèles de scoring (AUC 0,92) en SQL analytique.*

☐ Prépare un **pitch de deux minutes** : le contexte (une banque doit suivre et provisionner ses prêts), le problème (données éparpillées, défaut détecté à la main), ta solution (une base qui applique elle-même les règles), les résultats (les chiffres ci-dessus), ce que tu as appris (et ce que tu ferais ensuite).

5.6.4 Pour aller plus loin

Si tu veux enrichir le projet après ces six tâches, quelques extensions appréciées en entretien :

— **Surveillance de la dérive** : un indice de stabilité de la population (PSI) entre les scores d'apprentissage et ceux des nouveaux prêts, calculé en SQL ;

— **Planification réelle** du traitement de nuit avec l'extension `pg_cron` ;
— **Tests automatiques** des fonctions et triggers (extension pgTAP) ;
— **Partitionnement** de l'échéancier par année, pour des volumes encore plus grands ;
— **Docker** : un `docker-compose.yml` qui lance PostgreSQL et charge tout en une commande ;
— **Un tableau de bord** (Metabase, Streamlit) branché sur les vues du chapitre 2.

Corrigés des exercices à faire seul

« Un corrigé ne sert qu'à celui qui a d'abord cherché. »

Tu trouveras ici le code, l'explication et les résultats de chaque exercice à faire seul. Les mêmes corrigés, exécutables, sont dans le dossier `corriges/`. Si ton résultat diffère, ne corrige pas ta requête pour « tomber juste » : cherche d'abord *pourquoi* les deux diffèrent.

Chapitre 1

Correction de l'exercice 1.3

Le journal. L'ancien statut peut être NULL (premier statut d'un prêt). Avec `CHECK (ancien_statut <> nouveau_statut)`, la ligne `(NULL, 'en_defaut')` serait acceptée, mais par accident : la comparaison vaut NULL, et un `CHECK` laisse passer NULL. `IS DISTINCT FROM` traite NULL comme une valeur ordinaire : `(NULL, 'en_defaut')` est acceptée parce que les deux valeurs diffèrent vraiment, `('sain', 'sain')` est refusée. L'intention est explicite, et la règle resterait juste si `nouveau_statut` devenait facultatif. Les valeurs par défaut remplissent l'horodatage et l'utilisateur sans que les triggers aient à s'en occuper. Le journal est consulté prêt par prêt : on indexe la clé étrangère.

Les provisions. La clé primaire composite `(date_arrete, pret_id)` garantit « une ligne par prêt et par arrêté » ; elle sert aussi de cible à `ON CONFLICT` dans la clôture rejouable (exercice 3.5). Toutes les colonnes sont `NOT NULL` : une provision incomplète n'a pas de sens.

```
-- 1. Journal d'audit : chaque changement de statut d'un prêt, avec la date et l'ancien statut.
CREATE TABLE IF NOT EXISTS journal_statuts (
    journal_id      BIGINT       GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    pret_id         BIGINT       NOT NULL REFERENCES demandes_pret (pret_id) ON DELETE RESTRICT,
    ancien_statut   VARCHAR(12),
    nouveau_statut  VARCHAR(12)  NOT NULL,
    jours_retard    INTEGER      CHECK (jours_retard >= 0),
    date_effet      DATE         NOT NULL,
    enregistre_le   TIMESTAMP    NOT NULL DEFAULT now(),
    enregistre_par  TEXT         NOT NULL DEFAULT session_user,   -- l'utilisateur connecté (voir chapitre 3,
     ↪ sécurité)
    CONSTRAINT statut_change CHECK (ancien_statut IS DISTINCT FROM nouveau_statut)
);
CREATE INDEX IF NOT EXISTS idx_journal_pret ON journal_statuts (pret_id);

-- 2. Provisions mensuelles : une photographie du risque de chaque prêt à chaque arrêté.
CREATE TABLE IF NOT EXISTS provisions_mensuelles (
    date_arrete   DATE          NOT NULL,
    pret_id       BIGINT        NOT NULL REFERENCES demandes_pret (pret_id) ON DELETE RESTRICT,
    jours_retard  INTEGER       NOT NULL CHECK (jours_retard >= 0),
    stage_ifrs9   SMALLINT      NOT NULL CHECK (stage_ifrs9 IN (1, 2, 3)),
    ead           NUMERIC(12,2) NOT NULL CHECK (ead >= 0),
    pd_utilisee   NUMERIC(7,6)  NOT NULL CHECK (pd_utilisee BETWEEN 0 AND 1),
    lgd           NUMERIC(4,3)  NOT NULL CHECK (lgd BETWEEN 0 AND 1),
    ecl           NUMERIC(12,2) NOT NULL CHECK (ecl >= 0),
    PRIMARY KEY (date_arrete, pret_id)       -- un prêt, une ligne par arrêté
);
```

Pourquoi `RESTRICT` pour un journal ? Un journal d'audit qui disparaîtrait avec le prêt n'aurait plus aucune valeur de preuve. **Pourquoi pas de clé étrangère vers une table**

de statuts ? On aurait pu créer une petite table des statuts autorisés et la référencer ; ici, les contraintes `CHECK` de `demandes_pret` garantissent déjà que seuls des statuts valides y existent, et le journal ne fait que les recopier.

Correction de l'exercice 1.4

a) Les règles par prêt. On agrège chaque prêt dans une CTE, puis on compte les prêts en anomalie. `IS DISTINCT FROM 0` attrape aussi un CRD final NULL (aucune échéance numéro n).

```
WITH par_pret AS (
    SELECT p.pret_id, p.montant_demande, p.duree_mois,
           sum(e.part_capital)                                                    AS capital_total,
           count(*)                                                               AS nb_echeances,
           max(e.num_echeance)                                                    AS dernier_numero,
           max(e.capital_restant_du) FILTER (WHERE e.num_echeance = p.duree_mois)   AS crd_final,
           count(*) FILTER (WHERE e.montant_du <> e.part_capital + e.part_interets) AS montants_faux
    FROM demandes_pret p
    JOIN echeancier_remboursements e USING (pret_id)
    WHERE p.statut_dossier = 'accorde'
    GROUP BY p.pret_id
)
SELECT count(*)                                                                AS prets_controles,
       count(*) FILTER (WHERE capital_total <> montant_demande)                  AS capital_different,
       count(*) FILTER (WHERE nb_echeances <> duree_mois OR dernier_numero <> duree_mois) AS
    ↪ echeances_manquantes,
       count(*) FILTER (WHERE crd_final IS DISTINCT FROM 0)                      AS crd_final_non_nul,
       sum(montants_faux)                                                        AS montants_faux
FROM par_pret;
```

b) Les règles entre deux échéances : l'auto-jointure. La table apparaît deux fois, sous deux alias : `prec` est l'échéance k, `suiv` l'échéance $k+1$ du même prêt. Grâce à l'index unique `(pret_id, num_echeance)`, la jointure est rapide (0,3 seconde).

```
SELECT count(*) FILTER (WHERE suiv.capital_restant_du <> prec.capital_restant_du - suiv.part_capital) AS
    ↪ ruptures_crd,
       count(*) FILTER (WHERE suiv.date_echeance - prec.date_echeance NOT BETWEEN 28 AND 31)         AS
    ↪ ecarts_non_mensuels
FROM echeancier_remboursements prec
JOIN echeancier_remboursements suiv
  ON suiv.pret_id = prec.pret_id AND suiv.num_echeance = prec.num_echeance + 1;
```

c) Une simple jointure avec un filtre :

```
SELECT count(*) AS paiements_avant_octroi
FROM echeancier_remboursements e
JOIN demandes_pret p USING (pret_id)
WHERE e.date_paiement < p.date_decision;
```

Ce que tu dois obtenir

32 410 prêts contrôlés ; **0** anomalie dans chacune des sept règles.

Une histoire vraie. La première version du corrigé de l'exercice 1.1 arrondissait séparément les intérêts, le capital et le CRD. Ces contrôles ont trouvé **10 ruptures d'un centime**, toutes sur le prêt d'Awa : le CRD d'un mois ne valait plus exactement celui du mois précédent moins la part de capital. D'où la méthode actuelle (la part de capital est la différence de deux CRD arrondis). C'est exactement à cela que servent des contrôles automatiques : trouver les erreurs qu'aucune relecture ne voit.

Chapitre 2

Correction de l'exercice 2.5

```sql
WITH test AS (
    SELECT ps.version_modele, ps.est_champion, ps.probabilite_defaut AS pd, ps.decision, s.en_defaut::int AS
     ↪ defaut
    FROM predictions_scoring_ml ps
    JOIN echantillons_modele em USING (pret_id)
    JOIN v_situation_prets s   USING (pret_id)
    WHERE em.echantillon = 'test'
),
rangs AS (
    -- ex æquo : RANK() donne le premier rang du groupe ; on prend le rang moyen du groupe
    SELECT *, RANK() OVER (PARTITION BY version_modele ORDER BY pd)
              + (count(*) OVER (PARTITION BY version_modele, pd) - 1) / 2.0 AS rang
    FROM test
),
mesures AS (
    SELECT version_modele, est_champion,
           (sum(rang) FILTER (WHERE defaut = 1) - sum(defaut) * (sum(defaut) + 1) / 2.0)
             / (sum(defaut) * sum(1 - defaut))                                    AS auc,
           100.0 * count(*) FILTER (WHERE decision = 'refuser' AND defaut = 1) / sum(defaut) AS rappel_pct,
           100.0 * count(*) FILTER (WHERE decision = 'refuser') / count(*)        AS taux_refus_pct,
           100.0 * count(*) FILTER (WHERE decision = 'accorder' AND defaut = 1)
                 / count(*) FILTER (WHERE decision = 'accorder')                  AS
     ↪ defaut_parmi_accordes_pct
    FROM rangs
    GROUP BY version_modele, est_champion
)
SELECT version_modele, est_champion,
       round(auc, 3)                    AS auc,
       round(2 * auc - 1, 3)            AS gini,
       round(rappel_pct, 1)             AS rappel_pct,
       round(taux_refus_pct, 1)         AS taux_refus_pct,
       round(defaut_parmi_accordes_pct, 1) AS defaut_parmi_accordes_pct
FROM mesures
ORDER BY auc DESC;
```

Les points délicats. Les deux fenêtres de `rangs` n'ont pas la même partition : la première classe les prêts d'un modèle, la seconde compte les ex æquo d'une même PD. Les `2.0` forcent des divisions décimales. `sum(1 - defaut)` compte les prêts sains.

Ce que tu dois obtenir

Modèle	AUC	Gini	Rappel	Refus	Défaut si accordé
`xgboost_monotone_v2` (champion)	**0,923**	**0,847**	76,4 %	**21,9 %**	**6,6 %**
`logistique_v1`	0,876	0,751	77,8 %	29,5 %	6,9 %
`grille_score_v1`	0,875	0,750	77,9 %	29,6 %	6,9 %

Le champion mérite-t-il sa place ? Oui. Les challengers attrapent à peine plus de défauts (77,8 % contre 76,4 %), mais en refusant **7,6 points de clients de plus** (29,5 % contre 21,9 %). Pour un risque un peu plus faible parmi les prêts accordés (6,6 % contre 6,9 %), le XGBoost accepte donc beaucoup plus de bons clients. Le seuil de chaque modèle est un arbitrage métier entre risque et volume : la base permet de le chiffrer.

Correction de l'exercice 2.6

```sql
WITH fins_de_mois AS (
    SELECT (d + INTERVAL '1 month - 1 day')::date AS date_arrete
```

```sql
    FROM generate_series(DATE '2025-09-01', DATE '2026-09-01', INTERVAL '1 month') AS d
),
-- Optimisation : ne garder que les échéances qui peuvent être impayées pendant la fenêtre
-- (échues avant la dernière fin de mois, pas encore soldées à la première). Filtrer AVANT de joindre : 7 s → 3
    ↪ s.
impayes_possibles AS (
    SELECT pret_id, date_echeance, date_paiement
    FROM echeancier_remboursements
    WHERE date_echeance <= (SELECT max(date_arrete) FROM fins_de_mois)
      AND (date_paiement IS NULL OR date_paiement > (SELECT min(date_arrete) FROM fins_de_mois))
),
retards AS (
    SELECT f.date_arrete, p.pret_id,
           (p.date_premiere_echeance + (p.duree_mois - 1) * INTERVAL '1 month')::date AS fin_contrat,
           coalesce(f.date_arrete - min(e.date_echeance), 0)                          AS jours_retard
    FROM fins_de_mois f
    JOIN demandes_pret p ON p.statut_dossier = 'accorde' AND p.date_decision <= f.date_arrete
    -- LEFT JOIN : un prêt sans impayé n'a aucune ligne ici et garde un retard de 0
    LEFT JOIN impayes_possibles e
           ON e.pret_id = p.pret_id
          AND e.date_echeance <= f.date_arrete
          AND (e.date_paiement IS NULL OR e.date_paiement > f.date_arrete)
    GROUP BY f.date_arrete, p.pret_id
),
tranches AS (
    SELECT pret_id, date_arrete,
           CASE WHEN jours_retard = 0  THEN '1. À jour'
                WHEN jours_retard <= 30 THEN '2. 1 à 30 jours'
                WHEN jours_retard <= 60 THEN '3. 31 à 60 jours'
                WHEN jours_retard <= 90 THEN '4. 61 à 90 jours'
                ELSE                         '5. Plus de 90 jours' END AS tranche
    FROM retards
    WHERE jours_retard > 0 OR fin_contrat > date_arrete      -- on écarte les prêts terminés et soldés
),
transitions AS (
    SELECT tranche AS depart,
           LEAD(tranche) OVER (PARTITION BY pret_id ORDER BY date_arrete) AS arrivee   -- tranche le mois
    ↪ suivant
    FROM tranches
)
SELECT depart, count(*) AS nb_prets,
       round(100.0 * count(*) FILTER (WHERE arrivee = '1. À jour')           / count(*), 1) AS vers_a_jour,
       round(100.0 * count(*) FILTER (WHERE arrivee = '2. 1 à 30 jours')     / count(*), 1) AS vers_1_30,
       round(100.0 * count(*) FILTER (WHERE arrivee = '3. 31 à 60 jours')    / count(*), 1) AS vers_31_60,
       round(100.0 * count(*) FILTER (WHERE arrivee = '4. 61 à 90 jours')    / count(*), 1) AS vers_61_90,
       round(100.0 * count(*) FILTER (WHERE arrivee = '5. Plus de 90 jours') / count(*), 1) AS vers_plus_90
FROM transitions
WHERE arrivee IS NOT NULL                                    -- dernier mois observé : pas de mois suivant
GROUP BY depart
ORDER BY depart;
```

La logique. On fabrique un « panel » (prêt × fin de mois), on y calcule le retard *à cette date*, puis `LEAD` met côte à côte la tranche d'un mois et celle du mois suivant. Le `WHERE arrivee IS NOT NULL` écarte le dernier mois, qui n'a pas de suivant.

Ce que tu dois obtenir (en % des prêts de la tranche de départ)

Départ \ mois suivant	Prêts	À jour	1–30	31–60	61–90	> 90
1. À jour	177 587	93,8	6,1	0,1	0,0	0,0
2. 1 à 30 jours	11 674	55,1	5,3	39,3	0,3	0,0
3. 31 à 60 jours	4 862	38,5	2,9	1,7	55,9	1,0
4. 61 à 90 jours	2 965	7,7	1,1	0,3	0,7	90,3
5. Plus de 90 jours	68 213	0,0	0,0	0,0	0,0	100,0

Lecture. Un prêt à jour le reste à 93,8 %. Un prêt à 1–30 jours qui ne régularise pas dans

le mois passe mécaniquement à 31–60 (39,3 %) : un mois de plus, c'est environ 30 jours de plus. Le point décisif est la tranche 61–90 : **90,3 %** de ces prêts finissent en défaut le mois suivant. Le recouvrement doit donc agir dès 31–60 jours, quand 38,5 % des prêts se régularisent encore. Enfin, la tranche « plus de 90 jours » est absorbante (100 %) : dans ces données, aucun prêt en défaut n'a repris ses paiements.

Correction de l'exercice 2.7

```sql
WITH arrete AS (
    SELECT max(valeur_date) FILTER (WHERE cle = 'date_reference')     AS date_ref,
           max(valeur)      FILTER (WHERE cle = 'seuil_defaut_jours') AS seuil
    FROM parametres_risque
),
entrees_defaut AS (
    SELECT e.pret_id, min(e.date_echeance) + (a.seuil::int + 1) AS date_defaut
    FROM echeancier_remboursements e
    CROSS JOIN arrete a
    WHERE e.date_echeance <= a.date_ref
      AND coalesce(e.date_paiement, a.date_ref) - e.date_echeance > a.seuil
    GROUP BY e.pret_id, a.seuil
),
prets AS (
    SELECT p.pret_id, date_trunc('quarter', p.date_decision)::date AS generation,
           p.date_decision, d.date_defaut, a.date_ref
    FROM demandes_pret p
    CROSS JOIN arrete a
    LEFT JOIN entrees_defaut d USING (pret_id)
    WHERE p.statut_dossier = 'accorde'
),
courbes AS (
    SELECT g.generation, h.mois,
           count(*)                                                                          AS nb_observes,
           count(*) FILTER (WHERE g.date_defaut <= g.date_decision + h.mois * INTERVAL '1 month') AS nb_defauts
    FROM prets g
    CROSS JOIN (VALUES (6), (12), (18), (24)) AS h (mois)
    -- seuls les prêts déjà observés pendant h mois comptent, sinon les jeunes générations paraissent saines
    WHERE g.date_decision + h.mois * INTERVAL '1 month' <= g.date_ref
    GROUP BY g.generation, h.mois
),
tableau AS (
    SELECT generation,
           max(nb_observes) FILTER (WHERE mois = 6)                                  AS nb_prets,
           round(100.0 * sum(nb_defauts) FILTER (WHERE mois = 6)  / sum(nb_observes) FILTER (WHERE mois = 6), 1)
      ↪    AS mob_6,
           round(100.0 * sum(nb_defauts) FILTER (WHERE mois = 12) / sum(nb_observes) FILTER (WHERE mois = 12),
      ↪ 1) AS mob_12,
           round(100.0 * sum(nb_defauts) FILTER (WHERE mois = 18) / sum(nb_observes) FILTER (WHERE mois = 18),
      ↪ 1) AS mob_18,
           round(100.0 * sum(nb_defauts) FILTER (WHERE mois = 24) / sum(nb_observes) FILTER (WHERE mois = 24),
      ↪ 1) AS mob_24
    FROM courbes
    GROUP BY generation
)
SELECT *, mob_12 - LAG(mob_12) OVER (ORDER BY generation) AS ecart_mob_12_vs_trimestre_precedent
FROM tableau
ORDER BY generation;
```

Ce que tu dois obtenir (taux de défaut cumulé, en %)

Génération	Prêts	MOB 6	MOB 12	MOB 18	MOB 24	Écart MOB 12
T1 2022	1 820	2,4	8,3	14,0	18,3	
T2 2022	1 894	2,3	7,7	11,9	16,5	−0,6
T3 2022	1 922	1,8	8,5	13,0	17,1	0,8
T4 2022	1 836	1,4	9,0	13,5	18,4	0,5
T1 2023	1 904	2,3	9,7	14,5	18,2	0,7
T2 2023	1 900	2,8	9,4	15,4	19,4	−0,3
T3 2023	1 867	1,8	9,0	14,1	18,7	−0,4
T4 2023	1 879	2,0	9,8	14,6	20,2	0,8
T1 2024	1 883	2,5	10,4	15,9	20,6	0,6
T2 2024	1 843	3,3	10,8	18,0	23,3	0,4
T3 2024	1 839	2,2	10,6	17,9	24,6	−0,2
T4 2024	1 931	2,0	12,4	19,0		1,8
T1 2025	1 819	4,3	14,3	23,5		1,9
T2 2025	1 840	5,0	17,6			3,3
T3 2025	1 825	5,1	23,1			5,5
T4 2025	1 568	2,1				
T1 2026	1 409	0,0				

Lecture, et une leçon importante. Les générations 2022–2023 sont stables (MOB 12 entre 8 et 10 %). À partir de fin 2024, le MOB 12 s'envole jusqu'à 23,1 %. Dans une vraie banque, ce serait un signal d'alerte majeur : la politique d'octroi se serait relâchée. **Ici, c'est un artefact de la simulation** : chaque prêt qui a réellement fait défaut devait être en défaut au 30/09/2026 ; pour les prêts récents, cela oblige le défaut à arriver tôt. Pour la même raison, la génération T1 2026 n'a aucun défaut : les dates des prêts en défaut ont été tirées avant novembre 2025. Réflexe à garder : *avant d'interpréter un signal, se demander s'il vient du phénomène étudié ou de la façon dont les données ont été construites.*

Correction de l'exercice 2.8

a) On classe dans une CTE (avec la part de chaque prêt dans l'encours en défaut de sa note, grâce à une seconde fenêtre), puis on filtre le rang dans la requête englobante.

```sql
WITH classement AS (
    SELECT s.note_risque, s.pret_id, c.nom_complet, s.encours, s.jours_retard,
           RANK() OVER (PARTITION BY s.note_risque ORDER BY s.encours DESC) AS rang,
           round(100 * s.encours / sum(s.encours) OVER (PARTITION BY s.note_risque), 2) AS part_de_la_note_pct
    FROM v_situation_prets s
    JOIN clients c USING (client_id)
    WHERE s.en_defaut
)
SELECT * FROM classement WHERE rang <= 3 ORDER BY note_risque, rang;
```

Extrait (21 lignes au total, noms fictifs)

Note	Client	Prêt	Encours (€)	Retard	Rang	Part de la note
A	Tristan Fischer	13	32 580,44	446	1	0,46 %
A	Adrienne Dias	29141	31 397,09	243	2	0,44 %
A	Daniel Pruvost	32159	31 257,48	1 052	3	0,44 %
			…			
G	Antoine Lopez	14759	23 836,89	470	1	2,92 %
G	Henri Colin	12221	23 692,79	180	2	2,91 %
G	Thomas Robert de la Renard	23159	23 469,55	245	3	2,88 %

b) Le signal précoce. Deux `LAG` sur la même fenêtre nommée, et `ROW_NUMBER` décroissant pour repérer la dernière échéance payée.

```sql
WITH delais AS (
    SELECT pret_id, num_echeance,
           date_paiement - date_echeance                                        AS retard,
           LAG(date_paiement - date_echeance, 1) OVER w                         AS retard_precedent,
           LAG(date_paiement - date_echeance, 2) OVER w                         AS retard_avant_precedent,
           ROW_NUMBER() OVER (PARTITION BY pret_id ORDER BY num_echeance DESC)  AS recence
    FROM echeancier_remboursements
    WHERE date_paiement IS NOT NULL
    WINDOW w AS (PARTITION BY pret_id ORDER BY num_echeance)
)
SELECT d.pret_id, s.note_risque, d.retard_avant_precedent, d.retard_precedent, d.retard, s.jours_retard, s.
    ↪ encours
FROM delais d
JOIN v_situation_prets s USING (pret_id)
WHERE d.recence = 1                                          -- la dernière échéance payée
  AND d.retard > d.retard_precedent AND d.retard_precedent > d.retard_avant_precedent
  AND d.retard > 0
  AND s.jours_retard <= 30
  AND s.encours > 0                                          -- piège : sans ce filtre, on surveille des prêts
    ↪ soldés
ORDER BY d.retard DESC, s.encours DESC
LIMIT 20;
```

Les cinq premiers des 20 prêts signalés

Prêt	Note	Retard $k-2$	Retard $k-1$	Retard k	Retard actuel	Encours (€)
27484	A	−2	−1	40	26	11 230,97
23306	A	−1	37	40	26	1 759,94
31436	C	−3	−2	31	13	11 167,63
15739	C	−2	0	20	0	32 651,38
27395	C	−2	−1	20	0	21 543,39

Pourquoi le filtre de l'ordre d'exécution ? `recence`, `retard_precedent`… sont des résultats de fenêtres : impossible de les filtrer dans le `WHERE` de la CTE qui les calcule. On les filtre dans la requête suivante. **Usage métier :** ces prêts ne sont pas encore en difficulté, mais leurs paiements dérivent ; un appel du conseiller coûte moins cher qu'un recouvrement.

Chapitre 3

Correction de l'exercice 3.4

a) La vérité terrain est chargée dans une table temporaire, qui disparaîtra à la fin de la session. La requête compte, sur les mêmes prêts, les défauts réels, les défauts détectés et les désaccords (<> entre deux booléens = « ou exclusif »).

```
CREATE TEMP TABLE verite_defauts (pret_id BIGINT PRIMARY KEY, defaut_reel SMALLINT NOT NULL);
\copy verite_defauts FROM 'donnees/verite_defauts.csv' WITH (FORMAT csv, HEADER true)
SELECT count(*) FILTER (WHERE v.defaut_reel = 1)                                    AS defauts_reels,
       count(*) FILTER (WHERE d.statut_pret = 'en_defaut')                          AS defauts_detectes,
       count(*) FILTER (WHERE (d.statut_pret = 'en_defaut') <> (v.defaut_reel = 1)) AS desaccords   -- attendu :
    ↪ 0
FROM demandes_pret d
JOIN verite_defauts v USING (pret_id);
```

b) Grâce à la date de référence paramétrée, « faire passer le temps » est une simple mise à jour, annulée à la fin.

```
BEGIN;
UPDATE parametres_risque SET valeur_date = DATE '2027-01-31' WHERE cle = 'date_reference';
SELECT statut_pret AS avant_traitement FROM demandes_pret WHERE pret_id = 32582;      -- « sain » : aucun
    ↪ événement
CALL actualiser_statuts(32582);
SELECT statut_pret AS apres_traitement, fn_jours_retard(32582, fn_date_reference()) AS jours_retard
FROM demandes_pret WHERE pret_id = 32582;                                           -- « en_defaut », 98 jours
SELECT nouveau_statut, date_effet FROM journal_statuts WHERE pret_id = 32582;
ROLLBACK;
```

Ce que tu dois obtenir

a) 7 088 défauts réels, 7 088 détectés, **0 désaccord**.

b) Avant le traitement : `sain` (aucun événement ne s'est produit). Après : `en_defaut`, avec **98 jours** de retard. Le journal contient la ligne `en_defaut`, date d'effet 31/01/2027.

Ce que prouve b). Le passage du temps seul n'a rien déclenché : le statut était encore « sain » avec 98 jours de retard. C'est le traitement planifié qui l'a corrigé, et le trigger de journal a alors tracé le changement et envoyé l'alerte. Les deux mécanismes sont complémentaires.

Correction de l'exercice 3.5

```
CREATE OR REPLACE PROCEDURE cloture_mensuelle(p_date_arrete DATE)
LANGUAGE plpgsql AS $$
DECLARE
    v_nb  INTEGER;
    v_lgd NUMERIC := fn_parametre('lgd');
BEGIN
    IF p_date_arrete > fn_date_reference() THEN
        RAISE EXCEPTION 'Arrêté du % impossible : postérieur à la date de référence (%)',
                        p_date_arrete, fn_date_reference();
    END IF;
    INSERT INTO provisions_mensuelles (date_arrete, pret_id, jours_retard, stage_ifrs9, ead, pd_utilisee, lgd,
     ↪ ecl)
    SELECT p_date_arrete, s.pret_id, s.jours_retard, c.stage, s.encours, c.pd, v_lgd,
           round(c.pd * v_lgd * s.encours, 2)          -- l'ECL se recalcule à partir des colonnes stockées
    FROM fn_situation_prets(p_date_arrete) s
    JOIN demandes_pret d          ON d.pret_id = s.pret_id
    JOIN predictions_scoring_ml ps ON ps.pret_id = s.pret_id AND ps.est_champion
    CROSS JOIN LATERAL (SELECT fn_stage_ifrs9(s.jours_retard) AS stage) st
    CROSS JOIN LATERAL (SELECT st.stage,
                               round(fn_pd_ifrs9(st.stage, ps.probabilite_defaut, s.mois_restants, d.duree_mois)
```

```
      ↪ , 6) AS pd) c
    WHERE s.encours > 0
    ON CONFLICT (date_arrete, pret_id) DO UPDATE          -- rejouer un arrêté remplace, ne duplique pas
        SET jours_retard = EXCLUDED.jours_retard, stage_ifrs9 = EXCLUDED.stage_ifrs9, ead = EXCLUDED.ead,
            pd_utilisee  = EXCLUDED.pd_utilisee,  lgd = EXCLUDED.lgd, ecl = EXCLUDED.ecl;
    GET DIAGNOSTICS v_nb = ROW_COUNT;
    RAISE NOTICE 'Arrêté du % : % prêts provisionnés', p_date_arrete, v_nb;
END $$;

-- Historique : les 12 derniers arrêtés de fin de mois (un CALL ne s'utilise pas dans un SELECT : bloc DO)
DO $$
DECLARE
    v_mois DATE;
BEGIN
    FOR v_mois IN
        SELECT (d + INTERVAL '1 month - 1 day')::date
        FROM generate_series(DATE '2025-10-01', DATE '2026-09-01', INTERVAL '1 month') AS d
    LOOP
        CALL cloture_mensuelle(v_mois);
    END LOOP;
END $$;

-- Évolution des provisions et dotation nette du mois (ce qui pèse sur le résultat de la banque)
WITH mensuel AS (
    SELECT date_arrete,
           sum(ead)                                    AS ead,
           sum(ecl)                                    AS ecl,
           sum(ecl) FILTER (WHERE stage_ifrs9 = 1)     AS ecl_stage1,
           sum(ecl) FILTER (WHERE stage_ifrs9 = 2)     AS ecl_stage2,
           sum(ecl) FILTER (WHERE stage_ifrs9 = 3)     AS ecl_stage3
    FROM provisions_mensuelles
    GROUP BY date_arrete
)
SELECT date_arrete, ead, ecl, ecl_stage1, ecl_stage2, ecl_stage3,
       ecl - LAG(ecl) OVER (ORDER BY date_arrete)      AS dotation_nette,
       round(100 * ecl / ead, 2)                       AS taux_couverture_pct
FROM mensuel
ORDER BY date_arrete;

-- Contrôle : rejouer le dernier arrêté ne change rien (idempotence)
CALL cloture_mensuelle(DATE '2026-09-30');
SELECT count(*) AS lignes_au_30_09, sum(ecl) AS ecl_au_30_09
FROM provisions_mensuelles WHERE date_arrete = DATE '2026-09-30';
```

`CROSS JOIN LATERAL`. Une sous-requête `LATERAL` peut utiliser les colonnes des tables qui la précèdent : la première calcule le stage, la seconde la PD qui dépend du stage. C'est une façon lisible d'enchaîner des calculs ligne par ligne sans répéter les expressions.

Ce que tu dois obtenir (montants en millions d'euros)

Arrêté	EAD	ECL	Stage 1	Stage 2	Stage 3	Dotation nette	Couverture
31/10/2025	147,01	21,54	5,03	1,35	15,16		14,66 %
30/11/2025	146,64	21,94	4,57	1,39	15,97	+0,39	14,96 %
31/12/2025	146,48	22,32	3,95	1,44	16,92	+0,38	15,24 %
31/01/2026	146,26	22,71	3,40	1,51	17,80	+0,40	15,53 %
28/02/2026	145,93	23,13	2,87	1,52	18,75	+0,42	15,85 %
31/03/2026	145,84	23,62	2,31	1,57	19,74	+0,48	16,19 %
30/04/2026	145,79	24,17	1,83	1,46	20,88	+0,55	16,58 %
31/05/2026	145,56	24,71	1,42	1,32	21,97	+0,54	16,97 %
30/06/2026	145,62	25,15	1,02	1,22	22,92	+0,44	17,27 %
31/07/2026	141,94	25,46	0,96	0,59	23,91	+0,31	17,93 %
31/08/2026	137,79	25,67	0,91	0,03	24,74	+0,22	18,63 %
30/09/2026	133,96	25,63	0,86	0,03	24,74	−0,05	19,13 %

Après avoir rejoué le 30/09 : 21 501 lignes, ECL 25 627 130,87 €, inchangés.

Lecture. Chaque mois, de nouveaux prêts passent en stage 3 : la provision augmente, et la **dotation nette** (environ 0,4 million par mois) pèse sur le résultat de la banque. La provision de stage 1 fond de 5,03 à 0,86 million : les prêts qui allaient faire défaut avaient une PD élevée et pesaient lourd en stage 1 ; en passant en stage 3, ils laissent en stage 1 des prêts peu risqués. En septembre, plus aucun nouveau défaut (le stage 3 ne bouge plus) : la dotation devient négative, c'est une **reprise de provision**.

Un écart de 1,72 € ? Le total au 30/09 (25 627 130,87 €) diffère de celui de l'exercice 2.2 (25 627 129,15 €). Ici, la PD est arrondie à 6 décimales et l'ECL au centime *prêt par prêt*, avant la somme. Savoir expliquer un écart d'arrondi, c'est montrer qu'on contrôle ses chiffres.

Correction de l'exercice 3.6

```sql
DO $$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'analyste_risque') THEN
        CREATE ROLE analyste_risque NOLOGIN;
    END IF;
    IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'application_paiement') THEN
        CREATE ROLE application_paiement NOLOGIN;
    END IF;
END $$;

-- L'analyste lit tout, ne modifie rien
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyste_risque;

-- L'application d'encaissement n'écrit PAS dans les tables : elle appelle la procédure, qui s'exécute
-- avec les droits de son propriétaire (SECURITY DEFINER). search_path figé : sinon, un utilisateur
-- pourrait créer une fausse table ou fonction homonyme et détourner la procédure.
ALTER PROCEDURE enregistrer_paiement(BIGINT, NUMERIC, DATE) SECURITY DEFINER SET search_path = public, pg_temp;
REVOKE ALL ON PROCEDURE enregistrer_paiement(BIGINT, NUMERIC, DATE) FROM PUBLIC;
GRANT EXECUTE ON PROCEDURE enregistrer_paiement(BIGINT, NUMERIC, DATE) TO application_paiement;

-- Vérification (transaction annulée)
BEGIN;
SET LOCAL ROLE application_paiement;
CALL enregistrer_paiement(32582, 100);                    -- autorisé : passe par la procédure
DO $$
```

```
BEGIN
    UPDATE echeancier_remboursements SET montant_paye = montant_du WHERE pret_id = 32582;
    RAISE NOTICE 'ANOMALIE : écriture directe acceptée';
EXCEPTION WHEN insufficient_privilege THEN
    RAISE NOTICE 'Écriture directe refusée, comme prévu';
END $$;
ROLLBACK;
```

Ce que tu dois obtenir

`NOTICE: Prêt 32582 : 100 € encaissés le 2026-09-30`, puis `NOTICE: Écriture directe refusée, comme prévu`.

Pourquoi `NOLOGIN` ? Ce sont des *groupes* de droits. Les vrais comptes (l'analyste Paul, le service d'encaissement) seraient des rôles `LOGIN` membres de ces groupes (`GRANT analyste_risque TO paul`). `SET LOCAL ROLE` ne dure que jusqu'à la fin de la transaction : le `ROLLBACK` rend aussi ton rôle d'origine.

Annexe A

Aide-mémoire PostgreSQL

Tout ce que tu as utilisé dans le projet, sur quelques pages, pour réviser la veille d'un entretien.

A.1 Définir les données

```
CREATE TABLE prets (
    pret_id    BIGINT        GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    client_id  BIGINT        NOT NULL REFERENCES clients (client_id) ON DELETE RESTRICT,
    montant    NUMERIC(12,2) NOT NULL CHECK (montant > 0),
    statut     VARCHAR(12)   NOT NULL DEFAULT 'sain' CHECK (statut IN ('sain', 'en_retard', 'en_defaut')),
    CONSTRAINT nom_regle CHECK (...),                    -- contrainte nommée, sur plusieurs colonnes
    UNIQUE (client_id, montant)                          -- unicité composite
);
ALTER TABLE prets ADD CONSTRAINT montant_max CHECK (montant <= 100000);
CREATE INDEX idx_prets_client ON prets (client_id);                      -- clé étrangère : à indexer soi-même
CREATE INDEX idx_impayes ON echeances (pret_id, date_echeance) WHERE montant_paye < montant_du;  -- partiel
DROP TABLE IF EXISTS prets CASCADE;
```

A.2 Transactions

```
BEGIN;  ...  COMMIT;              -- tout ou rien
BEGIN;  ...  ROLLBACK;            -- tester sans rien modifier
SAVEPOINT avant_test;  ...  ROLLBACK TO avant_test;     -- annuler une partie seulement
SELECT ... FROM prets WHERE pret_id = 61 FOR UPDATE;    -- verrouiller une ligne jusqu'à la fin de la transaction
```

A.3 Interroger

```
WITH etape1 AS (...), etape2 AS (SELECT ... FROM etape1) SELECT ... FROM etape2;   -- CTE
count(*) FILTER (WHERE statut = 'en_defaut')               -- agrégat conditionnel
coalesce(x, 0)   nullif(x, 0)   greatest(a, b)   least(a, b)
CASE WHEN ... THEN ... WHEN ... THEN ... ELSE ... END
DATE '2026-09-30' - DATE '2026-08-15'                       -- 46 (jours, entier)
date_debut + (k - 1) * INTERVAL '1 month'                   -- toujours depuis la date de départ
date_trunc('month', d)   date_trunc('quarter', d)   extract(year FROM d)
generate_series(1, 36)   generate_series(DATE '2025-01-01', DATE '2025-12-01', INTERVAL '1 month')
percentile_cont(0.5) WITHIN GROUP (ORDER BY revenu)         -- médiane
GROUP BY ROLLUP (stage)                                     -- sous-totaux et total
x IS DISTINCT FROM y                                        -- comparaison qui traite NULL comme une valeur
NOT EXISTS (SELECT 1 FROM ... WHERE ...)                    -- préférer à NOT IN (piège des NULL)
```

A.4 Fonctions de fenêtre

```
ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY date_demande DESC)      -- 1, 2, 3...
RANK()       OVER (ORDER BY pd DESC)                                      -- 1, 2, 2, 4
DENSE_RANK() OVER (ORDER BY pd DESC)                                      -- 1, 2, 2, 3
NTILE(10)    OVER (ORDER BY pd DESC)                                      -- déciles
LAG(x, 1, 0) OVER (PARTITION BY pret_id ORDER BY num_echeance)            -- ligne précédente (0 au bord)
```

```sql
LEAD(x)      OVER (PARTITION BY pret_id ORDER BY date_arrete)             -- ligne suivante
sum(x) OVER (ORDER BY mois ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)  -- cumul
avg(x) OVER (ORDER BY mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)       -- moyenne mobile
100 * x / sum(x) OVER ()                                                   -- part du total
sum(sum(x)) OVER ()                                                        -- fenêtre sur un agrégat
... WINDOW w AS (PARTITION BY pret_id ORDER BY num_echeance)               -- fenêtre nommée
-- Filtrer sur une fenêtre : CTE, puis WHERE dans la requête suivante (pas de QUALIFY)
```

A.5 PL/pgSQL

```sql
CREATE OR REPLACE FUNCTION f(p_x INTEGER) RETURNS NUMERIC
LANGUAGE plpgsql STABLE AS $$
DECLARE v NUMERIC;
BEGIN
    SELECT ... INTO STRICT v FROM ... WHERE ...;          -- exactement une ligne, sinon erreur
    IF v IS NULL THEN RAISE EXCEPTION 'Message %', p_x USING HINT = '...'; END IF;
    RETURN v;
END $$;

CREATE OR REPLACE FUNCTION fn_trg() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    -- NEW, OLD ; BEFORE : RETURN NEW (ou NULL pour ignorer la ligne) ; AFTER : RETURN NULL
    RETURN NEW;
END $$;
CREATE TRIGGER trg BEFORE UPDATE ON t FOR EACH ROW EXECUTE FUNCTION fn_trg();
CREATE TRIGGER trg2 AFTER UPDATE ON t REFERENCING OLD TABLE AS a NEW TABLE AS n
    FOR EACH STATEMENT EXECUTE FUNCTION fn_trg2();

CREATE OR REPLACE PROCEDURE p(p_id BIGINT DEFAULT NULL) LANGUAGE plpgsql AS $$
DECLARE r RECORD; v_nb INTEGER;
BEGIN
    FOR r IN SELECT ... LOOP ... END LOOP;
    UPDATE ...;  GET DIAGNOSTICS v_nb = ROW_COUNT;
    RAISE NOTICE '% lignes', v_nb;
END $$;
CALL p();          DO $$ BEGIN ... END $$;              -- appel ; bloc anonyme

INSERT INTO t (...) SELECT ... ON CONFLICT (cle) DO UPDATE SET col = EXCLUDED.col;   -- upsert
SELECT pg_notify('canal', 'message');   LISTEN canal;                                -- alertes
GRANT SELECT ON ALL TABLES IN SCHEMA public TO role;   REVOKE ALL ON PROCEDURE p FROM PUBLIC;
```

A.6 Commandes psql et diagnostic

`psql -d base -f fichier.sql`	exécuter un script (ajouter `-v ON_ERROR_STOP=1`)
`\l` , `\c base`	lister les bases, se connecter à une base
`\dt` , `\dv` , `\df`	lister les tables, les vues, les fonctions
`\d nom_table`	structure d'une table : colonnes, index, contraintes, triggers
`\x` , `\timing`	affichage vertical ; durée de chaque requête
`\copy t FROM 'f.csv' CSV HEADER`	importer un CSV depuis le poste client
`EXPLAIN ANALYZE requete`	plan d'exécution et temps réels
`ANALYZE`	mettre à jour les statistiques de l'optimiseur

Annexe B

Glossaire du risque de crédit

AUC Aire sous la courbe ROC : probabilité qu'un prêt en défaut ait un score plus risqué qu'un prêt sain. 0,5 = hasard, 1 = parfait.

Calibration Qualité d'un modèle dont les probabilités prédites correspondent aux fréquences observées (une PD de 10 % ⇒ environ 10 % de défauts).

Capital restant dû (CRD) Part du capital emprunté pas encore remboursée.

Champion / challenger Le modèle qui décide en production / les modèles concurrents évalués sur les mêmes prêts.

Courbe de gains Part cumulée des défauts captés en refusant les clients les plus risqués, décile par décile.

CRR Règlement européen sur les exigences de fonds propres des banques (accords de Bâle). Son article 178 définit le défaut.

Défaut Retard de plus de 90 jours sur une obligation significative, ou paiement jugé improbable.

Dotation nette Variation des provisions sur une période ; elle pèse sur le résultat de la banque (positive : charge ; négative : reprise).

DPD *Days Past Due* : nombre de jours depuis l'échéance impayée la plus ancienne.

EAD *Exposure At Default* : montant dû au moment du défaut ; ici, le capital restant dû.

ECL *Expected Credit Loss* : perte attendue IFRS 9, $PD \times LGD \times EAD$.

Échéance (mensualité) Somme due à une date donnée : part de capital + part d'intérêts.

Encours Total du capital restant dû d'un portefeuille.

Gini $2 \times AUC - 1$; indicateur de discrimination usuel en scoring de crédit.

IFRS 9 Norme comptable qui impose de provisionner les pertes attendues dès l'octroi, en trois stages.

LGD *Loss Given Default* : part de l'exposition perdue en cas de défaut (45 % ici).

MOB *Months On Book* : ancienneté d'un prêt en mois, utilisée dans l'analyse par génération.

PARx *Portfolio At Risk* : part de l'encours portée par les prêts en retard de plus de x jours.

PD *Probability of Default* : probabilité de défaut, prédite par le modèle de scoring.

Période probatoire Délai minimal (au moins trois mois) pendant lequel un prêt sorti d'impayé reste classé en défaut.

Provision Montant mis de côté pour couvrir une perte attendue.

Rappel Part des défauts réels que le modèle aurait refusés.

Roll rate Taux de passage d'une tranche de retard à une autre d'un mois sur l'autre (matrice

de transition).

Seuil de décision PD au-delà de laquelle le modèle recommande le refus.

SHAP Méthode qui décompose une prédiction en contributions de chaque variable.

Stage (IFRS 9) 1 : sain (PD à 12 mois) ; 2 : risque dégradé, plus de 30 jours de retard (PD sur la durée restante) ; 3 : défaut (PD = 100 %).

Taux de couverture Provisions rapportées à l'exposition.

Taux d'effort Part du revenu consacrée au remboursement ; ici, le montant emprunté rapporté au revenu annuel.

Vintage (génération) Ensemble des prêts accordés sur une même période, suivis dans le temps.

Annexe C

Questions d'entretien

Des questions réellement posées en entretien de stage data, avec une réponse courte. Entraîne-toi à répondre à voix haute, puis à écrire la requête sans regarder.

C.1 SQL

Le prêt le plus récent de chaque client ? `ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY date_demande DESC)` dans une CTE, puis `WHERE rn = 1`. (PostgreSQL offre aussi `DISTINCT ON (client_id)`.)

Les adresses e-mail en double ? Un regroupement filtré : `SELECT email FROM clients GROUP BY email HAVING count(*) > 1`.

COUNT(*), COUNT(col), COUNT(DISTINCT col) ? Toutes les lignes ; les lignes où `col` n'est pas NULL ; les valeurs distinctes non NULL.

Le piège de NOT IN ? Si la sous-requête renvoie un seul NULL, `x NOT IN (...)` ne renvoie aucune ligne (la comparaison avec NULL est inconnue). Utilise `NOT EXISTS`.

Le piège du LEFT JOIN ? Une condition sur la table de droite placée dans le `WHERE` élimine les lignes sans correspondance : le `LEFT JOIN` redevient un `INNER JOIN`. Place-la dans le `ON`.

UNION ou UNION ALL ? `UNION` supprime les doublons (donc trie, plus lent) ; `UNION ALL` garde tout.

Une médiane ? `percentile_cont(0.5) WITHIN GROUP (ORDER BY revenu_annuel)` : 55 000 € dans ta base.

Une requête est lente : que fais-tu ? `EXPLAIN ANALYZE` pour voir le plan ; un index sur les colonnes de filtre et de jointure ; filtrer avant de joindre ; éviter les fonctions sur les colonnes indexées ; `ANALYZE` si les statistiques sont anciennes.

C.2 Modélisation et ingénierie

Pourquoi ne pas stocker le retard ? Il change chaque jour sans événement : stocké, il serait faux dès le lendemain. On le calcule à partir des dates.

Comment historiser une donnée qui change ? Ne jamais l'écraser : un journal (comme `journal_statuts`), un registre daté (comme les prédictions), ou une table avec des dates de validité (« dimension à évolution lente de type 2 » dans un entrepôt de données).

Vue ou vue matérialisée ? La vue est recalculée à chaque lecture (toujours à jour) ; la vue matérialisée stocke le résultat (rapide, mais à rafraîchir).

Comment charger un million de lignes ? Avec `COPY` (ou `to_sql` avec une méthode `COPY`), dans une transaction, puis `ANALYZE`. Jamais un million d'`INSERT` séparés.

Comment éviter un double paiement ? Contraintes (`montant_paye <= montant_du`), verrou sur le prêt (`FOR UPDATE`), et en pratique un identifiant de transaction unique pour rendre l'encaissement idempotent.

C.3 Risque de crédit

Quand un prêt est-il en défaut ? Plus de 90 jours de retard sur une obligation significative, ou paiement improbable (CRR, article 178).

Qu'est-ce que l'ECL ? La perte attendue IFRS 9, $PD \times LGD \times EAD$, avec une PD à 12 mois en stage 1, sur la durée restante en stage 2, et 100 % en stage 3.

Pourquoi une LGD de 45 % ? C'est la valeur réglementaire de l'approche « fondation » de Bâle pour une créance non garantie de premier rang. Une banque en approche avancée l'estime sur ses propres recouvrements.

À quoi sert une matrice de transition ? À voir la dynamique du risque : ici, 90 % des prêts à 61–90 jours passent en défaut le mois suivant, donc le recouvrement doit agir plus tôt.

Pourquoi analyser par génération ? Pour comparer des prêts au même âge, et détecter qu'une politique d'octroi s'est dégradée, à condition de n'utiliser que des prêts observés assez longtemps.

C.4 Machine Learning et MLOps

Comment surveiller un modèle en production ? Sa discrimination (AUC sur les nouveaux prêts dont on connaît l'issue), sa calibration (PD prédite contre défaut observé par décile), la dérive des données (indice de stabilité de la population, PSI) et ses effets métier (taux de refus).

Pourquoi l'échantillon de test seulement ? Sur l'échantillon d'apprentissage, le modèle a déjà vu les réponses : ses performances y sont surestimées.

À quoi sert SHAP dans le crédit ? À expliquer chaque décision (obligation envers le client et le régulateur), à vérifier que le modèle s'appuie sur des variables légitimes, et à détecter des biais.

Pourquoi un XGBoost « monotone » ? Des contraintes de monotonie garantissent, par exemple, que le risque prédit ne baisse jamais quand le taux d'effort augmente : le modèle reste cohérent avec l'intuition métier, donc acceptable par la validation et le régulateur.

Quand remplacer le champion ? Quand un challenger fait mieux sur les mêmes prêts, sur des données récentes et hors apprentissage, à risque égal ou plus faible, et que l'écart est stable dans le temps. Le registre des prédictions permet exactement cette comparaison.